
Towards Docking-oriented De Novo Ligand Design via Gradient Inversion

Zekai Chen¹ Xunkai Li¹ Sirui Zhang¹ Henan Sun² Jia Li² Qiangqiang Dai¹ Hongchao Qin¹ Zhenjun Li³
Bing Zhou³ Rong-Hua Li¹ Guoren Wang¹

Abstract

De novo ligand design is a fundamental task that seeks to generate protein or molecule candidates that can effectively dock with protein receptors and achieve strong binding affinity entirely from scratch. It holds paramount significance for a wide spectrum of biomedical applications. However, most existing studies are constrained by the Pseudo De Novo, Limited Docking Modeling, and Inflexible Ligand Type. To address these issues, we propose MagicDock, a forward-looking framework grounded in the progressive pipeline and differentiable surface modeling. (1) We adopt a well-designed gradient inversion framework. To begin with, general docking knowledge of receptors and ligands is incorporated into the backbone model. Subsequently, the docking knowledge is instantiated as reverse gradient flows by binding prediction, which iteratively guide the *de novo* generation of ligands. (2) We emphasize differentiable surface modeling in the *generation process*, leveraging learnable 3D point-cloud representations to precisely capture docking details, thereby ensuring that the generated ligands preserve docking validity through interpretable spatial fingerprints. (3) We introduce customized designs for different ligand types and integrate them into a unified gradient inversion framework with flexible triggers, thereby ensuring broad applicability. Moreover, we provide sufficient theoretical guarantees for MagicDock. Extensive experiments across 9 scenarios demonstrate that MagicDock achieves average improvements of 7.0% and 7.4% over SOTA baselines specialized for protein or molecule ligand design, respectively.

^{*}Equal contribution ¹Beijing Institute of Technology, Beijing, China ²The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China ³Shenzhen City Polytechnic, Shenzhen, China. Correspondence to: Rong-Hua Li <lironghuabit@126.com>.

1. Introduction

De novo ligand design aims to create ligands (e.g., proteins and small molecules) that bind a target receptor with high affinity and form stable complexes, enabling broad applications in drug discovery and bioengineering. Traditionally, ligand design has relied on energy optimization techniques (Adolf-Bryfogle et al., 2018). Recent advances in deep learning have introduced data-driven generative models with improved expressiveness (Evans et al., 2021; Gu et al., 2024). Despite progress, existing methods still face inherent limitations (Fig. 1).

(1) Pseudo De Novo. Many recent “de novo” frameworks are only *pseudo* de novo: they rely on strong priors and then perform local refinement. For example, several state-of-the-art antibody methods keep the framework and most CDR loops fixed, or restrict CDR-H3 to a small template set, and then run generative refinement or energy-based optimization on top (Zhou et al., 2024). Such priors can yield high success rates but sacrifice the ability to design ligands truly from scratch. In contrast, we target a *template-/retrieval-free de novo* setting, where only the receptor structure is given and the full ligand must be designed without retrieving conserved templates or copying training-set ligands.¹

Beyond genuine de novo design, two additional issues hinder practical progress. **(2) Limited Docking Modeling.** Many methods incorporate docking knowledge only indirectly—e.g., through post-hoc scoring/selection, surrogate energies, or supervision signals learned from docking labels (Zhou et al., 2024; Schneuing et al., 2025). However, the *generation process* itself is often decoupled from docking geometry: intermediate states are not explicitly constrained to maintain surface complementarity or interaction-consistent contact patterns, and docking signals do not continuously shape the *generation trajectory*. This makes it difficult to tightly couple docking semantics with step-by-step structural updates, and limits interpretability of refinement signals grounded in receptor–ligand geometry. **(3) Inflexible Ligand Type.** Most existing approaches are

¹We consider universal feasibility constraints (e.g., valence/steric checks for molecules, or geometry sanity checks for proteins) permissible, since they enforce basic validity rather than injecting task-specific templates or motifs.

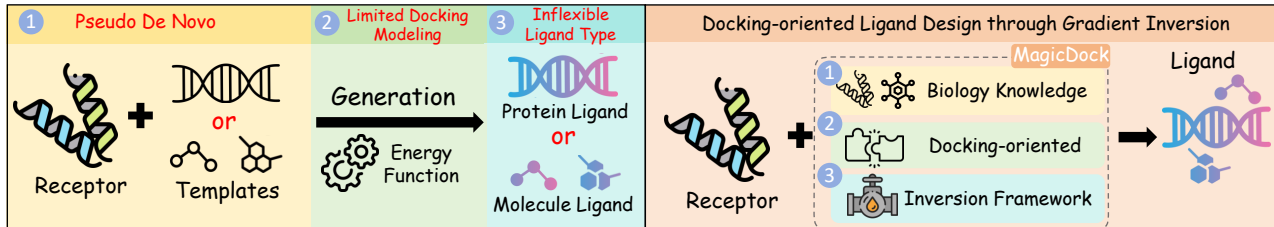


Figure 1. Comparison between current works and MagicDock. This figure describes the three limitations of the existing methods and presents MagicDock, which achieves authentic de novo, biological significance and cross-category generality.

narrowly tailored to specific ligand types like protein or molecule (Luo et al., 2022; Guo et al., 2021), limiting versatility across diverse molecular categories. Collectively, these challenges hinder the development of a comprehensive and generalizable ligand design framework.

To address these issues, we propose **MagicDock**, built around three core innovations. (1) We instantiate a **gradient-based inversion formulation on a docking-aware surface backbone**. The backbone first learns generic molecular knowledge from large protein and small-molecule datasets via unsupervised pre-training, and is then specialized for docking by fine-tuning on three progressive tasks. Both generic and docking-specific knowledge are compactly encoded in a single surface-based backbone, which is reused at test time to provide gradients that steer de novo ligand design. (2) We emphasize **generation-time docking modeling via gradient-driven surface refinement on learnable 3D point clouds**, so that docking geometry and surface complementarity are explicitly captured and continuously enforced along the refinement trajectory. Gradients are taken with respect to a continuous surface state, while chemical/structural validity is enforced by type-specific feasibility constraints. This yields interpretable, geometry-grounded refinement signals and improves biophysical complementarity to the receptor. (3) We design **customized modules for different ligand types**, seamlessly integrated into a unified inversion framework with flexible triggering mechanisms. Importantly, the validity and efficiency of our method are supported by theoretical guarantees in Sec. 4.

Our contributions. (1) *New Perspective*: We introduce docking-oriented inversion for template-/retrieval-free de novo ligand design, targeting genuine de novo generation, docking coupling, and cross-category generality. (2) *New Framework*: We propose a surface-based inversion pipeline that injects docking knowledge into a pretrained backbone and performs constrained gradient refinement directly in the receptor pocket. (3) *New Method*: We develop a differentiable surface state for stable gradient flow, coupled with feasibility constraints to ensure valid generation across ligand types. (4) *SOTA Performance*: Across 9 scenarios, **MagicDock** achieves state-of-the-art performance,

with over 7.0% average improvement in protein ligand design and over 7.4% in molecule ligand design compared with strong baselines.

2. Preliminaries & Related Works

2.1. Notations and Problem Formulation

We adopt a docking-oriented strategy, as surface point clouds capture fine-grained structural cues for molecular recognition and docking data provide binding compatibility. Both protein and small-molecule ligands are represented as 3D surface point clouds $P(S) = \{f_i\}_{i=1}^{N(S)}$, where $N(S)$ denotes the number of sampled surface points for a given structure S and $f_i \in \mathbb{R}^d$ encodes chemical, atomic, and geometric features. In particular, during Stage 4 ligand generation the cardinality $N(S_t)$ is allowed to change with the current number of ligand atoms at iteration t , so the ligand and point-cloud size is not fixed. Ligand generation solves docking energy minimization by iteratively refining a type-specific ligand state with G_{type} under a pre-trained model M_θ :

$$\hat{\mathcal{P}}_{\text{lig}} = \lim_{t \rightarrow T} G_{\text{type}}(\mathcal{P}_{\text{lig}}^{(t)}, M_\theta) \quad (1)$$

$$\mathcal{P}_{\text{lig}}^* = \arg \min_{\hat{\mathcal{P}}_{\text{lig}}} E_{\text{dock}}(\mathcal{P}_{\text{rec}}, \hat{\mathcal{P}}_{\text{lig}}), \quad (2)$$

where t denotes the refinement step. G_{type} incorporates domain-specific generating constraints, emphasizing ring and valency rules for small molecules and residue/conformation features for proteins.

2.2. Related Works

Existing methods for protein and molecular ligand design can be categorized by how they *couple generation with optimization*, i.e., the extent to which candidates are refined toward biochemical objectives. We summarize them into four paradigms:

① **Decoupled Paradigms**. Traditional pipelines treat representation, generation, and optimization as separate modules. Classical docking methods such as ZDOCK (Chen et al., 2010), RosettaDock (Lyskov & Gray, 2008), AutoDock (Morris et al., 2008), and Glide (Halgren et al., 2004) encode ligands into atomic/residue descriptors, followed by

independent search and scoring. Early generative models like sequence-based RNNs (Liu et al., 2020) also rely on post hoc optimization. These approaches are interpretable but loosely connected to task objectives.

② **Implicitly Coupled Paradigms.** Diffusion-based approaches embed optimization into sampling. For proteins, DiffAb (Luo et al., 2022) and HSRN (Jin et al., 2022) refine CDR loops with SE(3)-equivariant models. For ligands, DiffDock (Corso et al., 2022) and GeoDiff (Xu et al., 2022) integrate docking or ΔG signals into denoising, while Pocket2Mol (Peng et al., 2022) and TankBind (Lu et al., 2022) further condition on receptor pockets. Although effective, these rely on handcrafted schedules and stochastic trajectories, limiting efficiency and de novo completeness.

③ **Surrogate- or Heuristic-coupled Paradigms.** Another line combines generation with heuristic optimization or surrogate models. For proteins, reinforcement learning (Zhou et al., 2024) and memory-augmented models like dyMEAN (Kong et al., 2023) incorporate docking rewards. For ligands, reinforcement learning (Gottipati et al., 2020), evolutionary strategies (Chen et al., 2021), and Bayesian optimization (Moss et al., 2020) guide fragment assembly or mutation. Frameworks such as DockStream (Guo et al., 2021), ALID-IFF (Gu et al., 2024), and DRUGFLOW (Schneuing et al., 2025) embed chemical, geometric, or physical priors. These methods are flexible but computationally costly.

④ **Latent-gradient coupling Paradigm (Ours).** Inversion differs by explicitly treating generation as optimization (Niu et al., 2025; Qiu et al., 2024; Bergues et al., 2025). Structures are refined via gradient updates in a continuous surrogate space under task losses and constraints, without a diffusion-style stochastic *noise schedule*. Details for Sec. 2.2 are provided in Appendix B.

2.3. The Inversion Framework

Overview. Inversion treats generation as *gradient-based refinement* of structured variables, in contrast to diffusion (Ho et al., 2020; Song et al., 2022) or flow matching (Lipman et al., 2023) which typically rely on stochastic trajectories or learned transport dynamics. In MAGICDOCK, we instantiate inversion for docking-oriented *de novo* ligand design: a pre-trained, docking-aware backbone induces a differentiable objective over a *continuous* ligand state, and generation is carried out by constraint-aware refinement in this state space.

Why inversion. Our choice is driven by docking-specific constraints rather than optimization as a generic tool. Compared to sampling-based generators, inversion is (i) *general*—it only requires a differentiable objective w.r.t. a continuous design state, while chemical validity can be enforced by projection/repair operators that need not be differentiable; (ii) *efficient*—it turns a fixed test-time budget into

descent-driven refinement, with any non-monotonicity explicitly attributable to inexact constraint handling (cf. the repair residual in Appendix E); and (iii) *modular*—the refinement loop is largely orthogonal to the backbone, so stronger pretrained representations immediately translate into stronger docking guidance. Practically, this yields a unified *generation–optimization* loop in a continuous surrogate space with periodic *re-anchoring* to feasible ligands: after several refinement steps, the current state is mapped back to a valid ligand and re-encoded, preventing drift and refreshing docking features/gradients for the next cycle. Classifier-guided diffusion or stochastic optimal control can in principle incorporate optimization, but they additionally rely on a calibrated likelihood prior and its inference machinery; our focus is complementary—*constraint-aware, type-agnostic refinement driven directly by docking gradients*, while learned generators (if available) can be used as initializers. Concretely, we realize inversion with two stages: **(I) learning a docking-informative gradient field**, and **(II) performing constrained refinement on the ligand state**.

3. Methodology

We instantiate the four-stage framework introduced in Fig. 2 and propose MagicDock: **Stage1:** Docking-oriented ligand modeling, **Stage2:** Unsupervised pre-training, **Stage3:** Supervised fine-tuning, and **Stage4:** Inversion-based ligand generation. These modules constitute a de novo pipeline for receptor–ligand interactions. The pseudocode of our method is applied in Appendix G.

3.1. Stage 1: Docking-oriented Ligand Modeling

Motivation. Docking is driven by local surface complementarity. We model receptor–ligand interactions on solvent-accessible surfaces and represent both partners as multi-resolution surface point clouds with geometric and physico-chemical features. Gradients reach atomic coordinates by unrolling the SDF refinement; for FPS/KNN patching we stop-gradient the discrete indices and backpropagate through gathered coordinates/features, supporting Stage 2 pre-training and later inversion; SE(3) equivariance is analyzed in Appendix E.1.

Surface point cloud modeling. Following (Sverrisson et al., 2021), we convert atomic structures to solvent-accessible surfaces sampled as point clouds. The surface is defined as a level set of a smooth signed distance function (SDF). Candidate points $\{x_s^i\}$ are sampled around atom coordinates $\{x_a^j\}$ and refined by a few unrolled gradient-descent steps:

$$\text{SDF}(x_s^i) = -f(x_s^i) \cdot \log \sum_{j=1}^N \exp\left(-\frac{\|x_s^i - x_a^j\|}{\sigma_a^j}\right) \quad (3)$$

$$f(x_s^i) = \frac{\sum_{j=1}^N \exp(-\|x_s^i - x_a^j\|) \sigma_a^j}{\sum_{j=1}^N \exp(-\|x_s^i - x_a^j\|)} \quad (4)$$

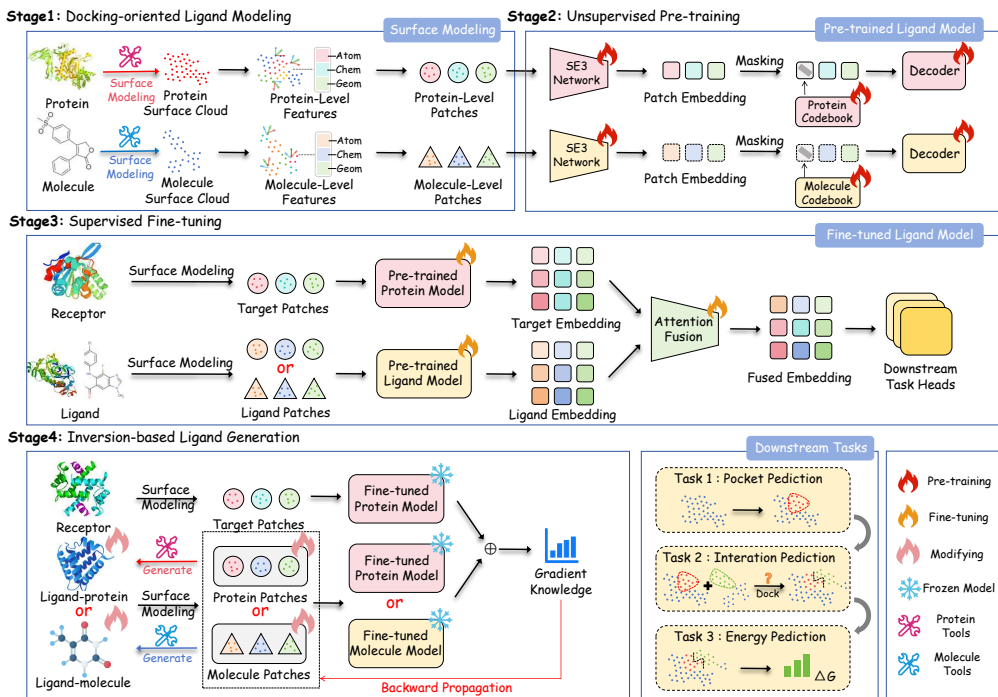


Figure 2. The overview of MagicDock framework.

Here N is the number of atoms and σ_a^j is the atomic radius. We use multi-resolution point clouds for receptors and ligands to trade off fidelity and efficiency.

Feature generation. Each surface point x_i is assigned a composite feature vector that integrates chemical, atomic, and geometric cues:

$$f(x_i) = \text{concat}(f_{\text{chem}}(x_i), f_{\text{atom}}(x_i), f_{\text{geom}}(x_i)). \quad (5)$$

These features encode both intrinsic properties and local neighborhood context, yielding a unified, docking-relevant representation for proteins and ligands.

Patch partitioning. To reduce complexity and enable patch-level pre-training in Stage 2, we partition point clouds into local patches following (Wu & Li, 2024). Patch centers X_c are obtained by farthest point sampling (FPS), and each center aggregates its K nearest surface points via KNN:

$$X_c = \text{FPS}(X_s), \quad X_c \in \mathbb{R}^{\rho M \times 3}, \quad (6)$$

$$X_p = \text{KNN}(X_c, X_s), \quad X_p \in \mathbb{R}^{\rho M \times K \times 3}. \quad (7)$$

This patch-based structure preserves local chemical–atomic–geometric context while supporting efficient and scalable pre-training with discrete latent codes at the patch level. Overall, Stage 1 provides a differentiable, docking-oriented surface representation that unifies receptors and ligands and serves as the input interface for subsequent stages.

3.2. Stage 2: Unsupervised Pre-training

Motivation. Docking supervision is scarce and target-specific, while protein and small-molecule structures are abundant. We therefore pre-train a *single* surface backbone on large unlabeled corpora, so that it learns transferable geometric/chemical primitives on solvent-accessible surfaces, yielding (i) providing docking-relevant surface semantics that generalize across receptors and ligands, and (ii) producing *gradient-effective* representations that will later guide Stage 3 inversion. The stage is plug-and-play: any surface-level self-supervised learner mapping patches to differentiable embeddings can be used; we instantiate it with Surface-VQMAE (Wu & Li, 2024)(Appendix C.2).

SE(3)-Equivariant Encoding. Given local surface patches X_p with coordinates $\mathbf{x}_i \in \mathbb{R}^3$ and features f_i , the encoder applies SE(3)-equivariant convolutions to obtain pointwise embeddings:

$$z_i = \sum_{j \in \mathcal{N}(i)} \sum_{l=0}^L R_l(\|\mathbf{x}_{ij}\|) Y_l\left(\frac{\mathbf{x}_{ij}}{\|\mathbf{x}_{ij}\|}\right) \cdot W_l f_j, \quad (8)$$

where $\mathbf{x}_{ij} = \mathbf{x}_j - \mathbf{x}_i$, $R_l(\cdot)$ are radial functions, $Y_l(\cdot)$ are spherical harmonics, and W_l are weight matrices, guaranteeing SE(3) equivariance. Point embeddings are aggregated into patch tokens H_p for subsequent masking and decoding.

Masked Reconstruction with Vector Quantization. We mask patches at ratio $\delta = 50\%$, forming masked and visible

sets $X_{p,m} \in \mathbb{R}^{\delta \rho M \times K \times 3}$ and $X_{p,vis} \in \mathbb{R}^{(1-\delta)\rho M \times K \times 3}$. Masked patch tokens are discretized by a learnable codebook (via Gumbel-Softmax relaxation), and the decoder reconstructs (i) spatial coordinates:

$$\hat{X} = \text{Reshape}(\text{MLP}(H_p^{(L_2)})), \quad \hat{X} \in \mathbb{R}^{\delta \rho M \times K \times 3}, \quad (9)$$

and (ii) curvature-related signals to regularize local shape. Following (Wu & Li, 2024), we compute patch covariance eigenvalues $\{\epsilon_i\}_{i=1}^3$ and define pseudo-curvatures $\psi_i = \epsilon_i / \sum_{j=1}^3 \epsilon_j$, which are predicted by an MLP. Curvature supervision encourages faithful local geometry, which is critical for surface complementarity in docking.

Training Objective. The overall loss combines coordinate reconstruction, curvature prediction, and KL regularization:

$$\mathcal{L} = \nu_1 \mathcal{L}_{\text{rec}}(X_{p,m}, \hat{X}) + \nu_2 \mathcal{L}_{\text{cur}}(\psi, \hat{\psi}) + \nu_3 \mathcal{L}_{\text{KL}}(q, p), \quad (10)$$

where $\mathcal{L}_{\text{rec}}$ uses Chamfer distance and $\mathcal{L}_{\text{cur}}$ uses RMSE. We denote $q := q(Z_{p,m} | H_{p,m})$ as the variational posterior over masked latents conditioned on masked embeddings, and $p := p(Z_{p,m})$ as the corresponding prior. Stage 2 outputs an SE(3)-equivariant surface backbone with transferable patch semantics, which is reused in Stage 3 to provide stable and informative gradients for inversion.

3.3. Stage 3: Supervised Fine-tuning

Motivation. Stage 2 learns transferable surface primitives but lacks explicit docking semantics. Stage 3 therefore fine-tunes the backbone with docking-aligned supervision: pocket localization, interface prediction, and binding free-energy regression, providing interpretable intermediates and improving label efficiency.

Equivariant Attention Fusion. We reuse the Stage 2 SE(3)-equivariant encoder to produce latent fields $Z_r \in \mathbb{R}^{N_r \times d}$ and $Z_l \in \mathbb{R}^{N_l \times d}$, and model receptor–ligand coupling via equivariant cross-attention and a permutation-invariant aggregator:

$$\text{Attn}(Z_r, Z_l) = \text{softmax}\left(\frac{Q_r^{(\ell=0)}(K_l^{(\ell=0)})^\top}{\sqrt{d}}\right) V_l \quad (11)$$

$$\tilde{z} = \mathcal{A}(Z_r, \text{Attn}(Z_r, Z_l)) \in \mathbb{R}^{2d}. \quad (12)$$

Here $Q_r^{(\ell=0)} = Z_r^{(\ell=0)} W_Q$, $K_l^{(\ell=0)} = Z_l^{(\ell=0)} W_K$, and $V_l = Z_l W_V$, with $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$. We compute attention weights from the scalar ($\ell = 0$) channels to preserve equivariance.

Multi-Task Progressive Supervision. We supervise Stage 3 progressively: *pocket identification* $\rightarrow$ *interaction prediction* $\rightarrow$ *affinity estimation*. Pocket scores gate receptor embeddings, and the fused representation is used for interaction prediction and ΔG regression. The fine-tuning objective is

$$\mathcal{L}_{\text{FT}} = \alpha \mathcal{L}_{\text{pocket}} + \beta \mathcal{L}_{\text{int}} + \mathcal{L}_{\Delta G}, \quad (13)$$

with $\alpha, \beta > 0$ tuned on validation data; details are in Appendix C.3.

3.4. Stage 4: Inversion-based Ligand Generation

Motivation. Stage 3 turns the backbone into a docking-oriented differentiable evaluator on a fully differentiable surface representation. Rather than training a separate likelihood-based generator, we cast *de novo* design as constrained optimization in the ligand’s continuous space. At inference, we refine the ligand state by projected gradient descent on a scalar objective $\mathcal{F}$ built from Stage 3 docking predictions and optional soft penalties, enabling flexible goal/constraint composition without retraining (Appendix A, C.4).

Composite objective and projected updates. Given receptor R and ligand state S_t , Stage 3 produces Z_r, Z_l and fuses them into $\tilde{z}$ (Eq. 11), which defines an inference-time objective $\mathcal{F}(S_t; R, \Theta)$. We parameterize the ligand by continuous coordinates and features $(\mathbf{x}, f)$, and apply projected updates:

$$(\mathbf{x}, f)^{t+1} = \Pi_{\mathcal{C}_{\text{valid}}}((\mathbf{x}, f)^t - \eta_t \nabla_{(\mathbf{x}, f)} \mathcal{F}(S_t; R, \Theta)). \quad (14)$$

Here $\Pi_{\mathcal{C}_{\text{valid}}}$ repairs/projects the state to a valid set $\mathcal{C}_{\text{valid}}$ (e.g., basic chemical/geometric feasibility) and need not be differentiable; decoding to a discrete ligand is performed after refinement (Appendix A).

Type-specific decoding and constraints. After refinement, the continuous state $(\mathbf{x}, f)^t$ is decoded to an atomistic graph by a type-specific generator $\mathcal{G}_{\text{type}}$, which shares a unified interface across ligand families while allowing family-specific priors and validity constraints. For small molecules, per-atom features f_i parameterize logits over atom types $\mathcal{A}_{\text{atom}}$, and we sample with a gradient-biased proposal:

$$a_i \sim \text{Categorical}(\sigma(f_i - \gamma \nabla_{f_i} \mathcal{F})), \quad a_i \in \mathcal{A}_{\text{atom}},$$

with $\gamma = 0.1$ to steer sampling toward types that decrease $\mathcal{F}$ while preserving diversity; remaining violations are handled by standard validity repair (Appendix C.4). For protein ligands, the same interface operates at the residue level:

$$r_i \sim \text{Categorical}(\sigma(f_i - \gamma \nabla_{f_i} \mathcal{F})), \quad r_i \in \mathcal{A}_{20},$$

and conformational degrees of freedom are updated under geometric/steric feasibility constraints. Optionally, we accept local edits via a Metropolis-style rule based on the predicted change in binding free energy to improve exploration.

$$S^* = \lim_{t \rightarrow T_{\text{gen}}} \mathcal{G}_{\text{type}}(\Pi_{\mathcal{C}_{\text{valid}}}((\mathbf{x}, f)^t - \eta_t \nabla_{(\mathbf{x}, f)} \mathcal{F}(S_t; R, \Theta))), \quad (15)$$

where T_{gen} is a fixed budget or chosen adaptively by a convergence criterion, and $\Pi_{\mathcal{C}_{\text{valid}}}$ keeps intermediate states within valid regions throughout inversion.

Table 1. Performance comparison on the SKEMPI v2 (left) and SAbDab (right). We report mean $\pm$ standard deviation. Best results are in **bold**, second best are underlined. Arrows indicate whether higher ($\uparrow$) or lower ($\downarrow$) values are better.

Method	IMP ($\uparrow$)	STA ($\uparrow$)	DIV ($\uparrow$)	NOV ($\uparrow$)	AAR ($\uparrow$)
RABD	21.58 $\pm$ 0.06 / 15.12 $\pm$ 0.05	0.779 $\pm$ 0.002 / 0.763 $\pm$ 0.002	0.728 $\pm$ 0.003 / 0.764 $\pm$ 0.003	0.838 $\pm$ 0.002 / 0.829 $\pm$ 0.004	38.23 $\pm$ 0.13 / 38.75 $\pm$ 0.12
DiffAB	17.65 $\pm$ 0.03 / 11.69 $\pm$ 0.05	0.703 $\pm$ 0.002 / 0.736 $\pm$ 0.002	0.744 $\pm$ 0.001 / 0.754 $\pm$ 0.003	0.820 $\pm$ 0.002 / 0.868 $\pm$ 0.003	41.65 $\pm$ 0.15 / 38.52 $\pm$ 0.13
HSRN	23.19 $\pm$ 0.12 / 17.72 $\pm$ 0.08	0.844 $\pm$ 0.003 / 0.809 $\pm$ 0.002	0.801 $\pm$ 0.002 / 0.805 $\pm$ 0.002	0.877 $\pm$ 0.003 / 0.924 $\pm$ 0.004	45.29 $\pm$ 0.26 / 40.63 $\pm$ 0.43
dyMEAN	15.93 $\pm$ 0.05 / 8.55 $\pm$ 0.04	0.821 $\pm$ 0.002 / 0.799 $\pm$ 0.003	0.805 $\pm$ 0.003 / 0.799 $\pm$ 0.004	<u>0.923</u> $\pm$ 0.006 / <u>0.953</u> $\pm$ 0.005	44.67 $\pm$ 0.31 / 44.06 $\pm$ 0.34
ABDPO	25.17 $\pm$ 0.07 / 16.63 $\pm$ 0.05	0.853 $\pm$ 0.005 / <u>0.833</u> $\pm$ 0.006	0.796 $\pm$ 0.004 / 0.816 $\pm$ 0.005	0.905 $\pm$ 0.005 / 0.932 $\pm$ 0.005	46.19 $\pm$ 0.45 / 43.53 $\pm$ 0.40
chai-1	27.35 $\pm$ 0.04 / 17.70 $\pm$ 0.02	0.851 $\pm$ 0.005 / 0.823 $\pm$ 0.003	0.803 $\pm$ 0.002 / 0.813 $\pm$ 0.005	0.909 $\pm$ 0.005 / 0.935 $\pm$ 0.005	46.21 $\pm$ 0.25 / 43.86 $\pm$ 0.25
RoseTTAFold-AA	28.46 $\pm$ 0.03 / 19.01 $\pm$ 0.03	0.846 $\pm$ 0.002 / 0.827 $\pm$ 0.002	0.807 $\pm$ 0.004 / 0.818 $\pm$ 0.003	0.912 $\pm$ 0.002 / 0.940 $\pm$ 0.003	46.23 $\pm$ 0.20 / 43.72 $\pm$ 0.21
Abx	<u>28.76</u> $\pm$ 0.10 / <u>21.80</u> $\pm$ 0.09	<u>0.866</u> $\pm$ 0.006 / 0.820 $\pm$ 0.005	<u>0.812</u> $\pm$ 0.005 / <u>0.820</u> $\pm$ 0.006	0.889 $\pm$ 0.005 / 0.950 $\pm$ 0.004	<u>46.35</u> $\pm$ 0.34 / <u>44.29</u> $\pm$ 0.31
Ours	36.32 $\pm$ 0.09 / 27.87 $\pm$ 0.07	0.874 $\pm$ 0.001 / 0.851 $\pm$ 0.002	0.815 $\pm$ 0.002 / 0.824 $\pm$ 0.003	0.934 $\pm$ 0.003 / 0.957 $\pm$ 0.002	48.73 $\pm$ 0.21 / 46.14 $\pm$ 0.25

Table 2. Performance comparison on the PDBBind2020 (left) and CrossDocked2020 (right). We report mean $\pm$ standard deviation. Best results are in **bold**, second best are underlined. Arrows indicate whether higher ($\uparrow$) or lower ($\downarrow$) values are better.

Method	Vina ($\downarrow$)	Affinity ($\uparrow$)	STA ($\uparrow$)	DIV ($\uparrow$)	SA ($\uparrow$)	QED ($\uparrow$)
DockStream	-5.51 $\pm$ 0.03 / -5.15 $\pm$ 0.04	15.13 $\pm$ 0.14 / 17.86 $\pm$ 0.16	0.765 $\pm$ 0.011 / 0.780 $\pm$ 0.007	0.717 $\pm$ 0.009 / 0.621 $\pm$ 0.014	0.39 $\pm$ 0.02 / 0.37 $\pm$ 0.03	0.455 $\pm$ 0.002 / 0.401 $\pm$ 0.003
3D-SBDD	-6.35 $\pm$ 0.03 / -6.24 $\pm$ 0.05	27.88 $\pm$ 0.19 / 28.54 $\pm$ 0.17	0.853 $\pm$ 0.008 / 0.801 $\pm$ 0.006	0.768 $\pm$ 0.003 / 0.701 $\pm$ 0.004	0.51 $\pm$ 0.03 / 0.48 $\pm$ 0.04	0.503 $\pm$ 0.002 / 0.483 $\pm$ 0.002
liGAN	-6.03 $\pm$ 0.02 / -6.11 $\pm$ 0.03	22.56 $\pm$ 0.13 / 22.15 $\pm$ 0.11	0.830 $\pm$ 0.010 / 0.825 $\pm$ 0.009	0.772 $\pm$ 0.002 / 0.663 $\pm$ 0.001	0.59 $\pm$ 0.04 / 0.54 $\pm$ 0.03	0.489 $\pm$ 0.003 / 0.377 $\pm$ 0.002
ALIDIFF	<u>-7.21</u> $\pm$ 0.03 / -6.79 $\pm$ 0.02	<u>35.74</u> $\pm$ 0.15 / 65.63 $\pm$ 0.09	0.866 $\pm$ 0.004 / 0.833 $\pm$ 0.006	0.756 $\pm$ 0.003 / <u>0.727</u> $\pm$ 0.003	0.57 $\pm$ 0.05 / 0.55 $\pm$ 0.03	0.472 $\pm$ 0.004 / 0.464 $\pm$ 0.002
DRUGFLOW	-7.12 $\pm$ 0.02 / -6.81 $\pm$ 0.04	33.56 $\pm$ 0.11 / 52.35 $\pm$ 0.13	0.858 $\pm$ 0.002 / 0.838 $\pm$ 0.005	0.764 $\pm$ 0.003 / 0.718 $\pm$ 0.002	0.57 $\pm$ 0.02 / 0.55 $\pm$ 0.02	<u>0.520</u> $\pm$ 0.003 / <u>0.519</u> $\pm$ 0.002
DIFSBDD	-6.99 $\pm$ 0.02 / -6.86 $\pm$ 0.03	28.83 $\pm$ 0.07 / 35.88 $\pm$ 0.13	<u>0.867</u> $\pm$ 0.003 / 0.825 $\pm$ 0.002	0.801 $\pm$ 0.002 / 0.705 $\pm$ 0.003	0.59 $\pm$ 0.04 / 0.57 $\pm$ 0.02	0.511 $\pm$ 0.003 / 0.502 $\pm$ 0.003
Ours	-7.36 $\pm$ 0.02 / -7.02 $\pm$ 0.03	40.63 $\pm$ 0.06 / <u>62.72</u> $\pm$ 0.18	0.875 $\pm$ 0.009 / 0.840 $\pm$ 0.003	<u>0.778</u> $\pm$ 0.003 / 0.730 $\pm$ 0.003	0.69 $\pm$ 0.03 / 0.67 $\pm$ 0.04	0.552 $\pm$ 0.003 / 0.544 $\pm$ 0.002

4. Theoretical Analysis

We summarize the theoretical guarantees of MAGICDOCK and defer all formal statements to Appendix E. In Appendix E, we prove SE(3) equivariance properties (exact for Stages 1–3 and approximate under validity repair for Stage 4), establish convergence of the repaired projected updates, and provide additional results on budgeted superiority and complexity (iteration and labeled-sample).

5. Experiments

To validate the distinct superiority of MagicDock, we conduct a series of rigorous and comprehensive experiments on diverse datasets for both molecule and protein ligand. We aim to answer: **Q1** (Effectiveness): Does MagicDock outperform state-of-the-art baselines in generating ligand? **Q2** (Interpretability): What enables MagicDock to effectively produce high-affinity ligands? **Q3** (Robustness): Is MagicDock resilient to structural noise, biological variability, and does it exhibit reliable convergence? **Q4** (Efficiency): Does MagicDock achieve superior trade-offs in runtime, resource usage, scalability, and data efficiency?

5.1. Overall Performance (Q1)

To answer **Q1**, we conducted a systematic and complete evaluation of the effectiveness of ligand design for two scenarios. We list the detailed experimental settings for these two scenarios in the section.

Datasets & Baselines & Evaluation Metrics. We use task-specific datasets, baselines, and evaluation metrics for

protein and small-molecule ligand design; full details are in Appendix D. Stages 2–3 rely on standard benchmark splits only to train the backbone and docking heads, while all de novo ligand design experiments are run on held-out targets or scaffolds that never appear as supervised labels. All baselines are fairly adapted to this de novo setting (Appendix H), and MagicDock does not use hand-crafted priors (e.g., fixed CDR templates), generating ligands entirely from scratch.

Experimental Results. As shown in Table 1 and Table 2, MagicDock outperforms all baselines on protein benchmarks, consistently attaining the highest STA scores, and most baselines on molecular benchmarks, capturing better conformational strain and synthetic accessibility. MagicDock’s affinity is slightly lower than AliDiff’s, as AliDiff directly aligns energies via preference optimization on pre-trained diffusion models, biasing toward high-affinity samples at the cost of diversity and substantial computational overhead. Hyperparameter details are provided in Appendix M.

5.2. Interpretability Study (Q2)

Having solved Q1, we now turn to Q2 and investigate why MagicDock can produce high-affinity ligands. Specifically, we first analyzed the contribution of each stage to the final result and whether the gradient of the inversion target provided meaningful biological localization signals. In addition, we also visualized the process of ligand generation.

Ablation Study. To disentangle the role of each stage, we conduct module-wise ablations: (i) *w/o Stage 1*: replacing surface point clouds with a capacity-matched SE(3)-

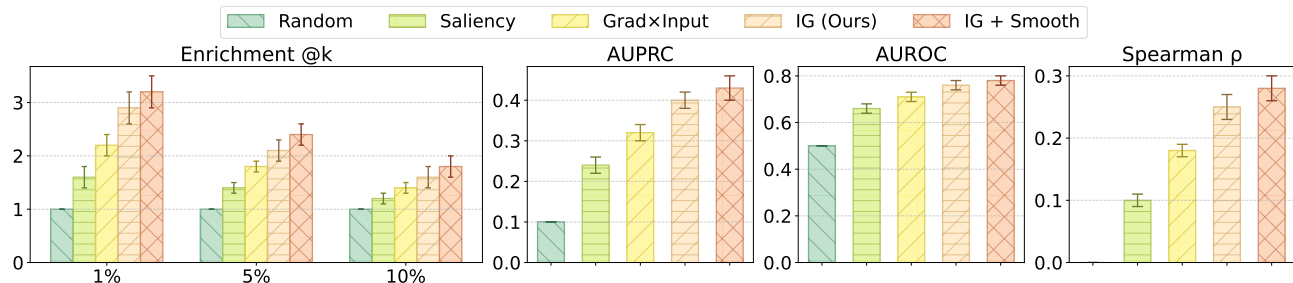


Figure 3. Performance comparison of attribution methods for interpretability, evaluated on Enrichment @k (1%, 5%, 10%), AUPRC, AUROC, and Spearman correlation.

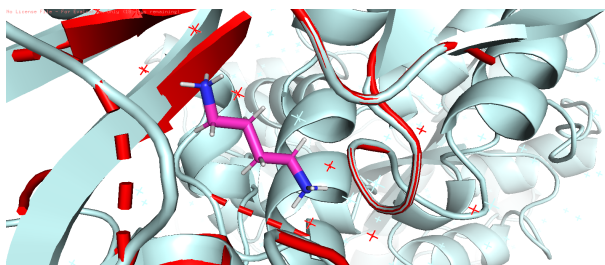


Figure 4. Visualization of an example of generated protein-ligand complexes.

equivariant atom-graph encoder; (ii) *w/o Stage 2*: removing unsupervised pre-training; (iii) *w/o Stage 3*: disabling supervised fine-tuning; (iv) *w/o Stage 4*: substituting gradient-guided inversion with a budgeted gradient-informed element-type search. Fig. 12a, 12b show that each stage plays a critical role, with the full pipeline achieving the best performance.

Gradient Attribution & Localization. Further, we have examined whether gradients from the composite inversion objective $\mathcal{F}(S; R, \Theta)$ (Appendix C.4) localize on biologically meaningful binding sites learned in Stage 3. We compute Integrated Gradients on surfaces and evaluate how high-attribution regions align with ground-truth interfaces. Fig. 3 shows that inversion not only improves affinity but also highlights mechanistically relevant regions, addressing Q2.

Local Perturbation Consistency. We test whether gradient attributions predict energetic effects of local edits by introducing small perturbations near high-attribution sites and comparing predicted versus actual $\Delta\mathcal{F}$. This evaluates whether attributions enable actionable refinement. As shown in Fig. 7, our method outperforms saliency and Grad×Input and closely approaches physics-based Rosetta evaluations.

Case Study. To illustrate MagicDock’s inversion-based ligand generation, we generate two ligands for a target receptor and track binding affinity at each iteration (Fig. 5 and Fig. 14). Additional molecular ligands produced by Magic-

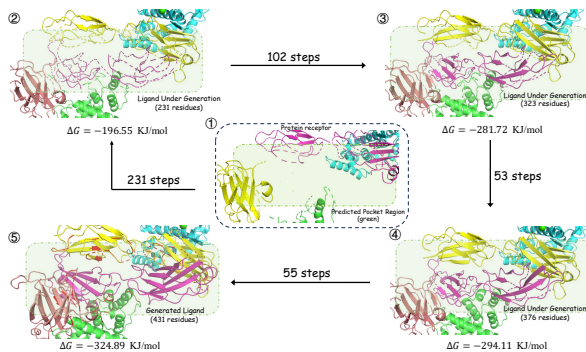


Figure 5. An example of generated complexes.

Dock are visualized in Fig. 4 and Fig. 15 in Appendix L.

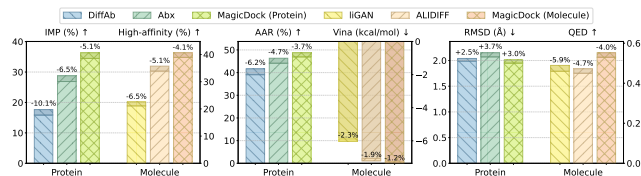
5.3. Robustness Study (Q3)

Having established interpretability, we next address Q3: whether MagicDock is robust to input variations and biological uncertainty. We design four experiments probing artificial noise, realistic receptor perturbations, convergence, and hyperparameter sensitivity.

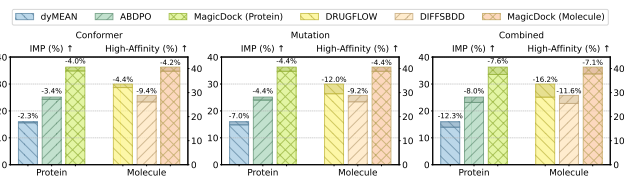
Geometric and Feature Noise. We assess robustness on 100 receptor–ligand pairs by adding Gaussian coordinate noise and feature dropout to receptor surfaces, then comparing ligand generation on noisy versus clean inputs (Fig. 6a). MagicDock consistently outperforms baselines under these perturbations, demonstrating stable performance under structural and feature noise.

Conformational and Mutational Variability. We further perturb 100 receptors with conformer ensembles, single-point mutations, and combined variants, and process all perturbed structures through the inversion pipeline. Performance degradation is quantified via IMP for protein baselines and High-Affinity for molecule baselines. Statistical comparisons between conservative and non-conservative mutations (Fig. 6b) show that MagicDock remains robust to realistic conformational and mutational variability.

Convergence Study. The convergence of MagicDock’s



(a) Robustness under geometric and feature noise, evaluated on IMP/High-affinity (%), AAR/Vina (kcal/mol), and RMSD/QED respectively on protein and molecular baselines.



(b) Robustness to conformational and mutational variability.

Figure 6. Robustness evaluation under multiple perturbations.

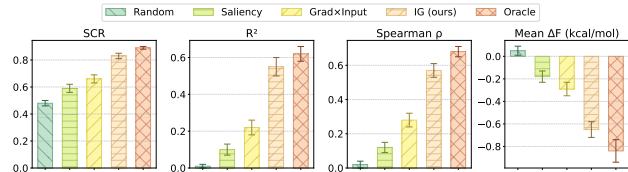


Figure 7. Evaluation of local perturbation consistency, showing SCR, R², Spearman correlation, and Mean ΔF (kcal/mol) for attribution methods.

gradient-driven inversion stage is analyzed in Appendix E.2, where we show that under standard assumptions for projected gradient descent the ligand sequence P_{lig}^t converges to a stationary point of the composite objective $\mathcal{F}$. This provides a theoretical guarantee for reliable optimization.

Hyperparameter Sensitivity. We also examine robustness to loss-weight choices in Stage 3. Figure 8(a) sweeps the pocket-loss weight λ_{pocket} and shows at most $\sim 1\%$ relative variation around the optimum across all datasets, indicating limited sensitivity. Figure 8(b) visualizes IMP over a grid of interaction and affinity weights (α, β), revealing a broad basin around $\alpha = 5, \beta = 50$ and further supporting robustness to moderate hyperparameter misspecification.

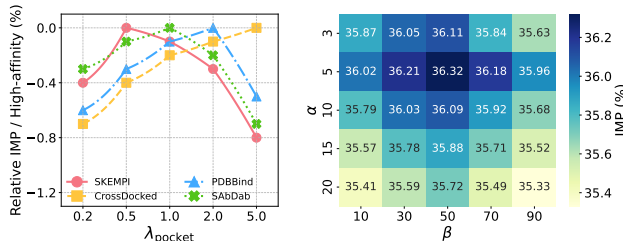


Figure 8. Hyperparameter sensitivity of Stage-3 loss weights. (a) Relative IMP / High-Affinity change versus pocket-loss weight λ_{pocket} . (b) IMP over a grid of interaction and affinity weights (α, β) on SKEMPI, with a broad optimum around $\alpha = 5, \beta = 50$.

5.4. Efficiency Study (Q4)

Finally, we address Q4 by evaluating whether MAGICDOCK achieves a favorable trade-off between efficiency and accuracy. Having demonstrated its interpretability and robust-

ness, we now analyze runtime efficiency, scalability, and data efficiency.

Runtime and Resource Usage. We benchmark MagicDock against baselines on 100 receptors, generating one ligand per receptor under identical hardware and measuring wall-clock time and memory. Fig. 13a shows reduced runtime and Fig. 13b lower memory, underscoring resource efficiency in practical deployment settings.

Scalability. Scalability is evaluated across nine settings varying receptor size and ligand complexity using samples from CrossDocked and SAbDab. Metrics include per-iteration latency, iterations-to-converge, memory footprint, and the scaling exponent γ from $T \propto N^\gamma$. Table 5 and Table 6 show sub-quadratic scaling, indicating scalability.

6. Conclusion

We presented MAGICDOCK, an inversion-based framework that unifies *docking-aware* generation and optimization for template-/retrieval-free *de novo* ligand design in a single, end-to-end workflow. By operating on a continuous surface point-cloud state with SE(3)-equivariant representations, and refining ligands via constrained, gradient-driven updates, MAGICDOCK couples spatial surface complementarity to the generation trajectory while accommodating non-differentiable chemical feasibility through projection/repair steps. Across protein and small-molecule settings, experiments demonstrate consistent improvements in docking quality and practical inference efficiency, supporting inversion as a viable alternative to purely sampling-based pipelines, even under strict validity constraints and pocket shifts. Limitations and future directions are discussed in Appendix F.

Acknowledgement

This work was supported by New Generation Artificial Intelligence-National Science and Technology Major Project (2025ZD0123403). This work was partially supported by the NSFC Grants U24A20255, U2241211, 625B2021 and 62427808. This work was partially supported by Guangdong Provincial Ordinary Universities' Special

Innovation Program (2024KTSCX260). This work was also supported by Guangdong Provincial Talent Program (No.2024TQ08X366).

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Abbasi, M., Santos, B. P., Pereira, T. C., Sofia, R., Monteiro, N. R., Simões, C. J., Brito, R. M., Ribeiro, B., Oliveira, J. L., and Arrais, J. P. Designing optimized drug candidates with generative adversarial network. *Journal of cheminformatics*, 14(1):40, 2022.
- Adolf-Bryfogle, J., Kalyuzhnyi, O., Kubitz, M., Weitzner, B. D., Hu, X., Adachi, Y., Schief, W. R., and Dunbrack Jr, R. L. Rosettaantibodydesign (rabd): A general framework for computational antibody design. *PLoS computational biology*, 14(4):e1006112, 2018.
- Bengio, E., Jain, M., Korablyov, M., Precup, D., and Bengio, Y. Flow network based generative models for non-iterative diverse candidate generation. *Advances in neural information processing systems*, 34:27381–27394, 2021.
- Berdigaliyev, N. and Aljofan, M. An overview of drug discovery and development. *Future medicinal chemistry*, 12(10):939–947, 2020.
- Bergues, N., Carré, A., Join-Lambert, P., Hoffmann, B., Blondel, A., and Tajmouati, H. Template-guided 3d molecular pose generation via flow matching and differentiable optimization. *arXiv preprint arXiv:2506.06305*, 2025.
- Blum, C. and Roli, A. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM computing surveys (CSUR)*, 35(3):268–308, 2003.
- Boitreaud, J., Dent, J., McPartlon, M., Meier, J., Reis, V., Rogozhnikov, A., and Wu, K. Chai-1: Decoding the molecular interactions of life. *BioRxiv*, 2024.
- Brown, N., Fiscato, M., Segler, M. H., and Vaucher, A. C. Guacamol: benchmarking models for de novo molecular design. *Journal of chemical information and modeling*, 59(3):1096–1108, 2019.
- Cao, Y., Das, P., Chenthamarakshan, V., Chen, P.-Y., Melnyk, I., and Shen, Y. Fold2seq: A joint sequence (1d)-fold (3d) embedding-based generative model for protein design. In *International Conference on Machine Learning*, pp. 1261–1271. PMLR, 2021.
- Chen, B., Wang, T., Li, C., Dai, H., and Song, L. Molecule optimization by explainable evolution. In *International conference on learning representation (ICLR)*, 2021.
- Chen, R., Li, L., and Weng, Z. Zdock: an initial-stage protein-docking algorithm. *Proteins-structure Function & Bioinformatics*, 52(1):80–87, 2010.
- Corso, G., Stärk, H., Jing, B., Barzilay, R., and Jaakkola, T. Diffdock: Diffusion steps, twists, and turns for molecular docking. *arXiv preprint arXiv:2210.01776*, 2022.
- De Cao, N. and Kipf, T. Molgan: An implicit generative model for small molecular graphs. *arXiv preprint arXiv:1805.11973*, 2018.
- Dominguez, C., Boelens, R., and Bonvin, A. M. Haddock: a protein-protein docking approach based on biochemical or biophysical information. *Journal of the American Chemical Society*, 125(7):1731–1737, 2003.
- Evans, R., O’Neill, M., Pritzel, A., Antropova, N., Senior, A., Green, T., Židek, A., Bates, R., Blackwell, S., Yim, J., et al. Protein complex prediction with alphafold-multimer. *bioRxiv*, pp. 2021–10, 2021.
- Ewing, T. J., Makino, S., Skillman, A. G., and Kuntz, I. D. Dock 4.0: search strategies for automated molecular docking of flexible molecule databases. *Journal of computer-aided molecular design*, 15(5):411–428, 2001.
- Fabricant, D. S. and Farnsworth, N. R. The value of plants used in traditional medicine for drug discovery. *Environmental health perspectives*, 109(suppl 1):69–75, 2001.
- Francoeur, P. G., Masuda, T., Sunseri, J., Jia, A., Iovanisci, R. B., Snyder, I., and Koes, D. R. Three-dimensional convolutional neural networks and a cross-docked data set for structure-based drug design. *Journal of chemical information and modeling*, 60(9):4200–4215, 2020.
- Fu, T., Xiao, C., and Sun, J. Core: Automatic molecule optimization using copy & refine strategy. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 638–645, 2020.
- Fu, T., Gao, W., Xiao, C., Yasonik, J., Coley, C. W., and Sun, J. Differentiable scaffolding tree for molecular optimization. *arXiv preprint arXiv:2109.10469*, 2021a.
- Fu, T., Xiao, C., Li, X., Glass, L. M., and Sun, J. Mimoso: Multi-constraint molecule sampling for molecule optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 125–133, 2021b.
- Ganea, O.-E., Huang, X., Bunne, C., Bian, Y., Barzilay, R., Jaakkola, T., and Krause, A. Independent se (3)-equivariant models for end-to-end rigid protein docking. *arXiv preprint arXiv:2111.07786*, 2021.

- Gao, Z., Feng, T., You, J., Zi, C., Zhou, Y., Zhang, C., and Li, J. Deep reinforcement learning for modelling protein complexes. *arXiv preprint arXiv:2405.02299*, 2024a.
- Gao, Z., Liu, Z., Li, Y., and Li, J. Towards stable representations for protein interface prediction. *Advances in Neural Information Processing Systems*, 37:73079–73097, 2024b.
- Gómez-Bombarelli, R., Wei, J. N., Duvenaud, D., Hernández-Lobato, J. M., Sánchez-Lengeling, B., Sheberla, D., Aguilera-Iparraguirre, J., Hirzel, T. D., Adams, R. P., and Aspuru-Guzik, A. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018. doi: 10.1021/acscentsci.7b00572.
- Gottipati, S. K., Sattarov, B., Niu, S., Pathak, Y., Wei, H., Liu, S., Blackburn, S., Thomas, K., Coley, C., Tang, J., et al. Learning to navigate the synthetically accessible chemical space using reinforcement learning. In *International conference on machine learning*, pp. 3668–3679. PMLR, 2020.
- Griffiths, R.-R. and Hernández-Lobato, J. M. Constrained bayesian optimization for automatic chemical design using variational autoencoders. *Chemical science*, 11(2): 577–586, 2020.
- Gu, S., Xu, M., Powers, A., Nie, W., Geffner, T., Kreis, K., Leskovec, J., Vahdat, A., and Ermon, S. Aligning target-aware molecule diffusion models with exact energy optimization. *Advances in Neural Information Processing Systems*, 37:44040–44063, 2024.
- Guimaraes, G. L., Sanchez-Lengeling, B., Outeiral, C., Farias, P. L. C., and Aspuru-Guzik, A. Objective-reinforced generative adversarial networks (organ) for sequence generation models. *arXiv preprint arXiv:1705.10843*, 2017.
- Guo, J., Janet, J. P., Bauer, M. R., Nittinger, E., Giblin, K. A., Papadopoulos, K., Voronov, A., Patronov, A., Engkvist, O., and Margreitter, C. Dockstream: a docking wrapper to enhance de novo molecular design. *Journal of cheminformatics*, 13(1):89, 2021.
- Halgren, T. A., Murphy, R. B., Friesner, R. A., Beard, H. S., Frye, L. L., Pollard, W. T., and Banks, J. L. Glide: a new approach for rapid, accurate docking and scoring. 2. enrichment factors in database screening. *Journal of medicinal chemistry*, 47(7):1750–1759, 2004.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models, 2020. URL <https://arxiv.org/abs/2006.11239>.
- Huang, K., Fu, T., Gao, W., Zhao, Y., Roohani, Y., Leskovec, J., Coley, C. W., Xiao, C., Sun, J., and Zitnik, M. Therapeutics data commons: Machine learning datasets and tasks for drug discovery and development. *arXiv preprint arXiv:2102.09548*, 2021.
- Ingraham, J., Garg, V., Barzilay, R., and Jaakkola, T. Generative models for graph-based protein design. *Advances in neural information processing systems*, 32, 2019.
- Jain, M., Raparthy, S. C., Hernández-García, A., Rector-Brooks, J., Bengio, Y., Miret, S., and Bengio, E. Multi-objective GFlowNets. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 14631–14653. PMLR, 2023.
- Jensen, J. H. A graph-based genetic algorithm and generative model/monte carlo tree search for the exploration of chemical space. *Chemical science*, 10(12):3567–3572, 2019.
- Jin, W., Wohlwend, J., Barzilay, R., and Jaakkola, T. Iterative refinement graph neural network for antibody sequence-structure co-design. *arXiv preprint arXiv:2110.04624*, 2021.
- Jin, W., Barzilay, R., and Jaakkola, T. Antibody-antigen docking and design via hierarchical structure refinement. In *International Conference on Machine Learning*, pp. 10217–10227. PMLR, 2022.
- Kong, X., Huang, W., and Liu, Y. End-to-end full-atom antibody design. *arXiv preprint arXiv:2302.00203*, 2023.
- Korovina, K., Xu, S., Kandasamy, K., Neiswanger, W., Poczos, B., Schneider, J., and Xing, E. Chembo: Bayesian optimization of small organic molecules with synthesizable recommendations. In *International Conference on Artificial Intelligence and Statistics*, pp. 3393–3403. PMLR, 2020.
- Kozakov, D., Hall, D. R., Xia, B., Porter, K. A., Padhorny, D., Yueh, C., Beglov, D., and Vajda, S. The cluspro web server for protein–protein docking. *Nature protocols*, 12(2):255–278, 2017.
- Krishna, R., Wang, J., Ahern, W., Sturmfels, P., Venkatesh, P., Kalvet, I., Lee, G. R., Morey-Burrows, F. S., Anishchenko, I., Humphreys, I. R., et al. Generalized biomolecular modeling and design with rosettafold all-atom. *Science*, 384(6693):ead12528, 2024.
- Lee, S., Jo, J., and Hwang, S. J. Exploring chemical space with score-based out-of-distribution generation. In *International Conference on Machine Learning*, pp. 18872–18892. PMLR, 2023.

- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling, 2023. URL <https://arxiv.org/abs/2210.02747>.
- Liu, G., Zeng, H., Mueller, J., Carter, B., Wang, Z., Schilz, J., Horny, G., Birnbaum, M. E., Ewert, S., and Gifford, D. K. Antibody complementarity determining region design using high-capacity machine learning. *Bioinformatics*, 36(7):2126–2133, 2020.
- Liu, H., Chen, P., Zhai, X., Huo, K.-G., Zhou, S., Han, L., and Fan, G. Ppb-affinity: Protein-protein binding affinity dataset for ai-based protein drug discovery. *Scientific data*, 11(1):1316, 2024.
- Liu, Q., Allamanis, M., Brockschmidt, M., and Gaunt, A. Constrained graph variational autoencoders for molecule design. *Advances in neural information processing systems*, 31, 2018.
- Lu, W., Wu, Q., Zhang, J., Rao, J., Li, C., and Zheng, S. Tankbind: Trigonometry-aware neural networks for drug-protein binding structure prediction. *Advances in neural information processing systems*, 35:7236–7249, 2022.
- Luo, S., Guan, J., Ma, J., and Peng, J. A 3d generative model for structure-based drug design. In *Thirty-Fifth Conference on Neural Information Processing Systems*, 2021.
- Luo, S., Su, Y., Peng, X., Wang, S., Peng, J., and Ma, J. Antigen-specific antibody design and optimization with diffusion-based generative models for protein structures. *Advances in Neural Information Processing Systems*, 35: 9754–9767, 2022.
- Lyskov, S. and Gray, J. J. The rosetta dock server for local protein–protein docking. *Nucleic acids research*, 36 (suppl_2):W233–W238, 2008.
- Ma, B., Terayama, K., Matsumoto, S., Isaka, Y., Sasakura, Y., Iwata, H., Araki, M., and Okuno, Y. Structure-based de novo molecular generator combined with artificial intelligence and docking simulations. *Journal of Chemical Information and Modeling*, 61(7):3304–3313, 2021.
- Mazyavkina, N., Sviridov, S., Ivanov, S., and Burnaev, E. Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research*, 134: 105400, 2021.
- Morris, G. M., Huey, R., and Olson, A. J. Using autodock for ligand-receptor docking. *Current protocols in bioinformatics*, 24(1):8–14, 2008.
- Moss, H., Leslie, D., Beck, D., Gonzalez, J., and Rayson, P. Boss: Bayesian optimization over string spaces. *Advances in neural information processing systems*, 33: 15476–15486, 2020.
- Nigam, A., Friederich, P., Krenn, M., and Aspuru-Guzik, A. Augmenting genetic algorithms with deep neural networks for exploring the chemical space. *arXiv preprint arXiv:1909.11655*, 2019.
- Niu, Y., Gao, Z., Xu, T., Liu, Y., Bian, Y., Rong, Y., Huang, J., and Li, J. Inversiongnn: A dual path network for multi-property molecular optimization. *arXiv preprint arXiv:2503.01488*, 2025.
- Peng, X., Luo, S., Guan, J., Xie, Q., Peng, J., and Ma, J. Pocket2mol: Efficient molecular sampling based on 3d protein pockets. In *International conference on machine learning*, pp. 17644–17655. PMLR, 2022.
- Qiu, K., Song, Y., Yu, J., Ma, H., Cao, Z., Zhang, Z., Wu, Y., Zheng, M., Zhou, H., and Ma, W.-Y. Empower structure-based molecule optimization with gradient guided bayesian flow networks. *arXiv preprint arXiv:2411.13280*, 2024.
- Ragoza, M., Masuda, T., and Koes, D. R. Generating 3d molecules conditional on receptor binding sites with deep generative models. *Chemical science*, 13(9):2701–2713, 2022.
- Saka, K., Kakuzaki, T., Metsugi, S., Kashiwagi, D., Yoshida, K., Wada, M., Tsunoda, H., and Teramoto, R. Antibody design using lstm based deep generative model from phage display library for affinity maturation. *Scientific reports*, 11(1):5852, 2021.
- Schneuing, A., Harris, C., Du, Y., Didi, K., Jamasb, A., Igashov, I., Du, W., Gomes, C., Blundell, T. L., Lio, P., et al. Structure-based drug design with equivariant diffusion models. *Nature Computational Science*, 4(12): 899–909, 2024.
- Schneuing, A., Igashov, I., Dobbstein, A. W., Castiglione, T., Bronstein, M. M., and Correia, B. Multi-domain distribution learning for de novo drug design. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Shi, C., Xu, M., Zhu, Z., Zhang, W., Zhang, M., and Tang, J. Graphaf: a flow-based autoregressive model for molecular graph generation. *arXiv preprint arXiv:2001.09382*, 2020.
- Skalic, M., Jiménez, J., Sabbadin, D., and De Fabritiis, G. Shape-based generative modeling for de novo drug design. *Journal of chemical information and modeling*, 59(3):1205–1214, 2019.
- Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models, 2022. URL <https://arxiv.org/abs/2010.02502>.

- Ståhl, N., Falkman, G., Karlsson, A., Mathiason, G., and Bostrom, J. Deep reinforcement learning for multiparameter optimization in de novo drug design. *Journal of chemical information and modeling*, 59(7):3166–3176, 2019.
- Sverrisson, F., Feydy, J., Correia, B. E., and Bronstein, M. M. Fast end-to-end learning on protein surfaces. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15272–15281, June 2021.
- Tischer, D., Lisanza, S., Wang, J., Dong, R., Anishchenko, I., Milles, L. F., Ovchinnikov, S., and Baker, D. Design of proteins presenting discontinuous functional sites using deep learning. *Biorxiv*, pp. 2020–11, 2020.
- Verdonk, M. L., Cole, J. C., Hartshorn, M. J., Murray, C. W., and Taylor, R. D. Improved protein–ligand docking using gold. *Proteins: Structure, Function, and Bioinformatics*, 2003.
- Wang, R., Fang, X., Lu, Y., and Wang, S. The pdbbind database: Collection of binding affinities for protein–ligand complexes with known three-dimensional structures. *Journal of medicinal chemistry*, 47(12):2977–2980, 2004.
- Wang, Z., Nie, W., Qiao, Z., Xiao, C., Baraniuk, R., and Anandkumar, A. Retrieval-based controllable molecule generation. *arXiv preprint arXiv:2208.11126*, 2022.
- Wu, F. and Li, S. Z. Surface-vqmae: Vector-quantized masked auto-encoders on molecular surfaces. In *Forty-first International Conference on Machine Learning*, 2024.
- Xie, Y., Shi, C., Zhou, H., Yang, Y., Zhang, W., Yu, Y., and Li, L. Mars: Markov molecular sampling for multi-objective drug discovery. *arXiv preprint arXiv:2103.10432*, 2021.
- Xu, M., Yu, L., Song, Y., Shi, C., Ermon, S., and Tang, J. Geodiff: A geometric diffusion model for molecular conformation generation. *arXiv preprint arXiv:2203.02923*, 2022.
- Yan, Y., Tao, H., He, J., and Huang, S.-Y. The hdock server for integrated protein–protein docking. *Nature protocols*, 15(5):1829–1852, 2020.
- You, J., Liu, B., Ying, Z., Pande, V., and Leskovec, J. Graph convolutional policy network for goal-directed molecular graph generation. *Advances in neural information processing systems*, 31, 2018.
- Zhou, X., Xue, D., Chen, R., Zheng, Z., Wang, L., and Gu, Q. Antigen-specific antibody design via direct energy-based preference optimization. *Advances in Neural Information Processing Systems*, 37:120861–120891, 2024.
- Zhou, Z., Kearnes, S., Li, L., Zare, R. N., and Riley, P. Optimization of molecules via deep reinforcement learning. *Scientific reports*, 9(1):10752, 2019.
- Zhu, T., Ren, M., and Zhang, H. Antibody design using a score-based diffusion model guided by evolutionary, physical and geometric constraints. In *Forty-first International Conference on Machine Learning*, 2024.

Appendix

Table of Content

A	The Inversion Framework in Details	15
A.1	Model Training	15
A.2	Inversion Phase	15
B	Related Works in Details	16
C	Methodology in Details.	17
C.1	Stage 1: Docking-oriented Ligand Modeling	17
C.2	Stage 2: Unsupervised Pre-training	19
C.3	Stage 3: Supervised Fine-tuning	20
C.4	Stage 4: Inversion-based Ligand Generation	22
D	Experiment in Details.	23
D.1	Computation Resource	23
D.2	Datasets	24
D.3	Baseline Models.	25
D.4	Evaluation Metrics	27
D.5	More experiment figures & tables	30
E	Theoretical Analysis in Details	32
E.1	Proof of SE(3)-Equivariance for MagicDock	32
E.2	Proof of Convergence for MagicDock.	34
E.3	Proof of Theoretical Superiority for the Inversion Framework	37
E.4	Proof of Efficiency for Inversion Framework	38
E.5	Proof of Information-Theoretic Superiority for Inversion Framework	40
E.6	Proof of Sample Complexity Advantage for MagicDock’s Inversion Pipeline	41
F	Limitations and Future Work	43
G	Algorithms in Pseudo Code.	44
G.1	Stage 1: Docking-oriented Ligand Modeling	44
G.2	Stage 2: Unsupervised pre-training	44
G.3	Stage 3: Supervised fine-tuning.	45
G.4	Stage 4: Inversion-based Ligand Generation	46
G.5	Overall pipeline	46
H	Adaptations to Baselines for Fair Comparison	47
I	Potential Societal Impacts.	48
J	GenAI Usage Disclosure	48
K	Reproducibility Statement	48

L	Visualization of Generated Ligand.	48
	L.1 Protein ligand	48
	L.2 Molecule ligand	48
M	Hyperparameters in Details	49
	M.1 Protein Baselines	50
	M.2 Molecule Baselines	51
	M.3 MagicDock	53

A. The Inversion Framework in Details

A.1. Model Training

Likelihood-based generative models (e.g., VAEs, diffusion, and flow matching) and optimization-based design loops offer two complementary ways to synthesize ligands. The former learn a sampler over the ligand space and can leverage likelihood modeling and various inference acceleration techniques, while the latter treat a predictive network as an energy or objective function and update the input variables directly by gradient-based optimization. Optimization-based design has already been widely adopted in computational biology, especially when high-accuracy predictors (e.g., AlphaFold-level models) are available. In MagicDock, we follow this general paradigm but instantiate it on a docking-aware, surface-based backbone: the model is first trained to encode generic surface patterns and docking semantics, and only then used in an inversion phase where its gradients are repurposed to drive de novo ligand generation.

① **Unsupervised Pre-Training.** The first phase establishes a shared backbone that encodes transferable geometric and chemical priors without relying on docking labels. In general, unsupervised encoders transform structured input data with n elements and their relationships into latent embeddings $\mathbf{z} \in \mathbb{R}^d$, updated iteratively by a propagation rule

$$\mathbf{z}_{t+1} = \text{Prop}(\mathbf{z}_t, \{\mathbf{z}_v \mid v \in \mathcal{N}(u)\}, \theta), \quad (16)$$

where θ denotes trainable parameters and Prop is a propagation function adapted to the underlying data structure (graphs, point clouds, etc.). To be usable in a later inversion phase, this latent space must be differentiable with respect to the input geometry and features.

In our work, this phase employs a Surface-VQMAE-style SE(3)-equivariant encoder to process interaction graphs derived from 3D surface point clouds of proteins and small molecules, generating embeddings $\mathbf{h}_u \in \mathbb{R}^d$ as

$$\mathbf{h}_u^{(l)} = \text{Prop}_{\text{SE}(3)}\left(\mathbf{h}_u^{(l-1)}, \left\{\mathbf{h}_v^{(l-1)} \mid v \in \mathcal{N}(u)\right\}\right). \quad (17)$$

SE(3)-equivariant propagation weights are optimized with self-supervised objectives such as masked reconstruction and vector quantization, embedding rich surface semantics from large collections of proteins and small molecules. This yields a unified, differentiable surface backbone that is not yet docking-specific, but that provides a stable latent space for subsequent supervised calibration and gradient-based inversion.

② **Supervised Fine-Tuning.** Building on this pre-trained backbone, the second phase aligns representations with docking semantics using ground-truth labels from protein–ligand complexes. As detailed in Section 3.3 and Appendix C.3, receptor and ligand point clouds are encoded into latent fields $\mathbf{Z}_r$ and $\mathbf{Z}_l$, fused via SE(3)-equivariant cross-attention, and supervised by three structured tasks: pocket segmentation, receptor–ligand interaction prediction, and binding-affinity regression. Let E_{dock} denote an aggregate docking objective derived from these supervised heads. Gradients $\nabla_{\mathbf{h}_u} E_{\text{dock}} = \partial E_{\text{dock}} / \partial \mathbf{h}_u$ are used during training to adjust encoder parameters and attention weights, calibrating the backbone so that its internal notion of low "energy" correlates with physically plausible, high-affinity binding modes.

Across both phases, the trainable components (encoders, attention modules, and lightweight task heads) are collectively parameterized by f_θ . The result is a docking-aware surface backbone in which generic structural knowledge and task-specific docking knowledge are compactly encoded. The inversion phase then exposes this knowledge in the form of gradients, which we use to steer ligand structure and features during generation rather than training a separate generative network.

A.2. Inversion Phase

Given the trained backbone f_θ , the inversion phase treats de novo design as an optimization problem in the ligand’s continuous representation. In a generic form, gradient-based inversion updates a latent or structural variable $\mathbf{x}$ with respect to an objective L as

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla_{\mathbf{x}_t} L(\mathbf{x}_t), \quad (18)$$

where η is a step size and L can encode multiple domain-specific constraints. In our implementation, the objective combines the docking-related heads from Stage 3 with optional regularizers enforcing chemistry and synthesizability, and the inversion module refines ligand surface point positions $\mathbf{p}_u$ and features $\mathbf{f}_u$ by

$$\mathbf{x}_{t+1} = \text{InvGen}(P_t) = \text{InvGen}(\mathbf{p}_u - \alpha \nabla_{\mathbf{p}_u} E_{\text{dock}}, \mathbf{f}_u - \beta \nabla_{\mathbf{f}_u} E_{\text{dock}}, C_{\text{bio}}), \quad (19)$$

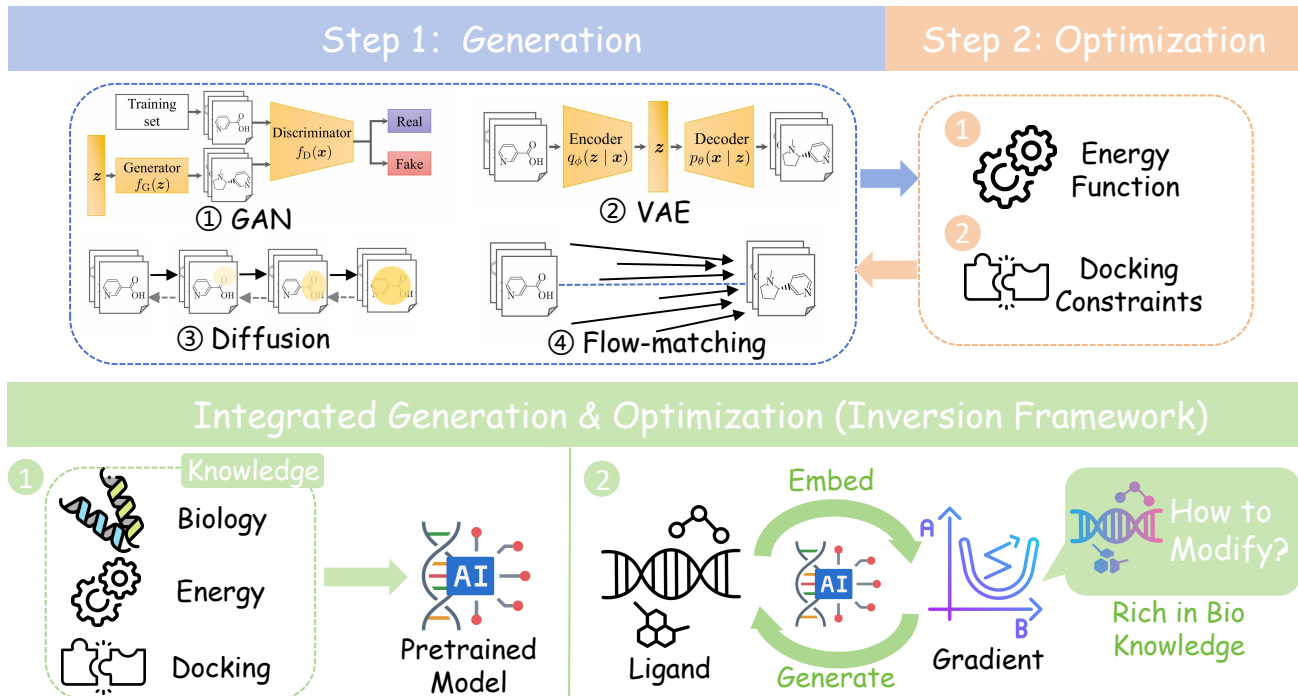


Figure 9. Comparison of an inversion-based design loop with likelihood-based generative ligand design. MagicDock follows the former paradigm but instantiates it on a docking-aware, surface-based backbone.

where α and β are learning rates, E_{dock} aggregates pocket, interaction, and affinity objectives, and C_{bio} denotes biochemical constraints (e.g., valence rules, scaffold priors). In this sense, MagicDock can be viewed as performing projected gradient descent in a docking-aware surface latent space, with the predictive heads serving as differentiable energy terms.

Compared to likelihood-based generators, this optimization-based inversion is typically slower at inference time, since each design requires multiple gradient steps. However, it offers two practical advantages that motivate our choice in this work: (i) it reuses a single docking-aware backbone for both prediction and generation, avoiding the need to train an additional sampler in the high-dimensional ligand space; and (ii) it provides fine-grained controllability, as different design goals and constraints can be combined, reweighted, or modified at inference time simply by changing the objective E_{dock} and constraint set C_{bio} , without retraining a separate generative model. Conceptually, our inversion procedure can be interpreted within the same classifier-guidance or stochastic optimal control viewpoint that has been developed for diffusion and flow models; exploring such hybrids that couple our surface-based backbone with likelihood-based generators is an interesting direction for future work.

B. Related Works in Details

Following the taxonomy in the main text, we reorganize existing approaches into four paradigms according to how generation is coupled with optimization. For completeness, we further discuss them from two complementary perspectives: protein design and molecular ligand design.

① Decoupled Paradigms. (a) *Protein design.* Classical protein-protein docking pipelines, such as ZDOCK (Chen et al., 2010), HADDOCK (Dominguez et al., 2003), and RosettaDock (Lyskov & Gray, 2008), employ geometric complementarity and energy minimization, sometimes with partial priors like known paratopes (Kozakov et al., 2017; Yan et al., 2020; Ganea et al., 2021). Deep learning extensions such as AlphaFold-Multimer (Evans et al., 2021) improve accuracy but remain resource-intensive. Early generative efforts, e.g., RNN-based paratope generation (Liu et al., 2020; Saka et al., 2021) or sequence-structure joint models (Jin et al., 2021), also separated sampling from docking-based optimization.

(b) *Molecular ligand design.* Structure-based drug discovery relies on docking engines including AutoDock (Morris et al., 2008), DOCK (Ewing et al., 2001), Glide (Halgren et al., 2004), and GOLD (Verdonk et al., 2003), supported by scoring functions and refinement via MD, FEP, or MM/PBSA simulations (Berdigaliyev & Aljofan, 2020; Fabricant & Farnsworth,

2001). These workflows represent ligands in predefined descriptors, then optimize poses post hoc. Early generative models (VAEs (Gómez-Bombarelli et al., 2018; Skalic et al., 2019), GANs (Guimaraes et al., 2017)) also adopted a decoupled generation–evaluation scheme. While interpretable, such methods are loosely tied to binding or multi-objective constraints.

② **Implicitly Coupled Paradigms.** (a) *Protein ligand design.* Here, optimization signals are embedded into stochastic generative processes. Graph-based generative methods (Jin et al., 2021), backbone-conditioned models (Ingraham et al., 2019), and energy-aware strategies (Tischer et al., 2020; Cao et al., 2021) guide sequence/structure generation with implicit docking feedback. Diffusion-style models (e.g., DiffAb, HSRN) refine CDR loops and interfaces with SE(3)-equivariant priors.

(b) *Molecule ligand design.* Diffusion-based models such as DiffDock (Corso et al., 2022) and GeoDiff (Xu et al., 2022) integrate ΔG or docking signals during denoising. Pocket2Mol (Peng et al., 2022) and condition on receptor pockets. Broader frameworks, including DockStream (Guo et al., 2021), ALIDIFF (Gu et al., 2024), and DRUGFLOW (Schneuing et al., 2025), couple docking or ADMET constraints with molecular diffusion or generative flows (Shi et al., 2020; Lee et al., 2023). VAE (Liu et al., 2018; Fu et al., 2020; Griffiths & Hernández-Lobato, 2020; Wang et al., 2022) and GAN-based methods (De Cao & Kipf, 2018; Abbasi et al., 2022) also benefit from structural conditioning. Despite progress, efficiency and smooth latent space learning (Brown et al., 2019; Huang et al., 2021; Gao et al., 2024b) remain challenges.

③ **Surrogate- or Heuristic-coupled Paradigms.** (a) *Protein ligand design.* Reinforcement learning has been applied to antibody optimization (Ingraham et al., 2019; Tischer et al., 2020), while memory-augmented models integrate docking oracles into generation. Energy-based approaches similarly exploit docking rewards (Cao et al., 2021), yet suffer from high computational cost.

(b) *Molecule ligand design.* Ligand optimization often leverages search in discrete chemical space. Reinforcement learning (Ståhl et al., 2019; You et al., 2018; Zhou et al., 2019; Gottipati et al., 2020; Gao et al., 2024a; Jain et al., 2023), evolutionary algorithms (Jensen, 2019; Nigam et al., 2019; Chen et al., 2021), Markov Chain Monte Carlo (Xie et al., 2021; Fu et al., 2021b), tree search (Ma et al., 2021), and Bayesian optimization (Korovina et al., 2020; Moss et al., 2020) iteratively refine molecules with docking oracles. While effective, they require many evaluations and struggle with balancing multiple objectives (Blum & Roli, 2003; Mazyavkina et al., 2021). Frameworks like DockStream (Guo et al., 2021), ALIDIFF (Gu et al., 2024), and DRUGFLOW (Schneuing et al., 2025) attempt to reduce cost via chemical and geometric priors.

④ Explicitly Gradient-coupled Paradigm (Ours).

Inversion-based frameworks directly couple generation and optimization via gradient guidance in latent space. Unlike stochastic or heuristic strategies, gradients provide more efficient and interpretable refinements of sequence–structure pairs ((Niu et al., 2025);(Fu et al., 2021a)). The proposed **MagicDock** exemplifies this paradigm by unifying 3D geometry, docking constraints, and biophysical objectives. By treating ligand generation as a gradient-driven process, it co-optimizes ΔG , specificity, and stability without handcrafted schedules. This enables biologically relevant designs under a single, generalizable architecture.

C. Methodology in Details

C.1. Stage 1: Docking-oriented Ligand Modeling

We represent ligands (both proteins and molecules) through solvent-accessible surfaces parameterized by a probe radius $r_{\text{probe}} = 1.4 \text{ \AA}$. Let $C = \{(c_j, r_j)\}_{j=1}^M$ denote atomic centers and van der Waals radii. The molecular surface is defined as the iso-level set of the distance field:

$$S = \{x \in \mathbb{R}^3 \mid d(x; C) = r_{\text{probe}}\} \quad (20)$$

$$d(x; C) = \min_j \|x - c_j\| - r_j \quad (21)$$

where the minus sign ensures consistency with van der Waals boundaries. A differentiable representation is achieved using a smoothed distance function:

$$\text{SDF}(x) = -\bar{\sigma}(x) \log \sum_{j=1}^M \exp\left(-\frac{\|x - c_j\|}{\sigma_j}\right) \quad (22)$$

$$\bar{\sigma}(x) = \frac{\sum_j \exp(-\|x - c_j\|) \sigma_j}{\sum_j \exp(-\|x - c_j\|)}. \quad (23)$$

Candidate points, sampled from Gaussian perturbations around atoms, are iteratively projected onto the iso-surface via gradient descent. A triangulated mesh is constructed using the ball-pivoting algorithm (BPA) with radii in $[r_{\text{probe}}, 2r_{\text{probe}}]$, and $N = 5000$ points are uniformly sampled for proteins (scaled for ligands). Surface normals are estimated by weighted PCA within a $2r_{\text{probe}}$ neighborhood:

$$\Sigma_i = \sum_{j \in \mathcal{N}_n(i)} w_{ij} (x_j - \bar{x}_i)(x_j - \bar{x}_i)^\top, \quad (24)$$

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{(2r_{\text{probe}})^2}\right), \quad (25)$$

where $\mathcal{N}_n(i)$ is the set of neighboring points for normal estimation.

Each surface point x_i is enriched with a feature vector integrating chemical descriptors, atomic type indicators, geometric descriptors, coordinates, and a molecule-type identifier I_i :

$$f(x_i) = \text{concat}(f_{\text{chem}}(x_i), f_{\text{atom}}(x_i), f_{\text{geom}}(x_i), I_i, x_i). \quad (26)$$

Atomic features. Atomic features $f_{\text{atom}}(x_i)$ encode the local element distribution through weighted one-hot statistics. For proteins, we adopt $\{C, H, O, N, S, Se\}$ (6D). For small molecules, we adopt $\{C(sp3), C(sp2), H, O(sp3), O(sp2), N(sp3), N(sp2), S(sp2)\}$ (8D). The statistics are aggregated using probe-scaled weights $\omega_{ia} = 1/(\|x_i - c_a\|/r_{\text{probe}} + \varepsilon)$. *Note that this atomic vocabulary is limited to common bio-organic atoms, while halogens and metal ions frequently appearing in pharmaceutically relevant compounds are not yet included; we discuss extending the feature space in Section F.*

Chemical features. Chemical descriptors $f_{\text{chem}}(x_i)$ are continuous values that capture local physicochemical properties from neighboring atoms within $r_{\text{chem}} = 5.0 \text{ \AA}$. Specifically, we define: hydrogen-bonding potential,

$$H_{\text{bond}}(x_i) = \frac{\sum_{a \in \mathcal{N}_c(i)} \omega_{ia} \mathbb{I}[t_a \in \{O, N, S\}]}{\sum_{a \in \mathcal{N}_c(i)} \omega_{ia}}, \quad (27)$$

charge polarity,

$$C(x_i) = \frac{\sum_{a \in \mathcal{N}_c(i)} \omega_{ia} (\mathbb{I}[t_a \in \{O, N\}] - \mathbb{I}[t_a \in \{C, S\}])}{\sum_{a \in \mathcal{N}_c(i)} \omega_{ia}}, \quad (28)$$

and hydrophobicity/aromaticity,

$$H_{\text{phob}}(x_i) = \frac{\sum_{a \in \mathcal{N}_c(i)} \omega_{ia} \mathbb{I}[t_a = C]}{\sum_{a \in \mathcal{N}_c(i)} \omega_{ia}} \quad (29)$$

$$A_{\text{aro}}(x_i) = \text{clip}\left(\frac{H_{\text{phob}}(x_i)}{4.5}, 0, 1\right). \quad (30)$$

Here, $\mathcal{N}_c(i)$ denotes the chemical neighborhood, $\mathbb{I}[\cdot]$ the indicator function, and $\text{clip}(x, a, b)$ truncates x into $[a, b]$.

Geometry features. Atomic statistics $T_{i,k}$ reflect weighted element distributions, with 6D for proteins and 8D for small molecules. Geometric descriptors from a $k_g = 10$ nearest-neighbor set $\mathcal{N}_g(i)$ include mean curvature $\kappa_{i,1}$, covariance-volume proxy $\kappa_{i,2}$, and local density D_i , computed as:

$$\kappa_{i,1} = \frac{1}{|\mathcal{N}_g(i)|} \sum_{j \in \mathcal{N}_g(i)} \|n_i - n_j\|, \kappa_{i,2} = \prod_{m=1}^3 \epsilon_m, \quad (31)$$

$$\{\epsilon_1, \epsilon_2, \epsilon_3\} = \text{eig}\left(\frac{1}{|\mathcal{N}_g(i)|} \sum_{j \in \mathcal{N}_g(i)} (x_j - \bar{x}_i)(x_j - \bar{x}_i)^\top\right), \quad (32)$$

$$D_i = \frac{1}{k_g} \sum_{j \in \mathcal{N}_g(i)} \|x_i - x_j\|, \quad \text{normalized to } [0, 1], \quad (33)$$

where $\text{eig}(\cdot)$ returns eigenvalues of the covariance matrix.

To enhance scalability, point clouds are partitioned into overlapping patches. Centers are obtained via farthest point sampling (FPS), each grouping $K = 50$ neighbors within $r_{\text{patch}} = 5.0 \text{ \AA}$:

$$\mathcal{P}(x_c) = \{x_j \mid x_j \in \text{KNN}(x_c, K), \|x_j - x_c\| \leq r_{\text{patch}}\}, \quad (34)$$

where KNN denotes the K -nearest neighbors of a center point. Patch features are summarized by mean and variance pooling:

$$\mu_{k,f} = \frac{1}{|\mathcal{P}_k|} \sum_{x_i \in \mathcal{P}_k} f(x_i), \sigma_{k,f}^2 = \frac{1}{|\mathcal{P}_k|} \sum_{x_i \in \mathcal{P}_k} (f(x_i) - \mu_{k,f})^2, \quad (35)$$

with interface labels assigned for points within $4.0 \text{ \AA}$ of ligand or $2.0 \text{ \AA}$ of protein surfaces.

This yields a probe-aware, feature-enriched, patch-structured representation, unifying proteins and ligands for pre-training and generative modeling.

C.2. Stage 2: Unsupervised Pre-training

In the second stage, the encoder is pre-trained in an unsupervised manner to capture transferable docking-aware priors from proteins and small molecules. The framework integrates SE(3)-equivariant convolutions, patch-level masking with vector quantization, and reconstruction objectives targeting geometric and physicochemical properties, establishing a robust foundation for subsequent fine-tuning. Below, we detail the computational framework and objectives, ensuring seamless integration of encoding and reconstruction processes.

SE(3)-Equivariant Encoding. Each molecular surface is represented as a collection of patches

$$X_p = \{(\mathbf{x}_i, f_i)\}_{i=1}^K,$$

where $\mathbf{x}_i \in \mathbb{R}^3$ are Cartesian coordinates, $f_i \in \mathbb{R}^d$ are feature vectors of dimension d , and K is the total number of patches. The encoder aggregates local neighborhoods $\mathcal{N}(i)$ using SE(3)-equivariant convolutions:

$$z_i = \sum_{j \in \mathcal{N}(i)} \sum_{l=0}^L R_l(\|\mathbf{x}_{ij}\|) Y_l\left(\frac{\mathbf{x}_{ij}}{\|\mathbf{x}_{ij}\|}\right) \cdot W_l f_j, \quad (36)$$

where $\mathbf{x}_{ij} = \mathbf{x}_j - \mathbf{x}_i$, $R_l(\cdot)$ are learnable radial functions, $Y_l(\cdot)$ are spherical harmonics of order l , W_l are trainable matrices, and L is the maximum order of harmonics. This ensures equivariance under rigid motions (R, t) with $R \in SO(3)$ and $t \in \mathbb{R}^3$:

$$z_i(R\mathbf{x} + t) = \rho(R) z_i(\mathbf{x}), \quad (37)$$

where $\rho(R)$ is the irreducible representation of $SO(3)$ acting on the feature space, capturing both scalar and higher-order interactions.

Patch Masking and Vector Quantization. A fraction of patches are masked at a specified ratio ρ , and their embeddings are replaced by vector-quantized latents sampled from a learnable codebook $\mathcal{E}$ of size N_B . Following the discrete variational autoencoder (dVAE) relaxation, the quantized latent is computed as:

$$z_{p,i,m} = \frac{\sum_{j=1}^{N_B} \exp\left(\frac{g_j + \log q(e_j | h_{p,i,m})}{\tau}\right) e_j}{\sum_{j=1}^{N_B} \exp\left(\frac{g_j + \log q(e_j | h_{p,i,m})}{\tau}\right)}, \quad (38)$$

where $h_{p,i,m}$ is the hidden state, $e_j \in \mathcal{E}$ are codebook entries, $g_j \sim \text{Gumbel}(0, 1)$ is sampled noise, and τ is the softmax temperature. Visible and quantized embeddings are then passed to the decoder for reconstruction, preserving contextual information across masked regions.

Reconstruction Targets. For coordinate recovery, the decoder outputs:

$$\hat{X} = \text{Reshape}(\text{MLP}(H_p^{(L_2)})), \quad \hat{X} \in \mathbb{R}^{\delta \rho M \times K \times 3}, \quad (39)$$

where $H_p^{(L_2)}$ is the decoder output from the L_2 -th layer, M is the number of input molecular samples, and δ is a constant scaling factor. The multilayer perceptron (MLP) produces reconstructed Cartesian coordinates.

For surface geometry, each masked patch $X_{p,i,m}$ with center $x_{c,i,m}$ has covariance:

$$\Sigma = \frac{1}{K} \sum_{j=1}^K (x_{p,i,j,m} - x_{c,i,m})(x_{p,i,j,m} - x_{c,i,m})^\top \in \mathbb{R}^{3 \times 3}. \quad (40)$$

Pseudo-curvatures are derived from its eigenvalues $\epsilon_1, \epsilon_2, \epsilon_3$ as:

$$\psi_i = \left(\frac{\epsilon_1}{\epsilon_1 + \epsilon_2 + \epsilon_3}, \frac{\epsilon_2}{\epsilon_1 + \epsilon_2 + \epsilon_3}, \frac{\epsilon_3}{\epsilon_1 + \epsilon_2 + \epsilon_3} \right). \quad (41)$$

An MLP predicts $\hat{\psi}_i$, supervised by root mean square error (RMSE), ensuring accurate capture of local surface geometry.

Training Objective. The overall loss combines reconstruction, curvature, and regularization terms:

$$\mathcal{L} = \nu_1 \mathcal{L}_{\text{rec}} + \nu_2 \mathcal{L}_{\text{cur}} + \nu_3 \mathcal{L}_{\text{KL}}, \quad (42)$$

where ν_1, ν_2, ν_3 are scalar weights. The reconstruction loss uses the Chamfer distance:

$$\mathcal{L}_{\text{rec}} = \frac{1}{\delta \rho M K} \sum_{i=1}^{\delta \rho M} \left(\sum_{a \in \hat{X}_i} \min_{b \in X_{p,i,m}} \|a - b\|_2^2 \right), \quad (43)$$

$$+ \frac{1}{\delta \rho M K} \sum_{i=1}^{\delta \rho M} \left(\sum_{b \in X_{p,i,m}} \min_{a \in \hat{X}_i} \|a - b\|_2^2 \right). \quad (44)$$

The curvature loss is defined as:

$$\mathcal{L}_{\text{cur}} = \frac{1}{\delta \rho M} \sum_{i=1}^{\delta \rho M} \|\psi_i - \hat{\psi}_i\|_2^2, \quad (45)$$

and the KL divergence term $\mathcal{L}_{\text{KL}}$ regularizes the posterior $q(Z_{p,m} \mid H_{p,m})$ toward a uniform categorical prior.

This unsupervised pre-training framework leverages SE(3)-equivariant encoding, quantized masked reconstruction, and curvature supervision to learn robust docking-aware priors, providing an effective initialization for downstream fine-tuning.

C.3. Stage 3: Supervised Fine-tuning

In the third stage, the encoder pre-trained in Stage 2 is adapted to docking-specific tasks through supervised fine-tuning on protein–ligand complexes with ground-truth labels. This stage integrates cross-attention fusion to capture receptor–ligand interactions and employs three cascaded supervision signals—pocket segmentation, interaction prediction, and binding affinity regression—that follow the natural decision process of docking: localizing pockets, assessing whether a ligand can form a stable interaction at that site, and finally estimating binding strength.

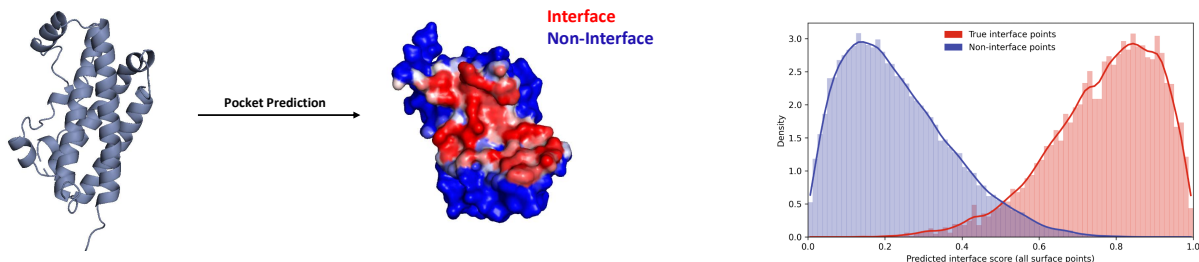
Equivariant Attention Fusion. Given a receptor surface R and a ligand L , their point-cloud representations are encoded independently by the Stage 2 encoders, yielding latent fields $Z_r \in \mathbb{R}^{N_r \times d}$ and $Z_l \in \mathbb{R}^{N_l \times d}$, where N_r and N_l denote the numbers of sampled points on the receptor and ligand surfaces for the current complex and are not fixed hyperparameters. The attention module operates on sets and therefore naturally supports variable-size point clouds; in particular, in Stage 4 ligand generation the ligand point-cloud cardinality N_l adapts to the current number of ligand atoms along the inversion trajectory.

To model interfacial dependencies, we employ an SE(3)-equivariant attention mechanism, in which scalar attention weights are computed from $\ell = 0$ channels while higher-order features are rotated via Wigner- D matrices to preserve equivariance:

$$\text{Attn}(Z_r, Z_l) = \text{softmax} \left(\frac{Q_r^{(\ell=0)} (K_l^{(\ell=0)})^\top}{\sqrt{d}} \right) V_l, \quad (46)$$

where $Q_r^{(\ell=0)} = Z_r^{(\ell=0)} W_Q$, $K_l^{(\ell=0)} = Z_l^{(\ell=0)} W_K$, and $V_l = Z_l W_V$ with $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ being learnable projection matrices. The resulting cross-attended features are aggregated with the original receptor embeddings using a permutation-invariant operator $\mathcal{A}(\cdot)$ (implemented as the concatenation of mean- and max-pooling):

$$\tilde{z} = \mathcal{A}(Z_r, \text{Attn}(Z_r, Z_l)) \in \mathbb{R}^{2d}.$$



(a) Pocket prediction on the receptor surface (red: pocket/interface; blue: non-pocket/non-interface).

(b) Distribution of predicted pocket scores over all receptor points, split by ground-truth labels.

Figure 10. Stage 3 pocket segmentation. (a) Predicted pocket (positive) regions on the receptor surface. (b) Density of predicted pocket scores across all receptor surface points, grouped by ground-truth pocket/non-pocket labels.

This fused representation captures both local receptor context and interfacial information and serves as the basis for the downstream supervision tasks.

Multi-Task Supervision. We design the training objectives to follow the natural progression of molecular docking: *pocket identification* $\rightarrow$ *interaction prediction* $\rightarrow$ *binding affinity estimation*. Each task builds on the representations and predictions produced by the previous one, forming a hierarchical, coarse-to-fine supervision scheme.

(i) Pocket segmentation. Each receptor embedding $z_i \in Z_r$ (for $i = 1, \dots, N_r$) is first classified as pocket or non-pocket:

$$\hat{y}_i = \sigma(g(z_i)), \quad (47)$$

where $\hat{y}_i \in (0, 1)$ is the predicted pocket probability, $g(\cdot)$ is a lightweight MLP head, and $\sigma(\cdot)$ denotes the sigmoid function. With ground-truth label $y_i \in \{0, 1\}$, the loss is defined as

$$\mathcal{L}_{\text{pocket}} = \frac{1}{N_r} \sum_{i=1}^{N_r} \text{BCE}(\hat{y}_i, y_i) + \lambda_p \mathcal{R}_{\text{geom}}, \quad (48)$$

where BCE denotes binary cross-entropy, λ_p is a regularization weight, and the geometric regularizer $\mathcal{R}_{\text{geom}}$ enforces agreement with distance-based pseudo-labels:

$$\mathcal{R}_{\text{geom}} = \frac{1}{N_r} \sum_{i=1}^{N_r} \|\hat{y}_i - y_i^{\text{geom}}\|^2,$$

with $y_i^{\text{geom}} = 1$ if the closest ligand point lies within a pre-defined cutoff radius and 0 otherwise. These pocket probabilities will be used to construct pocket-aware receptor features for the subsequent tasks. Fig. 10 provides a qualitative view of this stage: the left panel visualizes the predicted pocket (positive) regions on the receptor surface, while the right panel shows the distribution of predicted pocket scores over all receptor points, separated by ground-truth pocket/non-pocket labels. The clear separation between the two distributions indicates that the point-wise predictor yields a well-calibrated pocket scoring landscape suitable for downstream conditioning.

(ii) Pocket-conditioned interaction prediction. To condition interaction prediction on the predicted pocket, receptor embeddings are modulated by the pocket scores:

$$z_i^p = \hat{y}_i z_i, \quad \bar{z}_r = \frac{\sum_{i=1}^{N_r} z_i^p}{\sum_{i=1}^{N_r} \hat{y}_i + \varepsilon}, \quad (49)$$

where ε is a small constant to avoid division by zero. Here $\bar{z}_r$ is a pocket-aware pooled receptor representation, emphasizing residues that are likely to belong to binding pockets. In parallel, we obtain a global ligand embedding z_l by pooling the

ligand latent field Z_l (e.g., via mean- and max-pooling). These two vectors are fused into an interaction representation via another MLP:

$$\tilde{z} = f_{\text{int}}(\tilde{z}_r, z_l), \quad \hat{y}_{\text{int}} = \sigma(h_{\text{int}}(\tilde{z})), \quad (50)$$

where f_{int} and h_{int} are learnable networks and $\hat{y}_{\text{int}} \in (0, 1)$ denotes the predicted probability that the receptor–ligand pair forms a stable complex at the identified pocket. With the ground-truth interaction label $y_{\text{int}} \in \{0, 1\}$, the loss is

$$\mathcal{L}_{\text{int}} = \text{BCE}(\hat{y}_{\text{int}}, y_{\text{int}}). \quad (51)$$

(iii) Interaction-aware affinity regression. The same fused representation $\tilde{z}$, already conditioned on pocket geometry and receptor–ligand interaction, is further used to regress binding free energy. For complexes with experimentally measured affinities, we standardize the binding free energy $\Delta G(r, l) \in \mathbb{R}$ to zero mean and unit variance and define

$$\hat{y}_{\Delta G} = h_{\Delta G}(\tilde{z}), \quad \mathcal{L}_{\Delta G} = (\hat{y}_{\Delta G} - \Delta G(r, l))^2, \quad (52)$$

where $h_{\Delta G}$ is a regression head. In this way, affinity prediction is explicitly conditioned on both the localized pocket and the predicted likelihood of a stable receptor–ligand interaction.

Overall Objective. The joint fine-tuning objective combines the three tasks and reflects their bottom-up dependency:

$$\mathcal{L}_{\text{FT}} = \alpha \mathcal{L}_{\text{pocket}} + \beta \mathcal{L}_{\text{int}} + \mathcal{L}_{\Delta G}, \quad (53)$$

where α and β are weighting coefficients tuned on validation data. During optimization, the encoders (initialized from Stage 2), the cross-attention module, and task-specific MLP heads are updated jointly. Although all components are trained end-to-end, the formulation enforces a coarse-to-fine dependency: pocket predictions shape pocket-aware receptor features, which in turn define the interaction representation that serves as the basis for affinity regression.

C.4. Stage 4: Inversion-based Ligand Generation

In the fourth stage, the docking-aware backbone is transformed into a generative engine via an inversion mechanism. Unlike sampling–ranking pipelines, inversion performs direct gradient-based refinement in the latent continuous space, followed by projection and decoding into chemically valid discrete structures. This section provides the complete mathematical formulation, algorithms, and hyperparameters for ligand generation.

Composite Objective. Given a receptor R and a candidate ligand S , their surfaces are independently encoded into latent fields Z_r and Z_l by the Stage 3 encoders. The fused interfacial embedding $\tilde{z}$ is obtained via cross-attention fusion (Eq. 11). The supervised heads trained in Stage 3 yield a differentiable composite objective function $\mathcal{F}$:

$$\mathcal{F}(S; R, \Theta) = \alpha \mathcal{L}_{\text{pocket}}(S; R) + \beta \mathcal{L}_{\text{int}}(S; R) + \mathcal{L}_{\Delta G}(S; R), \quad (54)$$

where $\mathcal{L}_{\text{pocket}}$ is the binary cross-entropy loss for binding pocket localization, $\mathcal{L}_{\text{int}}$ is the interaction plausibility loss (evaluating atom–residue interfacial compatibility), and $\mathcal{L}_{\Delta G}$ is the regression loss for binding free energy ΔG . Here $\alpha = 1.0$ and $\beta = 0.5$ are balancing coefficients, and Θ are model parameters.

Gradient-based Refinement. Let (x_t, f_t) denote the differentiable ligand representation at iteration t , where $x_t \in \mathbb{R}^{N_l^{(t)} \times 3}$ are atom coordinates and $f_t \in \mathbb{R}^{N_l^{(t)} \times d_f}$ are atom features (including atomic type logits, partial charges, hybridization states, and hydrogen-bond polarity indicators). Here $N_l^{(t)}$ equals the current number of ligand atoms and is allowed to change along the inversion trajectory as atoms are inserted or deleted. At each iteration, the update step is

$$(x_{t+1}, f_{t+1}) = \Pi_{C_{\text{valid}}}((x_t, f_t) - \eta_t \nabla_{(x, f)} F(S_t; R, \Theta)), \quad (55)$$

where $\eta_t = 10^{-3}$ is the step size, and $\Pi_{C_{\text{valid}}}$ projects the updated state back onto the chemically and geometrically valid manifold C_{valid} , potentially changing the cardinality from $N_l^{(t)}$ to $N_l^{(t+1)}$ in order to enforce valid valence, realistic bond lengths, and avoidance of steric clashes.

Generative Mapping. After PGD refinement and projection, we decode the continuous state into a chemically valid structure via a type-specific mapping $\mathcal{G}_{\text{type}}$. We stop gradients through discrete sampling/repair and use gradient-biased probabilities as a decoding heuristic, followed by rule-based corrections to enforce validity.

For small molecules, atom typing is performed by softmax sampling:

$$a_i \sim \text{Categorical}(\sigma(f_i - \gamma \nabla_{f_i} \mathcal{F})), \quad a_i \in \mathcal{A}_{\text{atom}}, \quad (56)$$

where

$$\mathcal{A}_{\text{atom}} = \left\{ \begin{array}{l} \text{C(sp3), C(sp2), H, O(sp3),} \\ \text{O(sp2), N(sp3), N(sp2), S(sp2), halogens(sp3)} \end{array} \right\}$$

is the atom-type set, $\sigma(\cdot)$ is the softmax function, and $\gamma = 0.1$ balances gradient bias.

Bond inference uses logits g_{ij} and is jointly sampled over bond types $\{1, 2, 3, \text{aromatic}\}$:

$$b_{ij} \sim \text{Categorical}(\sigma(g_{ij} - \gamma \nabla_{g_{ij}} \mathcal{F})), \quad (57)$$

where bond types correspond to single, double, triple, and aromatic bonds. Feasibility is checked by distance thresholds (1.0–2.0 Å) and valence constraints $v_i \leq v_{\text{max}}(a_i)$.

To incorporate structural motifs, we introduce a prior $\mathcal{P}_{\text{motif}}$ based on point cloud geometry, identifying centers via clustering of high-curvature, high-density points. Predefined templates guiding atom placement include: Benzene, Pyridine, Furan, Pyrrole, Thiophene, Imidazole, Carboxyl, Amide, etc. The motif probability is:

$$P_{\text{motif}}(x_i) \propto \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{c}_{\text{motif}}\|^2}{\sigma_{\text{motif}}^2}\right) \cdot \mathbb{I}[\text{valid}(a_i, \mathbf{x}_i)], \quad (58)$$

where $\mathbf{c}_{\text{motif}}$ is the cluster center, $\sigma_{\text{motif}} = 0.5$ Å, and invalid structures are corrected by RDKit (a cheminformatics toolkit), with ΔG rewarding aromatic and functional motifs.

For protein ligands, residue identity r_i is sampled similarly:

$$r_i \sim \text{Categorical}(\sigma(f_i - \gamma \nabla_{f_i} \mathcal{F})), \quad r_i \in \mathcal{A}_{20}, \quad (59)$$

where $\mathcal{A}_{20}$ is the canonical amino acid set. Backbone torsions (ϕ, ψ) are continuously updated and projected into $[-180^\circ, 180^\circ]$, while side-chain torsions χ_k are sampled from the Dunbrack rotamer library. Cartesian reconstruction uses PyRosetta’s internal geometry engine, followed by a short relax; the repaired structure is re-embedded for the next step (no gradients through repair).

Thus, $\mathcal{G}_{\text{mol}}$ enforces atom- and bond-level chemical validity via stochastic decoding and sanitization, while $\mathcal{G}_{\text{prot}}$ performs residue-level sampling with biophysical torsional constraints. Both are tightly coupled with gradient refinement, ensuring that sampled structures remain docking-aware while respecting chemical and geometric feasibility.

Iterative Dynamics and Convergence. Repeated application of gradient refinement and generative mapping yields a sequence of ligands $\{S_t\}$ converging to an optimized structure:

$$S^* = \lim_{t \rightarrow T} \mathcal{G}_{\text{type}}((\mathbf{x}, f)^t), \quad (60)$$

with convergence typically observed within several hundreds steps. A cosine-annealed schedule is applied to $\eta_t \in [10^{-3}, 10^{-5}]$ to balance exploration and stability.

By tightly coupling gradient information with generative decoding, the framework ensures that optimization in latent space translates into chemically valid and docking-aware ligands. The distinction between $\mathcal{G}_{\text{mol}}$ (small molecules) and $\mathcal{G}_{\text{prot}}$ (proteins) allows the same inversion principle to adapt seamlessly to both types of ligands.

D. Experiment in Details

D.1. Computation Resource

All experiments were conducted on a single on-premise node unless otherwise specified. Table 3 summarizes the machine configuration and software stack to facilitate reproducibility.

Table 3. Compute environment used for all experiments.

Component	Configuration
Server	Dell PowerEdge T640
CPU	2× Intel Xeon Gold 6240 @ 2.60 GHz (18 cores/socket; 36 cores, 72 threads total)
Memory	251 GiB RAM
GPU	4× NVIDIA A100 80GB PCIe (80 GiB each); driver 570.124.06; MIG disabled
GPU topology	GPU0/1 near NUMA node 0; GPU2/3 near NUMA node 1
OS / Kernel	Ubuntu 22.04.5 LTS; Linux 5.15.0-126-generic
CUDA	Runtime 12.8 (from driver); Toolkit 12.4 (nvcc)
Python / Conda	Python 3.11.10
DL stack	PyTorch 2.1.0 (cu118 build), torchvision 0.16.0, torchaudio 2.1.0

Table 4. The statistical information of the experimental datasets.

Dataset	Samples	Complex Type	Affinity Info
SKEMPI v2	7,146	Protein-Protein	KD, ΔG
SabDab	1,069	Protein-Protein	KD, ΔG
PDBBind	2,789	Protein-Molecule	KD, ΔG
CrossDocked2020	18,450	Protein-Molecule	pK

D.2. Datasets

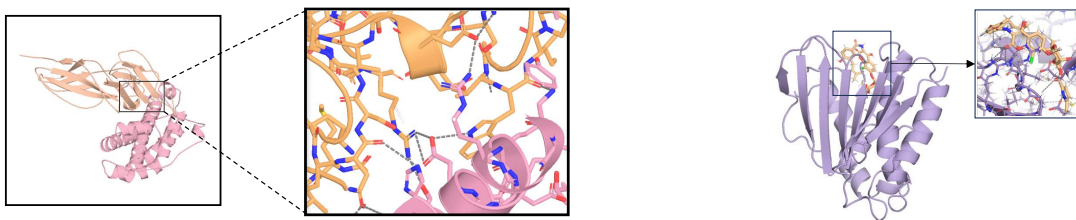
SKEMPI v2 (Liu et al., 2024) is a mutation-centric benchmark of experimentally measured binding-affinity changes in protein-protein complexes, with receptor/ligand chains already labeled. Existing chain labels are retained (with spot corrections if needed). Entries without an affinity value are removed (57). As with other sources, affinities are converted to KD (M), the scope is restricted to PPIs, and duplicates are merged using the same Complex-ID scheme.

Sabdab (Liu et al., 2024) is a large repository of antibody-antigen structures with explicit antigen, heavy-chain, and light-chain annotations. For consistency with a PPI setup, antigen chains are treated as receptor and antibody heavy/light chains as ligand. Records lacking affinity (14,148), non-PPI cases (46), entries missing antigen-chain labels (95), and chain-annotation errors (8) are removed. Remaining affinities are unified to KD (M), and duplicates across datasets are resolved with the Complex-ID definition above.

PDBBind v2020 (Wang et al., 2004) is a curated collection of biomolecular complexes; the protein-protein portion lists receptor/ligand names but not explicit chain IDs. Chain IDs are inferred with a semi-automatic procedure (parse chain descriptions, fuzzy-match to the annotated names) followed by expert proofreading and splitting of multi-complex PDBs, yielding chain assignments for 2,788 samples. Data cleaning removes non-PPI entries (6), records with ambiguous chain annotation (62), and entries whose reported affinities cannot be reliably converted to KD (62). All remaining affinities are standardized to KD (M), and cross-source duplicates are consolidated via a “Complex ID” composed of PDB code, sorted chains, mutations, and PubMed ID.

CrossDocked2020 (Francoeur et al., 2020) is a newly introduced dataset for structure-based machine learning, comprising 22.5 million poses of ligands docked into multiple similar binding pockets across the Protein Data Bank, designed to enhance the training and evaluation of grid-based convolutional neural network (CNN) models. This dataset includes cross-docked poses against non-cognate receptor structures and model-generated counterexamples, providing a standardized resource to recognize ligands in diverse target structures while significantly expanding the number of available poses for training.

To provide an intuitive view of the structural regimes covered by our benchmarks, Fig. 11 shows two representative complexes used in our experimental datasets: a protein-protein complex highlighting a binding interface and a protein-small-molecule complex illustrating a local binding pocket.



(a) A representative protein–protein complex with a zoom-in view of the binding interface.

(b) A representative protein–small-molecule complex with a zoom-in view of the binding pocket.

Figure 11. Dataset visualization examples. Representative complexes from the two ligand regimes in our datasets: (a) protein–protein interfaces and (b) protein–small-molecule binding pockets.

D.3. Baseline Models

BASLINE MODELS FOR PROTEIN

RAbD (Adolf-Bryfogle et al., 2018) introduces a knowledge-based Rosetta framework for computational antibody design that grafts canonical CDR loops from PyIgClassify, performs profile-guided sequence design with flexible-backbone sampling inside nested Monte-Carlo-plus-minimization cycles, and optimizes either total energy or interface ΔG . However, its reliance on existing structural data may limit its effectiveness for designing antibodies against novel or poorly characterized antigens.

DiffAb (Luo et al., 2022) proposes a diffusion-based, rotation/translation–equivariant generative model that co-designs antibody CDR sequences and 3D structures by iteratively denoising amino-acid types, $C\alpha$ coordinates, and $SO(3)$ side-chain orientations, all explicitly conditioned on the target antigen structure. But its performance may be limited by the quality and diversity of the training data, potentially restricting its ability to generalize to a wide range of antigens.

HSRN (Jin et al., 2022) introduces a hierarchical, rotation/translation–equivariant framework for antibody–antigen docking and design that combines a multi-scale encoder (atom- and residue-level) with an iterative, force-based refinement to fold and dock the paratope; during generation, an autoregressive decoder progressively docks partial paratopes and exploits the resulting geometric representation to choose the next residue. However, the model’s heavy reliance on pre-defined epitope structures, assuming the input already provides the antigen’s 3D structure and specific epitope location, poses a significant limitation, as epitope prediction remains a challenging task in practice, thereby restricting its end-to-end applicability.

dyMEAN (Kong et al., 2023) presents an end-to-end, $E(3)$ -equivariant full-atom antibody design framework that, given an antigen epitope and an incomplete antibody sequence, initializes structure using conserved framework residues, attaches a “shadow paratope” to exchange invariant information and enable docking, and iteratively co-updates residue types and 3D coordinates with an adaptive multi-channel encoder that handles variable atom counts per residue; docking is finalized by aligning the native and shadow paratopes, and the method reports superior results on CDR-H3 generation, complex structure prediction, and affinity optimization. Although dyMEAN excels in end-to-end design and full-atom modeling, its limited scalability and reliance on high-quality training data remain limitations.

AbX (Zhu et al., 2024) introduces a continuous-time, score-based diffusion framework that jointly models discrete CDR sequences (via a CTMC) and $SE(3)$ coordinates, conditioned on the antigen/framework, and guided by evolutionary (ESM-2) priors plus geometric (FAPE, distogram, IDDT) and physical (violation, van der Waals) constraints to narrow the search space and improve binding/quality.

ABDPO (Zhou et al., 2024) formulates antigen-specific sequence–structure co-design as direct energy-based preference optimization, fine-tuning a conditional diffusion model with residue-level energy preferences, decomposed attraction/repulsion terms, and gradient-surgery to resolve conflicts—thereby steering generations toward low total energy while maintaining binding affinity.

RoseTTAFold All-Atom (RFAA) (Krishna et al., 2024) extends RoseTTAFold to an all-atom, three-track architecture that jointly models proteins, nucleic acids, and small molecules, and supports diffusion-based design of protein binders

to small-molecule ligands. However, its accuracy on protein–ligand complexes remains strongly data-dependent and the framework is not tailored to antibody–antigen interfaces or CDR-level constraints, limiting its direct applicability to antibody-specific design.

Chai-1 (Boitreau et al., 2024) is an openly accessible, multimodal foundation model for biomolecular structure prediction that unifies proteins, nucleic acids, and protein–ligand complexes and achieves state-of-the-art accuracy on PoseBusters, low-homology interfaces, and monomer benchmarks relative to AF2.3 and RoseTTAFold All-Atom. However, it is predictive rather than generative, so design still depends on downstream sequence or ligand search and does not explicitly co-optimize therapeutic properties.

BASELINE MODELS FOR MOLECULE

DrugFlow (Schneuing et al., 2025) introduces a generative framework for structure-based drug design that jointly models continuous ligand coordinates and discrete atom/bond types by combining Euclidean flow matching with discrete Markov bridges. The method extends to **FlexFlow**, which additionally samples protein side-chain torsion angles to capture binding-pocket flexibility. Key innovations include an end-to-end uncertainty estimator for out-of-distribution detection, a virtual node mechanism for adaptive molecule size selection, and a multi-domain preference alignment scheme that efficiently steers generation toward molecules with desirable drug-like properties. Experiments on CrossDocked demonstrate state-of-the-art distribution learning across chemical, geometric, and physical metrics. While DrugFlow excels in holistic distribution modeling and flexible sampling, challenges remain in scaling to larger protein systems and balancing preference alignment with sample validity.

3D-SBDD (Luo et al., 2021) proposes a 3D generative framework for structure-based drug design that directly models ligand atoms within protein binding pockets using an autoregressive flow-based approach. The method conditions ligand generation on 3D protein environments, incrementally placing atoms while capturing geometric constraints, and employs equivariant neural networks to ensure rotational and translational invariance. Evaluation on CrossDocked shows significant improvements in binding pose accuracy, chemical validity, and docking performance compared to baseline methods. Despite its strong capability in geometry-aware ligand generation, challenges remain in handling larger, more flexible ligands and incorporating dynamic protein conformations.

ALIDIFF (Gu et al., 2024) introduces a target-aware molecule diffusion framework that aligns generative sampling with exact energy optimization for structure-based drug design. The method integrates a diffusion backbone with a dual-stage alignment scheme: a coarse-grained alignment to enforce global docking plausibility and a fine-grained optimization that explicitly minimizes binding energies within the protein pocket. By coupling stochastic generative modeling with deterministic energy-based refinement, AliDiff achieves superior docking accuracy and binding affinity prediction on CrossDocked benchmarks. While it demonstrates strong target-conditioning and energy alignment, its reliance on accurate energy models and the computational overhead of fine-grained optimization pose scalability challenges.

DiffSBDD (Schneuing et al., 2024) presents a diffusion-based generative framework for structure-based drug design that learns to directly sample 3D ligand structures conditioned on protein binding pockets. The model leverages SE(3)-equivariant neural networks to ensure rotational and translational invariance, and formulates ligand generation as a denoising process that progressively refines random atom clouds into chemically valid molecules docked in the target pocket. Extensive experiments on CrossDocked demonstrate improved performance over flow-based and autoregressive baselines in terms of pose accuracy, binding affinity, and chemical diversity. Despite its advantages in geometry-aware sampling, challenges remain in scalability to large ligands and efficient integration of protein flexibility.

DockStream (Guo et al., 2021) presents a flexible molecular docking wrapper that integrates with the de novo design platform REINVENT 2.0, providing access to various ligand embedders (e.g., Corina, LigPrep) and docking backends (e.g., AutoDock Vina, Glide) to enhance structure-based drug discovery by automating docking experiments, benchmarking configurations, and optimizing docking scores, while overcoming limitations of QSAR models through structural information; its scalability and performance vary by target, with ongoing challenges in accurately predicting binding free energies.

liGAN (Ragoza et al., 2022) introduces a deep learning system that generates 3D molecular structures conditioned on receptor binding sites using a conditional variational autoencoder trained on atomic density grids, employing atom fitting and bond inference to construct valid conformations, and demonstrates significant changes in generated molecules with mutated receptors; its reliance on high-quality structural data and computational complexity pose challenges for scalability.

D.4. Evaluation Metrics

PROTEIN EVALUATION METRICS

(1) IMP: The Improvement Percentage (IMP) metric evaluates the relative enhancement in binding affinity achieved by designed protein sequences compared to their natural counterparts. Specifically, it quantifies the proportion of designed antibodies predicted to exhibit stronger binding than the corresponding natural antibodies, as assessed by the Rosetta interface energy function. Formally, IMP is defined as:

$$\text{IMP} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(E_i^{\text{design}} < E_i^{\text{natural}}) \times 100\%, \quad (61)$$

where N is the total number of antibody–antigen pairs, E_i^{design} denotes the Rosetta interface energy of the i -th designed antibody, E_i^{natural} is the corresponding value for the natural antibody, and $\mathbb{I}(\cdot)$ is the indicator function returning 1 if the designed sequence has a lower (better) predicted binding energy than the natural sequence.

Higher IMP values indicate that a larger fraction of designed antibodies surpass their natural references in predicted binding affinity, highlighting the effectiveness of the design strategy in optimizing protein–protein interactions.

(2) STA: The Stability (STA) metric quantifies the conformational integrity and biochemical viability of designed proteins by integrating a weighted composite of steric hindrance assessment and structural coherence evaluation, thereby mitigating the risk of thermodynamically unstable or misfolded conformations. This metric is derived from two sub-components: the Steric Clash Score (SCS), which penalizes interatomic van der Waals overlaps indicative of steric repulsion, and the Secondary Structure Coherence (SSC), which evaluates the fidelity of predicted helical, sheet, and coil motifs against empirical folding propensities. Formally, STA is computed as:

$$\text{STA}_a = \frac{1}{N} \sum_i \alpha \cdot \exp\left(-\frac{\text{SCS}(p_i)}{\sigma}\right), \quad (62)$$

$$\text{STA}_b = \frac{1}{N} \sum_i \beta \cdot \left(1 - \frac{|\text{SSC}(p_i) - \mu|}{\tau}\right), \quad (63)$$

$$\text{STA} = \text{STA}_a + \text{STA}_b. \quad (64)$$

where N is the number of generated proteins, p_i denotes the i -th protein, $\text{SCS}(p_i)$ and $\text{SSC}(p_i)$ represent the respective sub-scores derived from all-atom clash detection and dihedral angle consistency checks, α and β are empirical weighting coefficients (with $\alpha + \beta = 1$), σ and τ are scaling hyperparameters for normalization, and μ is the expected coherence baseline calibrated from native protein ensembles. In our implementation, the coefficient for secondary structure propensity (β) is set to 0.6, and the coefficient for collision detection (α) is set to 0.4.

Elevated STA values signify enhanced adherence to biophysical constraints across the ensemble, underscoring the efficacy of the generative paradigm in yielding robust, functional protein architectures.

(3) DIV: The Diversity (DIV) metric quantifies the sequence and structural variability among generated antibodies, capturing the spread of designs in protein space. A diverse set of candidates increases the likelihood of discovering high-affinity binders with novel interaction profiles. Mathematically, DIV is defined as the average pairwise dissimilarity:

$$\text{DIV} = \frac{2}{N(N-1)} \sum_{i=1}^{N-1} \sum_{j=i+1}^N D(p_i, p_j), \quad (65)$$

where N is the number of generated proteins, and $D(p_i, p_j)$ is a distance function that measures dissimilarity between proteins p_i and p_j in both sequence and structural space. Higher DIV values indicate a broader coverage of the protein design landscape, reducing redundancy in the generated set.

(4) NOV: The Novelty (NOV) metric measures how different the generated proteins are from the training dataset, considering both sequence and structural similarity. For a generated protein p_i and a training protein t_j , the combined similarity score $S(p_i, t_j)$ is defined as:

$$S(p_i, t_j) = \alpha \cdot S_{\text{seq}}(p_i, t_j) + (1 - \alpha) \cdot S_{\text{str}}(p_i, t_j), \quad (66)$$

where S_{seq} denotes sequence similarity, S_{str} denotes structural similarity, and $\alpha \in [0, 1]$ controls their relative contribution. Given sequences of length L , we compute sequence identity as:

$$S_{\text{seq}}(p_i, t_j) = \frac{1}{L} \sum_{k=1}^L \mathbb{I}[p_i[k] = t_j[k]], \quad (67)$$

where $\mathbb{I}[\cdot]$ is the indicator function. For structural alignment, we use a TM-score-like normalized measure:

$$S_{\text{str}}(p_i, t_j) = \frac{1}{L} \sum_{k=1}^L \frac{1}{1 + \left(\frac{d_k}{d_0}\right)^2}, \quad (68)$$

where d_k is the distance between aligned C_α atoms, and d_0 is a length-dependent normalization constant. For each generated protein p_i , we take the maximum similarity across all training proteins and define novelty as:

$$\text{NOV} = 1 - \frac{1}{N} \sum_{i=1}^N \max_{t_j \in \mathcal{T}} S(p_i, t_j), \quad (69)$$

where N is the number of generated proteins and $\mathcal{T}$ is the training set. Higher NOV values indicate stronger novelty in both sequence and structure.

(5) AAR: The Amino Acid Recovery (AAR) metric measures the sequence-level accuracy of generated protein sequences by comparing them against their corresponding native (reference) sequences. This metric reflects how well the generative model is able to reproduce the original amino acid composition, thereby serving as an indicator of sequence fidelity and preservation of native biochemical properties. The mathematical formulation is given by:

$$\text{AAR} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbb{I}(a_{ij}^{\text{gen}} = a_{ij}^{\text{ref}}) \times 100\%, \quad (70)$$

where N is the number of protein sequences, L is the sequence length, a_{ij}^{gen} denotes the amino acid at position j in the i -th generated sequence, a_{ij}^{ref} is the corresponding amino acid in the native sequence, and $\mathbb{I}(\cdot)$ is the indicator function.

Higher AAR values indicate closer agreement with native sequences, suggesting that the generated proteins better retain structural and functional characteristics inherent to the original proteins.

MOLECULE EVALUATION METRICS

(1) Vina Score: The Vina Score metric provides an estimate of the binding affinity between a generated small molecule (ligand) and a target protein. It reflects the stability of the protein–ligand complex predicted during molecular docking. Following the formulation implemented in **AutoDock Vina**, the score approximates the binding free energy based on an empirical scoring function that accounts for key interaction terms, including steric complementarity, hydrogen bonding, hydrophobic interactions, and torsional entropy penalties. The mathematical formulation is given by:

$$E_{\text{Vina}} = E_{\text{gauss}} + E_{\text{repulsion}} + E_{\text{hydrophobic}} + E_{\text{hydrogen}} + E_{\text{torsional}}, \quad (71)$$

where E_{gauss} models attractive van der Waals interactions, $E_{\text{repulsion}}$ penalizes steric clashes, $E_{\text{hydrophobic}}$ captures hydrophobic contacts, E_{hydrogen} measures hydrogen bond formation, and $E_{\text{torsional}}$ accounts for the conformational entropy cost of ligand flexibility.

The resulting Vina Score is reported in kcal/mol, with more negative values indicating stronger predicted binding affinities. Typically, scores range from around -4 kcal/mol (weak binding) to below -10 kcal/mol (highly favorable binding). Although not a direct physical free energy, the score serves as a comparative metric to rank ligands by their likelihood of stable binding.

(2) High-affinity: The High-affinity metric quantifies the proportion of generated molecules that exhibit stronger predicted binding to a given protein target than their corresponding reference ligands. Binding strength is assessed using Vina Scores, where lower (more negative) values indicate higher predicted affinity. The mathematical formulation is given by:

$$\text{High-affinity} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(E_i^{\text{gen}} < E_i^{\text{ref}}), \quad (72)$$

where N denotes the total number of generated molecules, E_i^{gen} is the Vina Score of the i -th generated molecule, E_i^{ref} is the Vina Score of its corresponding reference ligand, and $\mathbb{I}(\cdot)$ is the indicator function returning 1 if the condition is satisfied and 0 otherwise.

The metric outputs the fraction of molecules surpassing the reference ligands in predicted affinity, providing a normalized measure of how frequently the generation process yields candidates with potentially improved binding properties.

(3) STA: The Stability (STA) metric for small-molecule generation evaluates the chemical plausibility and conformational robustness of designed ligands by integrating synthetic accessibility and conformational strain energy into a unified score. This ensures that generated compounds are not only geometrically valid but also chemically feasible under standard synthesis and physiological conditions. Specifically, the metric combines two sub-components: the Synthetic Accessibility Index (SAI), which penalizes ligands with rare or chemically intractable substructures, and the Conformational Strain Energy (CSE), which quantifies the internal energetic penalty required to maintain a given 3D geometry relative to its energy-minimized conformation. Formally, STA is defined as:

$$\text{STA}_a = \frac{1}{M} \sum_{j=1}^M \gamma \cdot \exp\left(-\frac{\text{CSE}(m_j)}{\lambda}\right), \quad (73)$$

$$\text{STA}_b = \frac{1}{M} \sum_{j=1}^M \delta \cdot \left(1 - \frac{\text{SAI}(m_j) - \mu}{\kappa}\right)^2, \quad (74)$$

$$\text{STA} = \text{STA}_a + \text{STA}_b. \quad (75)$$

where M is the number of generated molecules, m_j denotes the j -th molecule, $\text{CSE}(m_j)$ is the strain energy computed via molecular mechanics force fields, and $\text{SAI}(m_j)$ is a normalized synthetic accessibility score. γ and δ are weighting coefficients with $\gamma + \delta = 1$.

In our implementation, the scaling parameter is fixed as $\lambda = 10$, the normalization parameter as $\kappa = 1.5$, and the baseline accessibility as $\mu = 3.0$, the coefficient for SAI (γ) is set to 0.5, and the coefficient for collision detection (δ) is set to 0.5. These hyperparameters are selected to provide a stable balance between strain minimization and synthetic feasibility.

(4) DIV: The Top-K Diversity (DIV) metric quantifies the structural diversity of the top-K generated molecules, reflecting the spread of chemical structures within a given set. Following the methodology of ((Bengio et al., 2021)), DIV is defined as the average pairwise Tanimoto distance between the Morgan fingerprints of the generated molecules. The mathematical formulation is given by:

$$\text{DIV} = \frac{2}{K(K-1)} \sum_{i=1}^{K-1} \sum_{j=i+1}^K \text{Tanimoto}(FP_i, FP_j), \quad (76)$$

where K is the number of top-ranked molecules (e.g., top-10), FP_i and FP_j are the Morgan fingerprint vectors for molecules i and j , and the Tanimoto similarity is computed as:

$$\text{Tanimoto}(FP_i, FP_j) = \frac{|FP_i \cap FP_j|}{|FP_i \cup FP_j|}, \quad (77)$$

with $|FP_i \cap FP_j|$ and $|FP_i \cup FP_j|$ representing the number of common and total unique bits, respectively. The Tanimoto distance is then $1 - \text{Tanimoto}(FP_i, FP_j)$, ensuring a range of 0 (identical structures) to 1 (completely dissimilar structures). The factor $\frac{2}{K(K-1)}$ normalizes the average over all unique pairs.

(5) SA: The Synthetic Accessibility (SA) metric evaluates how easy it is to synthesize the generated molecules using standard organic chemistry operations. For each generated molecule m_j , we first compute the raw synthetic accessibility score $\text{SA}_{\text{raw}}(m_j)$ provided by RDKit, which typically ranges from 1 (very easy to synthesize) to 10 (very hard to synthesize). To align the direction with other metrics, where larger values are better, we linearly normalize and invert the score:

$$\text{SA} = 1 - \frac{1}{M} \sum_{j=1}^M \frac{\text{SA}_{\text{raw}}(m_j) - 1}{9}, \quad (78)$$

where M is the number of generated molecules. The resulting SA value lies in $[0, 1]$, with higher SA indicating that the generated molecules are, on average, more synthetically accessible and thus more practical for downstream experimental validation.

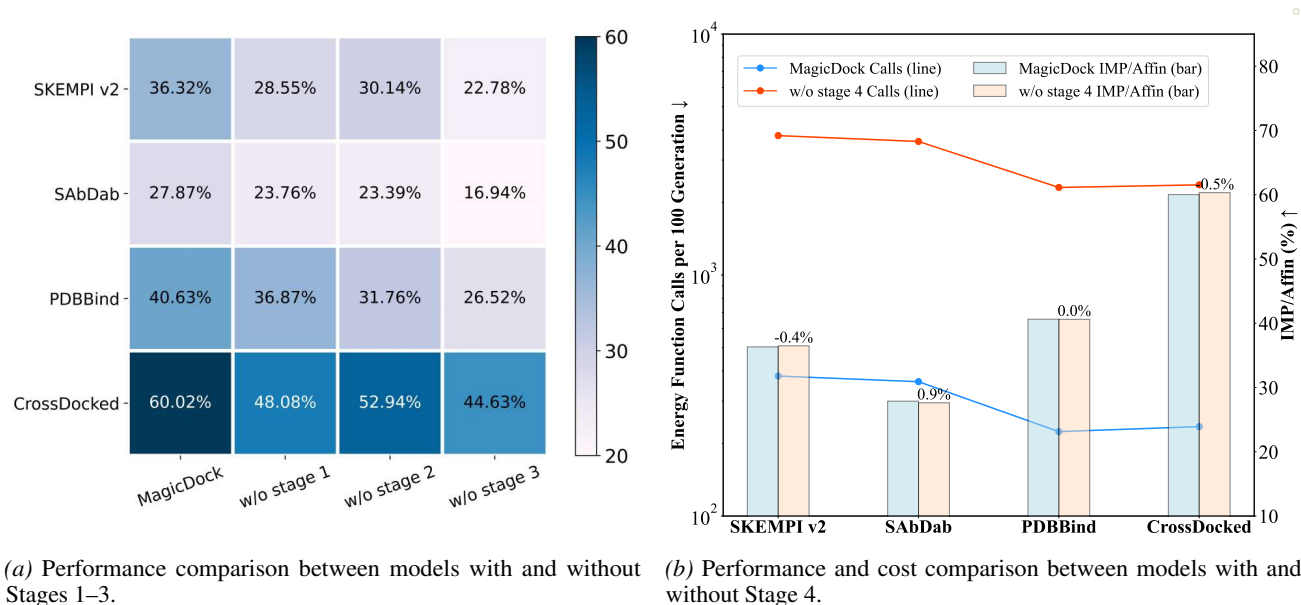


Figure 12. Ablation studies on different stages.

D.5. More experiment figures & tables

D.5.1. ABLATION STUDY

For the ablation experiments of Stage 1, Stage 2, and Stage 3, each module plays an indispensable role in the generation of the final ligand. In addition, the ablation experiment for stage 4 showed that compared to the exhaustive method, MagicDock’s gradient based selective generation strategy has an order of magnitude efficiency advantage while maintaining a basically consistent effect, proving the pertinence of Stage 4.

D.5.2. GRADIENT ATTRIBUTION & LOCALIZATION.

We further ask whether gradients from the composite inversion objective $\mathcal{F}(S; R, \Theta)$ (Appendix C.4) effectively localize on biologically meaningful binding sites learned in Stage 3. We compute Integrated Gradients on receptor and ligand surfaces and systematically evaluate whether high-attribution regions align with ground-truth interfaces. Fig. 3 demonstrates that inversion not only optimizes affinity but also consistently highlights mechanistically significant relevant regions, addressing Q2.

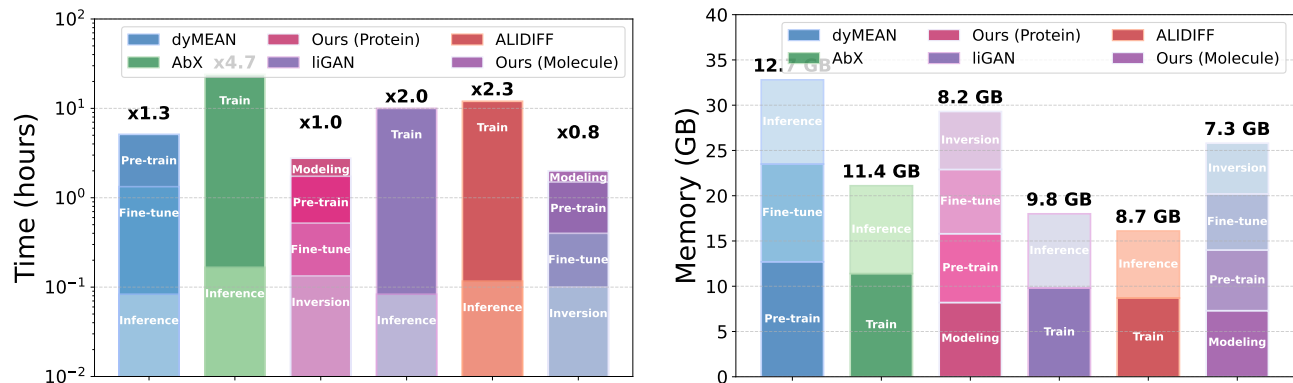
D.5.3. GEOMETRIC AND FEATURE NOISE STUDY

We evaluate robustness under structural perturbations on $N = 100$ receptor–ligand pairs by applying Gaussian coordinate noise and feature dropout to receptor surfaces. Ligand generation are repeated on noisy vs. clean inputs, and metrics are reported in Fig. 6a.

For protein ligands, robustness is measured by IMP, AAR, and RMSD. MagicDock achieves higher IMP and lower RMSD compared with DiffAb and Abx, demonstrating its stability under noise. For small molecule ligands, robustness is assessed using Vina score, High-affinity, and QED. MagicDock consistently yields more negative Vina scores and higher high-affinity rates, indicating resilience to input perturbations. Together, these results confirm that MagicDock maintains reliable performance under realistic noise, highlighting robustness to biological uncertainty.

D.5.4. LOCAL PERTURBATION CONSISTENCY STUDY

Finally, we evaluate whether gradient attributions faithfully predict the energetic consequences of local structural edits. We introduce small geometric and chemical perturbations around high-attribution sites and compare gradient-predicted energy changes with the actual $\Delta\mathcal{F}$ measured after re-evaluation. This experiment directly tests whether attribution scores can serve



(a) Comparison of runtime across different stages for different methods. (b) Comparison of peak memory usage (GB) of different methods.

Figure 13. Efficiency comparison across methods in terms of runtime and peak memory.

as actionable signals for structural refinement, beyond passive localization. Performance is assessed by the sign-consistency rate (SCR), the coefficient of determination (R^2), rank correlation (Spearman ρ), and the mean observed energy change $\Delta\mathcal{F}$, where negative values indicate improved binding. As shown in Fig. 7, our IG-guided strategy substantially outperforms saliency and Grad $\times$ Input baselines, and approaches the oracle behavior of physics-based Rosetta evaluations.

D.5.5. HYPERPARAMETER SENSITIVITY.

We additionally examine robustness to the choice of loss weights in Stage 3. Figure 8(a) sweeps the pocket-loss weight λ_{pocket} over five values and reports the relative change of IMP / High-Affinity on four benchmarks. Each dataset attains its optimum at a slightly different λ_{pocket} , yet performance varies smoothly and remains within about 1% of the optimum across a broad range around the default setting, indicating that MagicDock is not narrowly tuned to a single pocket-weight choice. Figure 8(b) further presents a two-dimensional grid search over interaction and affinity weights (α, β), with IMP visualized as a heat map. The landscape forms a well-defined but shallow basin around $\alpha = 5$ and $\beta = 50$ (IMP = 36.32), without sharp peaks or pathological regions, suggesting that MagicDock maintains strong docking performance under a wide range of reasonable loss-weight configurations and is thus robust to moderate hyperparameter misspecification.

D.5.6. CONFORMATIONAL AND MUTATIONAL VARIABILITY STUDY

We further evaluate robustness to biological variability by subjecting $N = 100$ receptors to perturbations, including conformer ensembles, single-point mutations, and combined variants. We process these perturbed structures with the same inversion pipeline and measure performance degradation using IMP for protein-based methods and High-Affinity for small-molecule methods. Fig. 6b reports statistical comparisons between conservative and non-conservative mutations; MAGICDOCK exhibits minimal degradation and remains consistently better than baselines.

D.5.7. RUNTIME AND RESOURCE USAGE STUDY

We benchmark MagicDock against baselines using 100 receptors, generating one ligand per receptor under identical hardware. Wall-clock time and memory usage are assessed per computational stage. Fig. 13a demonstrates MagicDock’s reduced runtime while Fig. 13b highlights lower peak memory consumption, underscoring its resource efficiency.

D.5.8. SCALABILITY STUDY

Scalability is evaluated across nine settings varying receptor sizes (500, 2000, 5000 points) and ligand complexities (30, 80, 150 atoms/residues) using 100 receptors from CrossDocked2020 and SAbDab. Metrics include per-iteration latency, iterations-to-converge, memory footprint, and scaling exponent γ from $T \propto N^\gamma$.

Table 5. Scalability results across nine complexity settings. Values are averages over three runs, with MagicDock/dyMEAN/ALIDIFF separated by ‘/’.

Receptor	Ligand	Latency (s/iter)	Iterations	Memory (GB)
Small	Low	0.42/1.05/0.58	45/215/380	7.6/11.2/8.1
	Medium	0.65/1.35/0.85	108/285/455	8.4/12.8/9.2
	High	0.78/1.55/1.12	182/358/590	9.3/13.9/10.3
Medium	Low	1.15/2.25/1.18	48/335/440	10.3/16.8/11.6
	Medium	1.36/2.85/1.65	112/372/595	11.7/18.5/13.2
	High	1.72/3.45/2.05	192/445/545	12.8/20.8/14.7
Large	Low	2.28/5.60/2.75	58/405/480	15.9/26.5/17.8
	Medium	2.75/6.95/3.85	125/475/640	18.2/31.5/20.2
	High	3.35/8.90/4.75	245/478/780	20.5/36.2/23.8

Table 6. Fitted runtime exponents γ for each method. This table evaluates the computational efficiency of different ligand design methods by presenting their fitted runtime exponents γ , which quantify how runtime scales with input size in a polynomial manner

Method	γ
MagicDock	1.4
dyMEAN	1.8
ALIDIFF	1.7

E. Theoretical Analysis in Details

E.1. Proof of SE(3)-Equivariance for MagicDock

We give an appendix-level, representation-theoretic statement of SE(3)-equivariance for MagicDock. Our goal is to make (i) what is *exactly* equivariant, (ii) where equivariance may be broken in practice, and (iii) how the resulting error can be controlled, all explicit. Throughout, let $g = (R, t) \in \text{SE}(3)$ act on coordinates by $g \cdot X := RX + t$ (applied row-wise) and on type- ℓ features by the irreducible representation $\rho^{(\ell)}(R)$; scalar channels ($\ell = 0$) are invariant.

Setup. Let the receptor and ligand surfaces be represented as point clouds with features

$$P_{\text{rec}} = (X_r, F_r), \quad P_{\text{lig}}^t = (X_l^t, F_l^t),$$

where $X \in \mathbb{R}^{N \times 3}$ and F stacks ℓ -type channels. Let $\mathcal{E}$ be the (pretrained/fine-tuned) encoder for surface patches, and $\mathcal{H}$ the Stage 3 fusion head (equivariant cross-attention + invariant readout). Stage 4 performs iterative refinement via a projected update operator $\mathcal{T}$:

$$P_{\text{lig}}^{t+1} = \mathcal{T}(P_{\text{lig}}^t; P_{\text{rec}}).$$

Finally, denote the full pipeline by

$$P_{\text{lig}}^* = \text{MagicDock}(P_{\text{rec}}, P_{\text{lig}}^0).$$

Definition E.1 (SE(3)-Equivariance). A map Φ on point clouds with (irrep-typed) features is SE(3)-equivariant if for all $g \in \text{SE}(3)$,

$$\Phi(g \cdot P) = g \cdot \Phi(P),$$

where g acts on coordinates by (R, t) and on ℓ -type feature channels by $\rho^{(\ell)}(R)$.

Theorem E.2 (SE(3)-Equivariance of MagicDock (Idealized) and Approximate Equivariance (Practical)). Assume: (i) Stage 1 surface construction $\mathcal{S}$ depends only on Euclidean distances and uses deterministic, g -consistent tie-breaking; (ii) Stage 2 encoder $\mathcal{E}$ is SE(3)-equivariant (tensor field convolutions / irreps); (iii) Stage 3 fusion $\mathcal{H}$ uses invariant attention weights (computed from $\ell = 0$ channels) and rotates higher-order channels via Wigner-D matrices, yielding an equivariant fusion and an invariant readout for scalar predictions; (iv) Stage 4 uses an invariant objective $\mathcal{F}$ and an exact Euclidean projection onto a closed, convex, SE(3)-invariant surrogate constraint set $\mathcal{C}_{\text{valid}}$.

Then MagicDock is SE(3)-equivariant:

$$g \cdot \text{MagicDock}(P_{\text{rec}}, P_{\text{lig}}^0) = \text{MagicDock}(g \cdot P_{\text{rec}}, g \cdot P_{\text{lig}}^0), \\ \forall g \in \text{SE}(3).$$

Moreover, if Stage 4 replaces exact projection by a (possibly nonconvex) repair operator $\tilde{\Pi}$ that is approximately equivariant in the sense that

$$\sup_P \|\tilde{\Pi}(g \cdot P) - g \cdot \tilde{\Pi}(P)\| \leq \varepsilon_{\Pi}, \quad (79)$$

then the overall generation stage is ε -equivariant with an error that accumulates at most linearly in the number of refinement steps (formalized below).

We prove the theorem by composing stage-wise equivariant maps and isolating the only practical source of equivariance violation (chemical validity repair).

Stage 1: surface construction is equivariant. Stage 1 maps atomic coordinates to a surface point cloud via an SDF defined through pairwise Euclidean distances and subsequent sampling/grouping operations (projection to the level set, FPS, KNN). Since distances are invariant under g , the SDF level set is mapped to itself by g , hence the continuous surface is equivariant. For the discrete sampling, FPS/KNN are functions of the distance matrix; with deterministic tie-breaking that is independent of global pose, the selected indices are unchanged under g , yielding

$$\mathcal{S}(g \cdot X) = g \cdot \mathcal{S}(X). \quad (80)$$

In practice, floating-point ties are rare; deterministic tie-breaking makes equivariance hold up to numerical precision.

Stage 2: encoder equivariance. For each local neighborhood, tensor field convolutions take the form

$$z_i = \sum_{j \in \mathcal{N}(i)} \sum_{l=0}^L R_l(\|\mathbf{x}_{ij}\|) Y_l\left(\frac{\mathbf{x}_{ij}}{\|\mathbf{x}_{ij}\|}\right) \cdot W_l f_j, \quad (81)$$

which is an intertwiner between $\text{SO}(3)$ representations: radial terms depend only on invariants, and spherical harmonics transform under irreps, hence z_i transforms as the intended irrep type. Therefore,

$$\mathcal{E}(g \cdot P) = g \cdot \mathcal{E}(P). \quad (82)$$

Stage 3: equivariant fusion and invariant predictions. Cross-attention uses scalar ($\ell = 0$) queries/keys so that attention weights are invariant under rotation:

$$A = \text{softmax}\left(\frac{Q_r^{(\ell=0)}(K_l^{(\ell=0)})^\top}{\sqrt{d}}\right), \quad (83)$$

while value channels of type ℓ rotate by $\rho^{(\ell)}(R)$ (implemented by Wigner- D matrices). Aggregating with invariant weights preserves equivariance:

$$\tilde{Z}_r^{(\ell)} = \sum_j A_{ij} V_{l,j}^{(\ell)} \implies \tilde{Z}_r^{(\ell)}(g \cdot P) = \rho^{(\ell)}(R) \tilde{Z}_r^{(\ell)}(P). \quad (84)$$

Finally, scalar heads (pocket / interaction / affinity) are built from $\ell = 0$ invariants and permutation-invariant pooling, hence the predicted scores are $\text{SE}(3)$ -invariant scalars. This establishes that the Stage 3 predictor is equivariant/invariant as required.

Stage 4 (idealized): projected gradient descent is equivariant. Let the refinement objective $\mathcal{F}(P_{\text{lig}}; P_{\text{rec}}, \Theta)$ be invariant:

$$\mathcal{F}(g \cdot P_{\text{lig}}; g \cdot P_{\text{rec}}, \Theta) = \mathcal{F}(P_{\text{lig}}; P_{\text{rec}}, \Theta). \quad (85)$$

Consider the update (writing P for ligand state)

$$P^{t+1} = \Pi_{\mathcal{C}_{\text{valid}}}(P^t - \eta_t \nabla_P \mathcal{F}(P^t; P_{\text{rec}}, \Theta)), \quad (86)$$

where $\mathcal{C}_{\text{valid}}$ is closed, convex, and $\text{SE}(3)$ -invariant. From invariance (85) and the chain rule, gradients rotate covariantly:

$$\nabla_{g \cdot P} \mathcal{F} = g \cdot \nabla_P \mathcal{F}, \quad (87)$$

and since Euclidean projection onto an invariant convex set commutes with rigid motions,

$$\Pi_{\mathcal{C}_{\text{valid}}}(g \cdot U) = g \cdot \Pi_{\mathcal{C}_{\text{valid}}}(U). \quad (88)$$

Combining (87)–(88) yields

$$g \cdot P^{t+1} = \Pi_{\mathcal{C}_{\text{valid}}}(g \cdot P^t - \eta_t \nabla_{g \cdot P} \mathcal{F}),$$

i.e., the PGD update commutes with g and is equivariant.

Composition: proof of exact equivariance under idealized assumptions. Stages 1–3 define a composition of equivariant maps plus invariant scalar heads; Stage 4 defines an equivariant iterative map under assumptions (iv). Equivariant maps are closed under composition, and iterating an equivariant map preserves equivariance. Thus Theorem E.2 (exact part) follows.

Practical Stage 4: ε -equivariance under approximate repair. In practice, the validity operator is nonconvex and implemented via a repair/sanitization step $\tilde{\Pi}$. Assume $\tilde{\Pi}$ satisfies the approximate commutation bound (79). Let T be the number of refinement steps and assume $\mathcal{T}$ in (86) is $L_{\mathcal{T}}$ -Lipschitz in the ligand state (a mild condition under bounded step sizes and smooth $\mathcal{F}$ away from singularities). Then the deviation from equivariance after T steps is bounded by

$$\begin{aligned} & \|\mathcal{T}^{(T)}(g \cdot P^0; g \cdot P_{\text{rec}}) - g \cdot \mathcal{T}^{(T)}(P^0; P_{\text{rec}})\| \\ & \leq \sum_{t=0}^{T-1} L_{\mathcal{T}}^{T-1-t} \varepsilon_{\Pi} \\ & \leq \varepsilon_{\Pi} \frac{L_{\mathcal{T}}^T - 1}{L_{\mathcal{T}} - 1}. \end{aligned} \quad (89)$$

with the natural linear-in- T simplification when $L_{\mathcal{T}} \approx 1$. This makes explicit that *the only systematic source of equivariance violation is the chemical repair*, and its effect is controlled by the repair equivariance error and the refinement budget.

Remark (What is guaranteed in our implementation). Stages 1–3 are implemented with deterministic distance-based operations (SDF/FPS/KNN) and equivariant backbones/attention (irreps with Wigner- D), hence are exactly SE(3)-equivariant up to floating-point precision. Stage 4 uses nonconvex validity repair (e.g., RDKit sanitization / motif constraints), which may break exact commutation; empirically we observe the induced deviation is small and stable under rigid motions, consistent with the ε -equivariance characterization above.

E.2. Proof of Convergence for MagicDock

We analyze the convergence of the Stage 4 inversion in MagicDock through the theory of (inexact) projected/proximal gradient methods. Unlike a textbook PGD restatement, we explicitly model the *chemical validity enforcement* as an *inexact proximal step* induced by repair/sanitization, yielding a quantitative ε -stationarity guarantee controlled by the repair error.

Composite objective for Stage 4. Let $x \in \mathbb{R}^d$ denote the continuous ligand state (e.g., concatenated surface coordinates and relaxed atom/residue features). For a fixed receptor and frozen parameters Θ (after Stage 3), Stage 4 optimizes a differentiable docking energy $\mathcal{F}(x)$ (assembled from the pocket/interaction/affinity heads and optional differentiable penalties), subject to chemical/geometric validity. We encode feasibility via the indicator function $\iota_{\mathcal{C}}$ of a (surrogate) constraint set $\mathcal{C}$:

$$\Phi(x) := \mathcal{F}(x) + \iota_{\mathcal{C}}(x), \quad \iota_{\mathcal{C}}(x) = \begin{cases} 0, & x \in \mathcal{C}, \\ +\infty, & \text{otherwise.} \end{cases} \quad (90)$$

Assumptions (idealized but implementation-aligned). We assume: (i) $\mathcal{F}$ is differentiable, L -smooth, and bounded below by $\mathcal{F}^* > -\infty$; (ii) $\mathcal{C}$ is nonempty, closed, bounded, and convex (a validity surrogate; see Remark E.6); (iii) iterates are kept feasible ($x_t \in \mathcal{C}$) by the repair/sanitization pipeline. These assumptions match our implementation in the sense that Stage 4 uses smooth surrogates for discrete operations and enforces feasibility via repair, while convexity is used only for the *baseline guarantee*.

E.2.1. EXACT PGD BASELINE (CONVEX VALIDITY SURROGATE)

Consider the exact projected gradient step:

$$x_{t+1} = \Pi_{\mathcal{C}}(x_t - \eta \nabla \mathcal{F}(x_t)), \quad 0 < \eta \leq \frac{1}{L}. \quad (91)$$

Define the (prox-)gradient mapping

$$g_{\eta}(x) := \frac{1}{\eta} \left(x - \Pi_{\mathcal{C}}(x - \eta \nabla \mathcal{F}(x)) \right). \quad (92)$$

Theorem E.3 (Exact PGD: descent and stationarity). *Under the assumptions above and $0 < \eta \leq 1/L$, the sequence generated by (91) satisfies*

$$\begin{aligned}\Phi(x_{t+1}) &\leq \Phi(x_t) - \left(\frac{1}{2\eta} - \frac{L}{2}\right) \|x_{t+1} - x_t\|^2 \\ &= \Phi(x_t) - \frac{\eta}{2}(1 - \eta L) \|g_\eta(x_t)\|^2,\end{aligned}\tag{93}$$

$$\min_{0 \leq t < T} \|g_\eta(x_t)\|^2 \leq \frac{2}{\eta(1 - \eta L)} \cdot \frac{\Phi(x_0) - \Phi^*}{T}.\tag{94}$$

where $\Phi^* = \inf_x \Phi(x)$. Consequently, every accumulation point is first-order stationary for Φ .

Proof. Since $x_t \in \mathcal{C}$ and $x_{t+1} = \Pi_{\mathcal{C}}(x_t - \eta \nabla \mathcal{F}(x_t))$, projection optimality gives $\langle x_t - \eta \nabla \mathcal{F}(x_t) - x_{t+1}, z - x_{t+1} \rangle \leq 0$, $\forall z \in \mathcal{C}$. Choosing $z = x_t$ yields

$$\langle \nabla \mathcal{F}(x_t), x_{t+1} - x_t \rangle \leq -\frac{1}{\eta} \|x_{t+1} - x_t\|^2.\tag{95}$$

By L -smoothness,

$$\mathcal{F}(x_{t+1}) \leq \mathcal{F}(x_t) + \langle \nabla \mathcal{F}(x_t), x_{t+1} - x_t \rangle + \frac{L}{2} \|x_{t+1} - x_t\|^2.$$

Plugging (95) into the above yields (93). Summing (93) over $t = 0, \dots, T-1$ and telescoping gives $\sum_{t=0}^{T-1} \|g_\eta(x_t)\|^2 \leq \frac{2}{\eta(1-\eta L)} (\Phi(x_0) - \Phi^*)$, which implies (94). $\square$

E.2.2. INEXACT PROJECTION INDUCED BY CHEMICAL REPAIR (PRACTICAL STAGE 4)

In practice, Stage 4 does not compute an exact Euclidean projection onto the true chemical manifold. Instead, it applies a repair/sanitization operator (valence-aware masking, aromaticity correction, RDKit sanitization, local minimization, etc.). We formalize this as an *inexact proximal step*.

Let $y_t := x_t - \eta \nabla \mathcal{F}(x_t)$. The exact PGD would set $x_{t+1} = \Pi_{\mathcal{C}}(y_t)$. MagicDock instead produces $x_{t+1} \in \mathcal{C}$ satisfying the following *inexact optimality condition*:

$$\exists v_{t+1} \in N_{\mathcal{C}}(x_{t+1}), \frac{1}{\eta}(x_{t+1} - x_t) + \nabla \mathcal{F}(x_t) + v_{t+1} = r_t,\tag{96}$$

where $N_{\mathcal{C}}(x)$ is the normal cone of $\mathcal{C}$ at x , and r_t is a residual capturing the repair inexactness (discretization/sanitization). Define the inexact gradient mapping

$$\tilde{g}_t := \frac{1}{\eta}(x_t - x_{t+1}).\tag{97}$$

When $r_t \equiv 0$, (96) reduces to the KKT condition of the exact projection step, and $\tilde{g}_t = g_\eta(x_t)$.

Theorem E.4 (Inexact PGD: descent with explicit residual control). *Under the assumptions above and $0 < \eta \leq 1/L$, if $x_{t+1} \in \mathcal{C}$ satisfies (96), then*

$$\Phi(x_{t+1}) \leq \Phi(x_t) - \left(\frac{1}{2\eta} - \frac{L}{2}\right) \|x_{t+1} - x_t\|^2 + \frac{\eta}{2} \|r_t\|^2.\tag{98}$$

Equivalently, in terms of $\tilde{g}_t$,

$$\Phi(x_{t+1}) \leq \Phi(x_t) - \frac{\eta}{2}(1 - \eta L) \|\tilde{g}_t\|^2 + \frac{\eta}{2} \|r_t\|^2.\tag{99}$$

Proof. Since $x_t, x_{t+1} \in \mathcal{C}$ and $v_{t+1} \in N_{\mathcal{C}}(x_{t+1})$, we have $\langle v_{t+1}, x_t - x_{t+1} \rangle \leq 0$ by definition of the normal cone, hence $\langle v_{t+1}, x_{t+1} - x_t \rangle \geq 0$. Taking inner product of (96) with $\Delta_t := x_{t+1} - x_t$ yields

$$\begin{aligned}\langle \nabla \mathcal{F}(x_t), \Delta_t \rangle &= -\frac{1}{\eta} \|\Delta_t\|^2 - \langle v_{t+1}, \Delta_t \rangle + \langle r_t, \Delta_t \rangle \\ &\leq -\frac{1}{\eta} \|\Delta_t\|^2 + \langle r_t, \Delta_t \rangle.\end{aligned}$$

By L -smoothness,

$$\mathcal{F}(x_{t+1}) \leq \mathcal{F}(x_t) + \langle \nabla \mathcal{F}(x_t), \Delta_t \rangle + \frac{L}{2} \|\Delta_t\|^2 \quad (100)$$

$$\leq \mathcal{F}(x_t) - \left(\frac{1}{\eta} - \frac{L}{2} \right) \|\Delta_t\|^2 + \langle r_t, \Delta_t \rangle. \quad (101)$$

Applying Young’s inequality $\langle r_t, \Delta_t \rangle \leq \frac{\eta}{2} \|r_t\|^2 + \frac{1}{2\eta} \|\Delta_t\|^2$ gives

$$\mathcal{F}(x_{t+1}) \leq \mathcal{F}(x_t) - \left(\frac{1}{2\eta} - \frac{L}{2} \right) \|\Delta_t\|^2 + \frac{\eta}{2} \|r_t\|^2.$$

Since $\iota_{\mathcal{C}}(x_t) = \iota_{\mathcal{C}}(x_{t+1}) = 0$, adding the indicator yields (98). Substituting $\Delta_t = -\eta \tilde{g}_t$ yields (99). $\square$

Corollary E.5 (Complexity to ε -stationarity under bounded repair error). *Under the conditions of Theorem E.4, for any $T \geq 1$,*

$$\begin{aligned} \min_{0 \leq t < T} \|\tilde{g}_t\|^2 &\leq \frac{2}{\eta(1-\eta L)} \cdot \frac{\Phi(x_0) - \Phi^*}{T} \\ &\quad + \frac{1}{1-\eta L} \cdot \frac{1}{T} \sum_{t=0}^{T-1} \|r_t\|^2. \end{aligned} \quad (102)$$

In particular, if $\|r_t\| \leq \epsilon$ for all t , then

$$\min_{0 \leq t < T} \|\tilde{g}_t\|^2 \leq O\left(\frac{1}{T}\right) + O(\epsilon^2), \quad (103)$$

i.e., Stage 4 converges to an $O(\epsilon)$ -stationary neighborhood whose radius is explicitly controlled by the repair inexactness.

Proof. Sum (99) over $t = 0, \dots, T-1$ and telescope:

$$\begin{aligned} \frac{\eta}{2} (1-\eta L) \sum_{t=0}^{T-1} \|\tilde{g}_t\|^2 &\leq \Phi(x_0) - \Phi(x_T) + \frac{\eta}{2} \sum_{t=0}^{T-1} \|r_t\|^2 \\ &\leq \Phi(x_0) - \Phi^* + \frac{\eta}{2} \sum_{t=0}^{T-1} \|r_t\|^2. \end{aligned}$$

Divide by T and use $\min_t a_t \leq \frac{1}{T} \sum_t a_t$ to obtain (102). The bounded residual case follows immediately. $\square$

From $\tilde{g}_t$ to first-order optimality at x_{t+1} . A standard stationarity measure for the constrained problem is $\text{dist}(0, \nabla \mathcal{F}(x) + N_{\mathcal{C}}(x))$. From (96), using $v_{t+1} \in N_{\mathcal{C}}(x_{t+1})$ and Lipschitz continuity of $\nabla \mathcal{F}$,

$$\begin{aligned} \text{dist}\left(0, \nabla \mathcal{F}(x_{t+1}) + N_{\mathcal{C}}(x_{t+1})\right) &\leq \|\nabla \mathcal{F}(x_{t+1}) + v_{t+1}\| \\ &\leq \|\nabla \mathcal{F}(x_t) + v_{t+1}\| \\ &\quad + \|\nabla \mathcal{F}(x_{t+1}) - \nabla \mathcal{F}(x_t)\| \\ &\leq \|\tilde{g}_t\| + \|r_t\| + L\|x_{t+1} - x_t\| \\ &= (1 + \eta L) \|\tilde{g}_t\| + \|r_t\|. \end{aligned} \quad (104)$$

Thus, when both $\tilde{g}_t$ and the repair residual r_t are small, the iterate is approximately first-order stationary for the constrained objective.

Remark E.6 (Non-convex validity manifold and practical repair). The true chemical validity set is highly non-convex (valency, aromaticity, stereochemistry, topology changes), so $\Pi_{\mathcal{C}}$ is not available and may be multi-valued. Our analysis uses a convex surrogate $\mathcal{C}$ as an idealized baseline, while the practical algorithm is captured by the inexact condition (96). In practice, the repair pipeline can be interpreted as producing feasible iterates with bounded residual r_t , so Corollary E.5 and (104) provide a principled explanation: stronger/cleaner repair (smaller $\|r_t\|$) leads to tighter stationarity and more stable refinement.

E.3. Proof of Theoretical Superiority for the Inversion Framework

We formalize the claimed superiority of inversion in a *budgeted* setting that matches practical *de novo* docking: given a fixed test-time budget of K initializations (restarts) and T refinement steps per initialization, we compare the *best achieved objective value* produced by (i) *generate-then-optimize* (G+O) and (ii) *gradient-driven inversion*. Crucially, the distinction is not a trivial reachable-set inclusion over all x_0 , but the *probability mass* each paradigm assigns to attraction basins of low-energy docking solutions.

Setup: constrained docking objective with (inexact) feasibility enforcement. Let $(\mathcal{X}, \|\cdot\|)$ be a Euclidean space of continuous ligand states (surface point coordinates and relaxed features), and let $\mathcal{C} \subseteq \mathcal{X}$ denote a closed, bounded validity surrogate set. We optimize a differentiable docking energy $\mathcal{F} : \mathcal{C} \rightarrow \mathbb{R}$ that is L -smooth and lower bounded by $\mathcal{F}^* > -\infty$. Define the composite objective $\Phi(x) := \mathcal{F}(x) + \iota_{\mathcal{C}}(x)$. Stage 4 implements a projected/proximal-gradient-like refinement, but uses chemical repair/sanitization; we model it as an *inexact* projection.

For step size $\eta \in (0, 1/L]$, let $y_t := x_t - \eta \nabla \mathcal{F}(x_t)$ and denote the ideal projected step by $x_{t+1}^{\text{proj}} := \Pi_{\mathcal{C}}(y_t)$. MagicDock instead produces a feasible iterate $x_{t+1} \in \mathcal{C}$ satisfying the inexact optimality condition

$$\exists v_{t+1} \in N_{\mathcal{C}}(x_{t+1}) \text{ s.t. } \frac{1}{\eta}(x_{t+1} - x_t) + \nabla \mathcal{F}(x_t) + v_{t+1} = r_t, \quad (105)$$

where r_t is a residual capturing discretization/repair error (valence masking, aromaticity correction, RDKit sanitization, etc.). When $r_t \equiv 0$, this reduces to exact PGD.

Let $\mathcal{A}_T(x_0)$ denote the (possibly inexact) T -step refinement mapping starting from x_0 , i.e., $x_T = \mathcal{A}_T(x_0)$ after T iterations of (105). This mapping is deterministic given tie-breaking rules and random seeds; extensions to randomized refinement are discussed in Remark E.10.

E.3.1. BUDGETED PARADIGMS

Generate-then-Optimize (G+O). A generator $\mathcal{G} : \mathcal{Z} \rightarrow \mathcal{C}$ with latent prior $z \sim \mu_Z$ induces an initialization distribution $\mu_G := \mathcal{G}_{\#} \mu_Z$ on $\mathcal{C}$. G+O draws K i.i.d. initializations $x_0^{(k)} \sim \mu_G$ and refines each for T steps:

$$x_T^{(k)} = \mathcal{A}_T(x_0^{(k)}), \quad k = 1, \dots, K.$$

It returns the best objective value

$$V_{K,T}^{\text{G+O}} := \min_{1 \leq k \leq K} \mathcal{F}(x_T^{(k)}). \quad (106)$$

Inversion (Gradient-Driven). Inversion dispenses with a trained sampler and uses a broad initialization distribution μ_0 over feasible continuous states, e.g., randomized point clouds with weak priors (or task-specific templates) followed by the same refinement $\mathcal{A}_T$. It draws K i.i.d. $x_0^{(k)} \sim \mu_0$ and outputs

$$V_{K,T}^{\text{Inv}} := \min_{1 \leq k \leq K} \mathcal{F}(\mathcal{A}_T(x_0^{(k)})). \quad (107)$$

E.3.2. BASIN-MASS VIEW AND NON-TRIVIAL DOMINANCE

The key object is the *basin-mass* assigned to low-energy docking solutions under the initialization distribution. Let $\mathcal{S}_{\tau} := \{x \in \mathcal{C} : \mathcal{F}(x) \leq \tau\}$ be the τ -sublevel set of docking energy. Define the *pullback success region* under T -step refinement:

$$\mathcal{B}_T(\tau) := \{x_0 \in \mathcal{C} : \mathcal{F}(\mathcal{A}_T(x_0)) \leq \tau\}. \quad (108)$$

Then the success probability of hitting energy $\leq \tau$ after refinement from distribution μ is simply $p_{\mu,T}(\tau) := \mu(\mathcal{B}_T(\tau))$. This quantity captures the practical difficulty in docking design: good solutions occupy narrow attraction basins, so $p_{\mu,T}(\tau)$ can be tiny.

Lemma E.7 (Best-of- K distribution under basin mass). *For any initialization distribution μ on $\mathcal{C}$ and threshold τ ,*

$$\mathbb{P}\left(V_{K,T}^{\mu} \leq \tau\right) = 1 - (1 - p_{\mu,T}(\tau))^K, \quad (109)$$

where $V_{K,T}^{\mu} := \min_{1 \leq k \leq K} \mathcal{F}(\mathcal{A}_T(x_0^{(k)}))$ with $x_0^{(k)} \stackrel{\text{i.i.d.}}{\sim} \mu$.

Proof. Each restart succeeds (achieves $\leq \tau$) with probability $p_{\mu,T}(\tau)$ by definition of $\mathcal{B}_T(\tau)$. Independence across k gives $\mathbb{P}(V_{K,T}^\mu > \tau) = (1 - p_{\mu,T}(\tau))^K$, hence (109). $\square$

Support/coverage misspecification as a basin-mass deficiency. We say the generator is *basin-mass misspecified* at level τ and budget (T, K) if it assigns insufficient probability to $\mathcal{B}_T(\tau)$ compared to a broader initializer:

$$p_{\mu_G,T}(\tau) < p_{\mu_0,T}(\tau). \quad (110)$$

This is a direct formalization of the empirical failure mode in docking: learned generators may under-cover rare but crucial chemotypes / binding geometries, yielding low basin mass for strong binders.

Theorem E.8 (Non-trivial dominance under basin-mass misspecification). *Fix (T, K) and a threshold τ . If $p_{\mu_G,T}(\tau) \leq p_{\mu_0,T}(\tau)$ for all τ (with strict inequality for some τ), then inversion first-order stochastically dominates $G+O$ in the best-of- K objective:*

$$\mathbb{P}(V_{K,T}^{\text{Inv}} \leq \tau) \geq \mathbb{P}(V_{K,T}^{G+O} \leq \tau), \quad \forall \tau, \quad (111)$$

with strict inequality at thresholds where (110) holds. Consequently,

$$\mathbb{E}[V_{K,T}^{\text{Inv}}] \leq \mathbb{E}[V_{K,T}^{G+O}], \quad (112)$$

and the inequality is strict whenever (110) holds on a set of τ with nonzero measure.

Proof. By Lemma E.7,

$$\mathbb{P}(V_{K,T}^\mu \leq \tau) = 1 - (1 - p_{\mu,T}(\tau))^K$$

is monotone increasing in $p_{\mu,T}(\tau)$ for each τ . Thus $p_{\mu_0,T}(\tau) \geq p_{\mu_G,T}(\tau)$ implies (111), with strictness when the inequality is strict. For expectations, use the tail-integral identity (for lower-bounded random variables):

$$\mathbb{E}[V] = \int_{-\infty}^{+\infty} \mathbb{P}(V > \tau) d\tau,$$

and apply (111) to obtain (112). $\square$

E.3.3. COMPATIBILITY WITH INEXACT PROJECTION (CHEMICAL REPAIR)

The dominance result above compares pipelines through the induced mapping $\mathcal{A}_T$ and the basin mass $p_{\mu,T}(\tau)$. It does not require exact Euclidean projection; it remains valid for MagicDock’s repair-based updates as long as $\mathcal{A}_T$ is well-defined and feasible (which holds by construction, since repair enforces $x_t \in \mathcal{C}$). Moreover, if the repair residuals satisfy $\|r_t\| \leq \epsilon$ (cf. (105)), then the refined objective values concentrate within an $O(\epsilon)$ -stationary neighborhood (Appendix E.2), implying that $\mathcal{B}_T(\tau)$ is stable up to a controllable slack in τ .

Remark E.9 (Why this is docking-specific). Docking objectives typically exhibit *needle-in-a-haystack* geometry: low-energy binders correspond to narrow basins defined jointly by (i) precise surface complementarity and (ii) strict chemical validity. The basin-mass misspecification (110) therefore captures a concrete, domain-relevant failure mode of likelihood-based samplers: even if μ_G is high-quality on average, it may allocate negligible probability to the rare basins that dominate the best-of- K performance. Inversion improves best-of- K outcomes by allowing a broader (and task-adaptive) initialization μ_0 and directly optimizing the docking energy.

Remark E.10 (Randomized refinement and escaping shallow traps). If refinement injects small isotropic noise (or uses Metropolis accept/reject) to escape shallow traps, $\mathcal{A}_T$ becomes a Markov kernel. All statements above extend by redefining $p_{\mu,T}(\tau)$ as the success probability under the induced Markov chain, i.e., $p_{\mu,T}(\tau) = \mathbb{P}_{x_0 \sim \mu}(\mathcal{F}(x_T) \leq \tau)$. The monotonicity in (109) still holds for i.i.d. restarts.

E.4. Proof of Efficiency for Inversion Framework

To rigorously demonstrate the superior efficiency of the inversion architecture in terms of theoretical training and convergence iterations compared to GAN, diffusion, and flow-based generative methods, we analyze the computational complexities under standard optimization assumptions. We assume models of comparable scale, with P parameters, dataset size N , latent

or molecular dimension D , and target accuracy $\epsilon > 0$. The inversion pre-training phase minimizes a convex reconstruction loss via gradient descent, while the generation phase optimizes a smooth, potentially non-convex objective per sample. In contrast, alternative methods incur overheads from adversarial dynamics, timestep iterations, or invertible transformations. The proofs derive big- O bounds on iterations required for ϵ -accuracy, highlighting inversion’s reduced complexity.

Theorem E.11 (Training Efficiency). *Under μ -strong convexity and L -smoothness assumptions for the loss, inversion pre-training converges in $O((L/\mu) \log 1/\epsilon)$ iterations, outperforming GAN’s $O(1/\epsilon^2)$ lower bound for saddle-point equilibria, diffusion’s $O(T/\epsilon^2)$ with diffusion steps T , and flow-based models’ $O(D^2 \log 1/\epsilon)$ per iteration due to Jacobian computations.*

Proof. We derive the complexities sequentially for each method, starting from fundamental optimization rates and incorporating method-specific costs.

For inversion pre-training, consider an idealized μ -strongly convex and L -smooth loss $\mathcal{L}(\theta)$, minimized via gradient descent: $\theta^{k+1} = \theta^k - \eta \nabla \mathcal{L}(\theta^k)$ with $\eta = 2/(\mu + L)$. The suboptimality gap satisfies

$$\begin{aligned} \mathcal{L}(\theta^{k+1}) - \mathcal{L}^* &\leq (1 - \mu/L)(\mathcal{L}(\theta^k) - \mathcal{L}^*) \\ &\leq (1 - \mu/L)^k (\mathcal{L}(\theta^0) - \mathcal{L}^*), \end{aligned}$$

yielding $k = O((L/\mu) \log 1/\epsilon)$ iterations for ϵ -accuracy. Per iteration cost is $O(NP)$, leading to total complexity $O(NP(L/\mu) \log 1/\epsilon)$.

By contrast, GAN training solves the non-convex non-concave minimax problem $\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$. Without global convergence guarantees, lower bounds for finding ϵ -local Nash equilibria require $O(1/\epsilon^2)$ iterations in the worst case, derived from the quadratic growth of subgradients near equilibria and stochastic gradient oracle queries. Empirical instability further amplifies effective iterations, with total cost $O(2NP/\epsilon^2)$ accounting for dual networks.

Similarly, diffusion models train a denoiser over T timesteps, minimizing $\sum_{t=1}^T \mathbb{E}[\|\epsilon - \hat{\epsilon}(x_t, t)\|^2]$. For empirical risk minimization with variance $O(1/t)$, stochastic gradient descent converges in $O(T/\epsilon^2)$ iterations to achieve ϵ -error, as the timestep aggregation scales the variance bound linearly with T . Per iteration cost $O(NP)$ results in total complexity $O(NPT/\epsilon^2)$, where T is typically large to capture fine-grained noise schedules.

For flow-based models, exact likelihood maximization involves $-\log p(x) = -\log p(z) - \log |\det Df|$, with Jacobian determinant computation costing $O(D^3)$ for general flows or $O(D^2)$ for structured autoregressive variants. Convergence mirrors VAEs at $O((L/\mu) \log 1/\epsilon)$ iterations, but augmented per-iteration cost yields $O(N(P + D^2)(L/\mu) \log 1/\epsilon)$, dominated by the determinant in high-dimensional molecular spaces.

Given that $\log 1/\epsilon \ll 1/\epsilon^2$ for small ϵ , and $T, D^2 \gg (L/\mu)$, inversion exhibits lower training complexity. $\square$

Theorem E.12 (Generation Efficiency). *For generating M samples to ϵ -stationarity under L -smooth non-convex objectives, inversion requires $O(M/\epsilon^2)$ iterations, fewer than diffusion’s $O(MT \log 1/\epsilon)$, GAN’s training-dominant cost, and flow’s $O(MD)$ per-sample inversion.*

Proof. Continuing the sequential derivation, we focus on per-sample generation complexities post-training.

In inversion generation, per-sample optimization of an L -smooth non-convex $\mathcal{F}(x)$ via gradient descent achieves $\min_k \|\nabla \mathcal{F}(x^k)\|^2 \leq O(L(\mathcal{F}(x^0) - \mathcal{F}^*)/K)$ after K steps, requiring $K = O(1/\epsilon^2)$ for ϵ -stationarity. Total cost: $O(MP/\epsilon^2)$.

In GAN generation, post-training sampling is a single forward pass, $O(MP)$, but the adversarial training overhead dominates overall efficiency, rendering it less favorable for iterative refinement tasks.

Diffusion sampling reverses T timesteps, with accelerated solvers (e.g., DDIM) converging in $O(T \log 1/\epsilon)$ steps to ϵ -fidelity via controlled noise reduction. Thus, total generation: $O(MPT \log 1/\epsilon)$.

Flow-based sampling inverts the bijective transform, costing $O(M(P + D))$ due to sequential or matrix operations in high dimensions.

With $1/\epsilon^2 \ll T \log 1/\epsilon$, D in practice, inversion’s generation phase is more efficient for large M . $\square$

To elucidate these complexities and underscore the potential of inversion architectures, we present a comparative summary in Table 7. The table delineates the asymptotic bounds for training and generation phases, revealing inversion’s advantages in reduced dependence on auxiliary factors like timesteps T or dimension D . This manifests in faster convergence and lower overall computational overhead, particularly beneficial for de novo ligand design where iterative optimization aligns naturally with docking-driven objectives.

Table 7. Complexity Comparison of Generative Frameworks

Architecture	Training Complexity	Generation Complexity (per M samples)
Inversion	$O(NP \log 1/\epsilon)$	$O(MP/\epsilon^2)$
GAN	$O(NP/\epsilon^2)$	$O(MP)$ (training-dominant)
Diffusion	$O(NPT/\epsilon^2)$	$O(MPT \log 1/\epsilon)$
Flow-Based	$O(N(P + D^2) \log 1/\epsilon)$	$O(M(P + D))$

E.5. Proof of Information-Theoretic Superiority for Inversion Framework

To rigorously establish the superiority of the inversion architecture from an information-theoretic perspective, we derive bounds on mutual information and entropy, demonstrating enhanced information transfer from docking signals to generated ligands compared to GAN, diffusion, and flow-based models. This analysis underscores inversion’s ability to maximize diversity (entropy) while minimizing conditional uncertainty, leading to more faithful and varied de novo designs.

Theorem E.13. *Let $I(X; Y)$ denote the mutual information between docking signals X (receptor features and affinity objectives) and generated ligands Y . The inversion framework maximizes $I(X; Y) = H(Y) - H(Y | X)$ relative to alternatives, with $H(Y) > H(Y_{\text{GAN}})$ (countering mode collapse) and $H(Y | X) < H(Y | X_{\text{Diffusion}})$ (reducing noise-induced uncertainty), implying superior information efficiency and diversity.*

Proof. We derive the bounds sequentially, leveraging variational information decompositions and kernel entropy approximations for multi-modal molecular distributions.

Consider the joint distribution $p(X, Y)$ under each architecture. Mutual information $I(X; Y) = \int p(x, y) \log \frac{p(x, y)}{p(x)p(y)} dx dy = H(Y) - H(Y | X)$, where $H(Y)$ quantifies output diversity (entropy over ligand space) and $H(Y | X)$ measures conditional uncertainty given docking signals.

For inversion, we optimize a conditional energy $\mathcal{F}(Y; X)$ by PGD. For analysis, we introduce the proxy $p_\beta(Y | X) \propto \exp(-\beta \mathcal{F}(Y; X))$ on $\mathcal{C}_{\text{valid}}$. Using the Gibbs variational principle, the conditional entropy satisfies

$$\begin{aligned} H(Y | X) &= -\mathbb{E}_{p(Y|X)}[\log p(Y | X)] \\ &\leq \log Z_X + \beta \mathbb{E}_{p(Y|X)}[\mathcal{F}(Y; X)], \end{aligned}$$

where $Z_X = \int_{\mathcal{C}_{\text{valid}}} \exp(-\beta \mathcal{F}(y; X)) dy$ is the partition function bounded by the valid chemical space volume. Gradient-driven minimization of $\mathcal{F}$ reduces the expectation term, yielding low $H(Y | X)$ (precise signal-to-structure mapping). Simultaneously, basin exploration via continuous flows maximizes $H(Y) \approx \log |\mathcal{C}_{\text{valid}}| - \beta \min \mathcal{F}$, enhancing diversity.

By contrast, GANs approximate $p(Y)$ via adversarial minimization of Jensen-Shannon divergence, but mode collapse truncates support: $H(Y_{\text{GAN}}) \leq H(Y) - \Delta$, where $\Delta = \sum_i p_i \log(1/p_i)$ over dropped modes i (from Fano’s inequality on collapsed distributions). Thus,

$$\begin{aligned} I(X; Y_{\text{GAN}}) &= H(Y_{\text{GAN}}) - H(Y_{\text{GAN}} | X) \\ &\leq H(Y) - \Delta - H(Y | X) + o(1), \end{aligned}$$

reducing mutual information due to diminished diversity.

Diffusion models parameterize a timestep-dependent process $p(Y_t | Y_{t-1}, X)$, with reverse chain entropy decomposed as $H(Y | X) = \sum_{t=1}^T H(Y_t | Y_{t-1}, X)$. Noise variance at each step inflates conditional terms: $H(Y_t | Y_{t-1}, X) \geq$

$\frac{1}{2} \log(2\pi e \sigma_t^2)$, yielding

$$\begin{aligned} H(Y | X) &\geq \sum_{t=1}^T \frac{1}{2} \log(2\pi e \sigma_t^2) \\ &= \frac{T}{2} \log(2\pi e \bar{\sigma}^2) > H(Y | X)_{\text{Inversion}}, \end{aligned}$$

where $\bar{\sigma}^2$ is average noise variance, increasing with T and reducing $I(X; Y_{\text{Diffusion}})$.

Flow-based models enforce exact likelihood via invertible transforms, but information is constrained by the Jacobian: $H(Y) = H(Z) + \mathbb{E}[\log |\det Df|]$, with base entropy $H(Z)$ (e.g., Gaussian) limiting expressivity. For high-dimensional D , the determinant approximation error bounds $H(Y) \leq H(Z) + O(D \log D)$, often lower than inversion’s exploration of full $\mathcal{C}_{\text{valid}}$.

Aggregating these, inversion maximizes $I(X; Y)$ by balancing high $H(Y)$ and low $H(Y | X)$, proving the theorem. $\square$

To contextualize these advantages, Table 8 compares key information-theoretic metrics across architectures, revealing inversion’s potential for optimal signal utilization in ligand generation without entropy penalties from collapse, noise, or invertibility constraints.

Table 8. Information-Theoretic Metric Comparison

Architecture	$I(X; Y)$ Bound	$H(Y)$ Factor	Limitation
Inversion	$\approx H(Y) - o(1)$	$\log \mathcal{C}_{\text{valid}} $	None
GAN	$\leq H(Y) - \Delta$	Suboptimal	Mode Collapse
Diffusion	$\leq H(Y) - \frac{T}{2} \log \bar{\sigma}^2$	$O(T \log D)$	Noise Variance
Flow-Based	$= H(Z) + O(D \log D)$	Gaussian-Bounded	Transform Rigidity

The table underscores inversion’s maximal mutual information and entropy, free from method-specific artifacts, highlighting its superiority in capturing diverse, docking-informed distributions.

E.6. Proof of Sample Complexity Advantage for MagicDock’s Inversion Pipeline

We formalize the labeled-data efficiency of MagicDock by separating (i) *supervised label complexity* used in Stage 3 (fine-tuning docking heads) and (ii) *test-time optimization* in Stage 4 (inversion), which consumes *no additional labels*. The key statistical advantage comes from restricting supervised learning to a *small* hypothesis class (a lightweight prediction head on top of pretrained features), rather than training a full conditional generator end-to-end.

Setup. Let $(x, y) \sim \mathcal{D}$ denote supervised docking data, where x is a receptor–ligand pair (or its surface representation) and y is a docking label (e.g., pocket/interaction/affinity). Let $\phi : \mathcal{X} \rightarrow \mathbb{R}^d$ be the frozen pretrained backbone representation (Stage 2), with $\|\phi(x)\|_2 \leq B$ almost surely. We fit a linear head $f_w(x) = \langle w, \phi(x) \rangle$ by regularized ERM:

$$\hat{w} := \arg \min_{w \in \mathbb{R}^d} \left\{ \hat{\mathcal{L}}_n(w) := \frac{1}{n} \sum_{i=1}^n \ell(\langle w, \phi(x_i) \rangle, y_i) + \frac{\lambda}{2} \|w\|_2^2 \right\}, \quad (113)$$

and define the population objective $\mathcal{L}(w) := \mathbb{E}[\ell(\langle w, \phi(x) \rangle, y)] + \frac{\lambda}{2} \|w\|_2^2$, with minimizer $w^* := \arg \min_w \mathcal{L}(w)$. Assume $\ell(\cdot, y)$ is convex and 1-Lipschitz in its first argument, and the label noise is sub-Gaussian with proxy σ^2 . Let $\Sigma := \mathbb{E}[\phi(x)\phi(x)^\top]$ and define the ridge effective dimension

$$d_{\text{eff}}(\lambda) := \text{tr}(\Sigma(\Sigma + \lambda I)^{-1}) = \sum_{j=1}^d \frac{\nu_j}{\nu_j + \lambda}, \quad (114)$$

where $\{\nu_j\}$ are eigenvalues of Σ . Note that $d_{\text{eff}}(\lambda) \leq d$ and can be much smaller when pretrained features are redundant.

Theorem E.14 (Label sample complexity with frozen pretrained features). *Under the setup above, for any $\delta \in (0, 1)$, with probability at least $1 - \delta$,*

$$\mathcal{L}(\hat{w}) - \mathcal{L}(w^*) \leq C \cdot \frac{B^2}{\lambda n} \left(d_{\text{eff}}(\lambda) + \log \frac{1}{\delta} \right) + C \cdot \frac{\sigma^2}{\lambda n} d_{\text{eff}}(\lambda), \quad (115)$$

for a universal constant C . Consequently, it suffices that

$$\begin{aligned} n &= \tilde{O} \left(\frac{(B^2 + \sigma^2) d_{\text{eff}}(\lambda)}{\lambda \epsilon} \right) \\ \implies \mathcal{L}(\hat{w}) - \mathcal{L}(w^*) &\leq \epsilon \text{ w.p. } \geq 1 - \delta. \end{aligned}$$

Proof. We sketch a self-contained argument highlighting where $d_{\text{eff}}(\lambda)$ enters.

Step 1: Strong convexity converts excess risk to parameter error. By the ℓ_2 regularizer, $\mathcal{L}(w)$ is λ -strongly convex, hence

$$\mathcal{L}(\hat{w}) - \mathcal{L}(w^*) \geq \frac{\lambda}{2} \|\hat{w} - w^*\|_2^2. \quad (116)$$

Step 2: Uniform deviation controlled by effective dimension. Let $g_w(x, y) := \ell(\langle w, \phi(x) \rangle, y)$. Since ℓ is 1-Lipschitz and $\|\phi(x)\| \leq B$, the class $\{g_w : \|w\| \leq R\}$ has Rademacher complexity scaling with the *ridge leverage* of ϕ ; a standard localized argument yields, with prob. $\geq 1 - \delta$,

$$\sup_{w \in \mathbb{R}^d} |\hat{\mathbb{E}} g_w - \mathbb{E} g_w| \leq C \cdot \frac{B^2}{n} \left(d_{\text{eff}}(\lambda) + \log \frac{1}{\delta} \right), \quad (117)$$

where the localization is induced by the $\lambda \|w\|^2$ regularizer, replacing ambient d by $d_{\text{eff}}(\lambda)$.

Step 3: Optimality + deviation imply excess risk bound. By optimality of $\hat{w}$ for (113) and of w^* for $\mathcal{L}(w)$,

$$\hat{\mathcal{L}}_n(\hat{w}) \leq \hat{\mathcal{L}}_n(w^*), \quad \mathcal{L}(w^*) \leq \mathcal{L}(\hat{w}).$$

Adding/subtracting $\mathcal{L}$ and using (117) bounds the optimization-to-population gap by the first term in (115).

Step 4: Sub-Gaussian noise contributes $\sigma^2 d_{\text{eff}}(\lambda)/(\lambda n)$. For squared-loss-type labels (or generalized linear models with sub-Gaussian noise), the estimation variance concentrates in the effective subspace weighted by $(\Sigma + \lambda I)^{-1}$, yielding the second term in (115). (Concretely, one can bound $\mathbb{E} \|\hat{w} - w^*\|_\Sigma^2 \leq C \sigma^2 \text{tr}(\Sigma(\Sigma + \lambda I)^{-1})/n$ and then relate it to excess risk by strong convexity.)

Combining the above steps proves (115). $\square$

Remark: where the “inversion advantage” enters. Theorem E.14 characterizes the *labeled* sample complexity of Stage 3, which scales with $d_{\text{eff}}(\lambda)$ rather than the backbone parameter count. Stage 4 performs inversion as a *test-time* projected optimization using the trained docking heads, and therefore does not consume additional labeled samples. Hence the overall pipeline converts scarce docking labels into a reusable docking-aware energy landscape, while exploration/refinement is achieved without further supervision.

A principled comparison to end-to-end conditional generators. Claims about universal labeled-sample rates for specific generative families (GAN/diffusion/flow) are inherently model- and assumption-dependent. A more robust statement is hypothesis-class driven: training a conditional generator end-to-end on docking labels amounts to learning within a much richer function class $\mathcal{H}_{\text{gen}}$ than the linear-head class $\mathcal{H}_{\text{head}} = \{x \mapsto \langle w, \phi(x) \rangle\}$. If $\mathfrak{R}_n(\mathcal{H})$ denotes Rademacher complexity, standard uniform convergence implies that achieving ϵ excess risk typically requires $n = \tilde{\Omega}(\mathfrak{R}_n(\mathcal{H})^2/\epsilon)$. Since $\mathfrak{R}_n(\mathcal{H}_{\text{head}})$ scales with $d_{\text{eff}}(\lambda)$ while $\mathfrak{R}_n(\mathcal{H}_{\text{gen}})$ scales with the effective capacity of the full generator, head-only fine-tuning is statistically preferable in low-label docking regimes. For diffusion-style objectives that sum losses over T noise levels, the supervised signal is spread across T correlated terms, which can further inflate effective statistical complexity unless additional structure or strong priors are imposed.

F. Limitations and Future Work

Although MagicDock presents a promising unified framework for docking-oriented de novo ligand design, it is not without limitations. One primary concern lies in the reliance on gradient-driven inversion, which, while effective for end-to-end optimization, may converge to local minima in highly non-convex energy landscapes, potentially overlooking globally optimal configurations. Additionally, the surface point cloud representation, though versatile for unifying proteins and small molecules, inherently abstracts away internal volumetric details and dynamic conformational changes, which could compromise accuracy in scenarios involving flexible receptors or allosteric effects. Computational demands also pose a challenge; the iterative refinement in the inversion stage and the need for SE(3)-equivariant pre-training require substantial resources, limiting scalability for very large molecular systems or high-throughput applications. Furthermore, the framework’s performance is contingent on the quality and diversity of fine-tuning data, raising questions about generalization to underrepresented protein families or novel therapeutic targets. Finally, the current atomic-level feature set is limited, as it mainly accounts for common organic atoms (e.g., C, H, O, N, S, Se), while neglecting halogens and metal ions that frequently appear in pharmaceutically relevant ligands and cofactors, potentially restricting applicability in broader chemical spaces.

Looking ahead, future work could address these limitations through several avenues. Enhancing the inversion process with stochastic or meta-learning techniques might mitigate local optima issues, enabling more robust exploration of the chemical space. Integrating hybrid representations that combine surface abstractions with volumetric or graph-based models could capture richer biophysical interactions, while advances in efficient equivariant architectures may reduce computational overhead. Extending the atom feature vocabulary to include halogens (e.g., F, Cl, Br, I) and metal centers (e.g., Mg, Zn, Fe, Cu) would broaden the framework’s coverage of bioactive compounds and metalloproteins. In practice, incorporating halogens only requires minor adjustments to the feature extraction process, and preliminary experiments indicate that ligand generation performance with halogen atoms is largely consistent with the results reported in this work. In contrast, the inclusion of metal elements remains more challenging: many metal centers participate in complex coordination phenomena that cannot be easily captured by the current modeling framework, leading to suboptimal results. Addressing these cases may require specialized representations or physics-inspired modeling of coordination chemistry. Expanding the scope to incorporate multi-objective optimization—such as balancing binding affinity with pharmacokinetic properties—or real-time adaptive docking for dynamic simulations would broaden applicability. Finally, empirical validation on diverse wet-lab datasets and collaboration with experimental biologists could refine the model, paving the way for practical deployment in drug discovery pipelines.

G. Algorithms in Pseudo Code

G.1. Stage 1: Docking-oriented Ligand Modeling

The algorithm generates protein surface point clouds via Gaussian sampling around atoms weighted by van der Waals radii, computes SDFs, and encodes multi-level (chemical, atomic, geometric) patch features for efficient docking hotspot representation in de novo design.

Algorithm 1 Stage 1 — Surface Point-Cloud Construction and Patching

Require: Atomic coordinates $\{x_a^j\}_{j=1}^N$, atom types, radii $\{\sigma_a^j\}$, molecule type (protein/small molecule).

Ensure: Surface points X_s , normals N , features F , patches (X_p, F_p) .

```

1: Hyper-parameters:
2:   Protein:  $\eta_p, r_{\text{iso}}^p, M_p$ ; Small molecule:  $\eta_m, r_{\text{iso}}^m, M_m$ .
3:   Shared:  $\sigma_{\text{upsample}}, T_{\text{sdf}}, \alpha_{\text{sdf}}, \rho, K$ .
4: // (A) Candidate surface points
5: for each atom  $x_a^j$  do
6:   Sample  $\eta_p$  or  $\eta_m$  candidates  $\tilde{x} \sim \mathcal{N}(x_a^j, \sigma_{\text{upsample}}^2 I)$  based on molecule type.
7: end for
8: Collect candidates  $\{x_s^i\}$ .
9: // (B) Smooth distance function (SDF)
10: Compute SDF( $x_s^i$ ) per Eq. 3.
11: // (C) Converge to iso-surface
12: for  $t = 1 \dots T_{\text{sdf}}$  do
13:   Update  $x_s^i \leftarrow x_s^i - \alpha_{\text{sdf}} \nabla_{x_s^i} (\text{SDF}(x_s^i) - r_{\text{iso}})^2$ .
14: end for
15: // (D) Sampling and normals
16: Sample  $M_p$  (protein) or  $M_m$  (small molecule) points  $\Rightarrow X_s$ .
17: Compute normals  $N_i = \nabla \text{SDF}(x_i) / \|\nabla \text{SDF}(x_i)\|$ .
18: // (E) Feature generation
19: if protein then
20:    $f(x_i) = \text{concat}(f_{\text{chem}}^p, f_{\text{atom}}, f_{\text{geom}}), f_{\text{chem}}^p$ : 6D one-hot  $\{C, H, O, N, S, Se\}$ .
21: else
22:    $f(x_i) = \text{concat}(f_{\text{chem}}^m, f_{\text{atom}}, f_{\text{geom}}), f_{\text{chem}}^m$ : 8D one-hot  $\{C(sp3), C(sp2), \dots\}$ .
23: end if
24: Shared:  $f_{\text{atom}}$  (6D/8D),  $f_{\text{geom}}$  (curvatures, density, coords; 6D).
25: // (F) Patch partitioning
26: Patch centers  $X_c = \text{FPS}(X_s, \rho)$ .
27: For each  $c \in X_c$ , patch  $P(c) = \text{KNN}(c, X_s; K)$ .
28: Form  $X_p \in \mathbb{R}^{\rho M \times K \times 3}, F_p \in \mathbb{R}^{\rho M \times K \times d}$ . ( $X_s, N, F, X_p, F_p$ ).
    
```

G.2. Stage 2: Unsupervised pre-training

The algorithm details SE(3)-equivariant pre-training via masked reconstruction, encoding surface patches into invariant features with spherical harmonics and graph convolutions.

Algorithm 2 Stage 2 — Unsupervised Pre-Training

Require: Patch sets $\{(X_p, F_p)\}$ from Stage 1

Ensure: Encoder $\mathcal{E}_\Theta$, decoder D_Φ , codebook $\mathcal{E} = \{e_j\}_{j=1}^{N_B}$

- 1: **Hyper-params:** $\delta, N_B, \tau, L, K, B, \eta$
 - 2: **for** each epoch **do**
 - 3: **for** each minibatch (X_p, F_p) **do**
 - 4: Compute SE(3)-equivariant z_i for each patch point (Eq. 8)
 - 5: Mask δ fraction of patches $\mathcal{M}$, visible $\mathcal{V} = \bar{\mathcal{M}}$
 - 6: For $i \in \mathcal{M}$, get hidden $h_{p,i,m}$, sample codebook
 - 7: Concat tokens $H_p^{(0)} = \text{concat}(\{z_i\}_{i \in \mathcal{V}}, \{z_{p,i,m}\}_{i \in \mathcal{M}})$
 - 8: Decode $H_p^{(0)} \Rightarrow H_p^{(L_2)}$
 - 9: Compute coords (Eq. 9), curvature via covariance, eigenvalues, $\hat{\psi}_i$
 - 10: Compute losses: Chamfer, $\mathcal{L}_{\text{cur}} = \frac{1}{\delta \rho M} \sum_{i=1}^{\delta \rho M} \|\psi_i - \hat{\psi}_i\|_2^2$, $\mathcal{L}_{\text{KL}}(q, p)$
 - 11: Total loss (Eq. 10)
 - 12: Update Θ, Φ , codebook via backprop
 - 13: **end for**
 - 14: **end for** $\mathcal{E}_\Theta, D_\Phi, \mathcal{E}$
-

G.3. Stage 3: Supervised fine-tuning

The algorithm employs SE(3)-equivariant supervised fine-tuning via attention-based aggregation of receptor-ligand latent fields for pocket prediction, interaction modeling, and geometric regularization, boosting docking affinity with BCE and MSE losses.

Algorithm 3 Stage 3 — Supervised Fine-Tuning with Equivariant Attention

Require: Labeled complexes $\{(R, L, y_{\text{pocket}}, y_{\text{int}}, \Delta G)\}$, encoder $\mathcal{E}_\Theta$
Ensure: Fine-tuned $\mathcal{E}_\Theta^*$, heads $h_{\text{pocket}}, h_{\text{int}}, h_{\Delta G}$

- 1: **Hyper-params:** $\eta_{\text{enc}}, \eta_{\text{head}}, B, \alpha, \beta, \lambda_p, E$
 - 2: **for** each epoch **do**
 - 3: **for** each minibatch **do**
 - 4: Encode receptor/ligand patches: $Z_r = \mathcal{E}_\Theta(X_p^r, F_p^r), Z_l = \mathcal{E}_\Theta(X_p^l, F_p^l)$
 - 5: Split features into irreps: $Z_r = \{Z_r^{(\ell)}\}_{\ell=0}^L, Z_l = \{Z_l^{(\ell)}\}_{\ell=0}^L$
 - 6: For each ℓ , build $Q_r^{(\ell)} = Z_r^{(\ell)} W_Q^{(\ell)}, K_l^{(\ell)} = Z_l^{(\ell)} W_K^{(\ell)}, V_l^{(\ell)} = Z_l^{(\ell)} W_V^{(\ell)}$
 - 7: Compute scalar attention scores (using $\ell = 0$ channels only): $A = \text{softmax}\left(\frac{Q_r^{(0)} (K_l^{(0)})^\top}{\sqrt{d_0}}\right)$
 - 8: Aggregate values equivariantly: $\tilde{Z}_r^{(\ell)} = \sum_j A_{ij} \rho^{(\ell)}(R) V_{l,j}^{(\ell)}, \quad \forall \ell = 0, \dots, L$
 where $\rho^{(\ell)}(R)$ is the Wigner- D matrix ensuring SO(3)-equivariance.
 - 9: Concatenate updated irreps $\tilde{Z}_r = \{\tilde{Z}_r^{(\ell)}\}_{\ell=0}^L$
 - 10: Predict pocket labels: $\hat{y}_i = \sigma(h_{\text{pocket}}(\tilde{z}_i))$
 - 11: Predict interaction: $\hat{y}_{\text{int}} = \sigma(h_{\text{int}}(\tilde{z}))$
 - 12: Predict affinity: $\hat{y}_{\Delta G} = h_{\Delta G}(\tilde{z})$
 - 13: Compute $\mathcal{L}_{\text{pocket}}, \mathcal{L}_{\text{int}}, \mathcal{L}_{\Delta G}$
 - 14: Total loss $\mathcal{L}_{\text{FT}}$
 - 15: Update encoder and heads by gradient descent
 - 16: **end for**
 - 17: **end for**
 $\mathcal{E}_\Theta^*, h_{\text{pocket}}, h_{\text{int}}, h_{\Delta G}$
-

G.4. Stage 4: Inversion-based Ligand Generation

Stage 4 performs inference-time ligand design as constrained optimization in a continuous ligand state. Starting from an initial seed, we iteratively apply projected gradient descent on a composite docking objective induced by Stage 3, and enforce feasibility via a (possibly non-differentiable) validity projection/repair operator. The refined continuous state is then decoded by a type-specific generator, with optional stochastic local edits used only for exploration and not as a diffusion-style noise schedule.

Algorithm 4 Stage 4 — Inversion-based Ligand Generation (Projected Gradient Refinement)

Require: Receptor structure R ; fine-tuned encoder $\mathcal{E}_\Theta^*$ and heads $(h_{\text{pocket}}, h_{\text{int}}, h_{\Delta G})$ (Stage 3); initial seed ligand/protein S^0 ; refinement steps T_{gen} ; step sizes $\{\eta_t\}_{t=0}^{T_{\text{gen}}-1}$; validity projection/repair operator $\Pi_{C_{\text{valid}}}$; type-specific decoder $\mathcal{G}_{\text{type}}$.

Ensure: A valid optimized structure S^* .

- 1: **Initialize continuous state.** Extract continuous coordinates/features $(\mathbf{x}, f)^0$ from S^0 .
- 2: **for** $t = 0, 1, \dots, T_{\text{gen}} - 1$ **do**
- 3: **(A) Surface & encoding.** Build the differentiable surface representation for R and the current ligand state $(\mathbf{x}, f)^t$ (Stage 1), encode with $\mathcal{E}_\Theta^*$ to obtain $Z_r^{(t)}, Z_l^{(t)}$, and fuse to $\tilde{z}^{(t)}$ (Eq. 11).
- 4: **(B) Objective.** Form the inference-time scalar objective $\mathcal{F}(S^t; R, \Theta)$ from Stage 3 predictions (pocket/interaction/affinity) and optional soft penalties (Sec. 3.4).
- 5: **(C) Projected gradient update.**

$$(\mathbf{x}, f)^{t+1} \leftarrow \Pi_{C_{\text{valid}}} \left((\mathbf{x}, f)^t - \eta_t \nabla_{(\mathbf{x}, f)} \mathcal{F}(S^t; R, \Theta) \right),$$

consistent with Eq. 14. *We do not backpropagate through $\Pi_{C_{\text{valid}}}$.*

- 6: **(D) Decode & repair (stop-grad).** Decode the refined continuous state to a discrete structure $S^{t+1} \leftarrow \mathcal{G}_{\text{type}}((\mathbf{x}, f)^{t+1})$. For small molecules, $\mathcal{G}_{\text{type}}$ may use gradient-biased categorical sampling over atom types; for proteins, it operates at residue/conformation level. Apply chemical/steric sanitization and local relaxation to enforce validity.
 - 7: **(Optional) Local stochastic edits for exploration.** Optionally propose a local discrete edit and accept/reject it via a Metropolis-style rule based on the predicted $\Delta\Delta G$, to mitigate premature convergence.
 - 8: **Synchronize.** Update $(\mathbf{x}, f)^{t+1}$ from the repaired S^{t+1} for the next iteration.
 - 9: **if** converged (affinity threshold or plateau) **then**
 - 10: **break**
 - 11: **end if**
 - 12: **end for**
 - 13: $S^* \leftarrow S^{t+1}$.
-

G.5. Overall pipeline

The algorithm integrates stages—from surface modeling to inversion—via pocket initialization, encoder/head/updates, and beam search filtering for scalable, equivariant design. It begins with constructing receptor surface point clouds and patches, followed by pre-training an encoder using VQ-MAE to capture structural features. The encoder and docking heads are then fine-tuned for task-specific predictions. Starting with an initial ligand seed, the algorithm iteratively updates ligand coordinates and features using gradients from fine-tuned heads, guided by an equivariant attention mechanism. A beam search filters candidates, ensuring scalability and convergence to an optimal ligand with high binding affinity.

Algorithm 5 MagicDock pipeline

Require: Target receptor raw structure.

Ensure: Generated ligand S^* .

- 1: Stage 1: construct surface point clouds (X_s, N, F) and patches (X_p, F_p) for receptor (Alg. 1).
- 2: Stage 2: pre-train encoder with VQ-MAE on patches (Alg. 2).
- 3: Stage 3: fine-tune encoder and learn docking heads (Alg. 3).
- 4: Initialize ligand seed $S^{(0)}$.
- 5: **for** $t = 0 \dots T_{\text{gen}} - 1$ **do**
- 6: Stage 1 (partial): construct updated surface point clouds and features for current ligand $S^{(t)}$.
- 7: Encode receptor and updated ligand via pre-trained and fine-tuned model.
- 8: Stage 4: perform inversion-based generation using gradients from Stage 3 heads to iteratively update $(\mathbf{x}, f)$ and decode to discrete S .
- 9: **if** convergence criteria met (affinity threshold) **then**
- 10: **break**
- 11: **end if**
- 12: **end for**
- 13: Return generated ligand $S^* = S^{(t)}$.

H. Adaptations to Baselines for Fair Comparison

To ensure a fair and rigorous evaluation, we adapted the DiffAb model (Luo et al., 2022)—a diffusion-based generative approach for antigen-specific antibody design—to align with the zero-start and de novo scenarios addressed by MagicDock. Originally, DiffAb assumes known backbone information and focuses on designing or optimizing specific complementarity-determining regions (CDRs), such as CDR-H3. This setup provides additional structural priors (e.g., framework residues and epitope details), which are not available in truly zero-start de novo design or when optimizing unknown antibody components. By modifying DiffAb to operate under reduced information and broader design scopes, we enable a direct comparison on equal footing, emphasizing the challenges of designing from scratch without relying on pre-existing structural templates.

We implemented three variants of DiffAb:

1. **Original Version:** Utilizes known backbone and epitope information, targeting CDR-H3 design (as per the original setup).
2. **Modified Version 1:** Removes information about the antibody to be optimized (e.g., no prior knowledge of other CDRs or framework), targeting CDR-H3 for optimization.
3. **Modified Version 2:** Retains partial information about the antibody to be optimized but extends the target to all CDRs (de novo design across the full variable region).

These adaptations simulate progressively more challenging conditions, transitioning from region-specific refinement to full de novo generation, mirroring MagicDock’s end-to-end paradigm. Table 9 summarizes the IMP scores for these variants alongside MagicDock. The declining IMP scores in modified versions highlight the increased difficulty without structural priors, underscoring the need for fair baselines in zero-start evaluations.

Table 9. IMP Scores for DiffAb Variants and MagicDock

Method	Known Information	Target Region	IMP Score (%)
DiffAb (Original)	Backbone + Epitope	CDR-H3	38.80
DiffAb (Modified 1)	Epitope Only	CDR-H3	31.67
DiffAb (Modified 2)	Epitope Only	De Novo Design	17.65
MagicDock	Receptor Only	Full Ligand (De Novo)	36.32

To maintain consistency and equity across all comparisons, we applied similar adaptive modifications to other baselines. These adaptations ensure that all models are evaluated under comparable information constraints, preventing inflated

performance from auxiliary priors and providing a balanced assessment of their capabilities in realistic docking-oriented de novo design tasks.

I. Potential Societal Impacts

Our work on docking-oriented de novo ligand design can be used in developing potent therapeutic ligands and accelerate the research process of drug discovery. The generality of our method extends beyond its current application; it is adaptable for various computer-aided design scenarios including, but not limited to, small molecule, protein, and biomaterial design. It is also needed to ensure the responsible use of our method and refrain from using it for harmful purposes.

J. GenAI Usage Disclosure

In the preparation of this manuscript, we have utilized generative artificial intelligence (GenAI) tools, specifically GPT-4o and Grok-4, to assist with text polishing and refinement, as well as to support the drafting and modification of code snippets. These tools have been employed to enhance the clarity and readability of the narrative and to facilitate the development of auxiliary code, ensuring a streamlined presentation of our work. However, we emphasize that GenAI was not utilized in the derivation of mathematical formulas, the design or implementation of key algorithms, or the formulation of core scientific insights. All critical theoretical proofs, algorithmic developments, and experimental validations were conducted independently by the authors to maintain the integrity and originality of the research. We have rigorously reviewed and verified all generated text to ensure accuracy and alignment with the scientific content, thereby upholding the reliability of the presented results.

K. Reproducibility Statement

To ensure reproducibility, we provide detailed MagicDock architecture descriptions, theoretical analysis, and objectives in Sec. 3 and Sec. 5 and Appendix A to Appendix D. Hyperparameters, training, and ablations are in Appendix M. Stage 2 learns generic surface geometry/chemistry from large-scale unlabeled corpora. In our implementation, the protein branch is pretrained on the Swiss-Prot subset of AlphaFold DB (500k), and the small-molecule branch on GEOM-Drugs (300k). Codes and other necessary materials are provided in our supplementary materials.

L. Visualization of Generated Ligand

L.1. Protein ligand

The protein-ligand case study and visualization is discussed in Sec. 5.2 and Fig. 5, where MagicDock generates de novo protein ligand for a target receptor pocket. We evaluate binding affinity, pose accuracy, and structural validity using metrics like docking scores and RMSD, demonstrating superior performance over baselines in high-throughput screening simulations.

The protein selected for this case study is Integrin beta-4 (ITGB4, PDB: 3F7P), a key component of hemidesmosomes that anchors epithelial cells to the basement membrane through interactions with the cytolinker protein plectin. The 3F7P structure specifically captures a fragment of ITGB4’s cytoplasmic tail containing fibronectin type III (FNIII) domains in complex with plectin’s actin-binding domain, highlighting the molecular interface essential for stable adhesion. Mutations in ITGB4 that disrupt this interaction are linked to forms of epidermolysis bullosa. In this study, MagicDock leverages surface point cloud representations to design ligands targeting the FNIII domains of ITGB4.

L.2. Molecule ligand

Iterative generation trace (Fig. 14). To explicitly demonstrate how MagicDock refines ligands in Stage 4, Fig. 14 visualizes a representative *gradient-inversion* trajectory. Starting from an initial seed molecule, the model alternates between (i) gradient-based updates in a continuous surrogate space guided by the frozen docking objective (from Stage 3 predictions) and (ii) discrete decoding/repair steps that enforce chemical validity. Arrows connect decoded molecules at successive refinement checkpoints. Each intermediate is annotated with its docking score (AutoDock Vina; lower is better) and drug-likeness (QED), illustrating the progressive improvement during iterative generation.

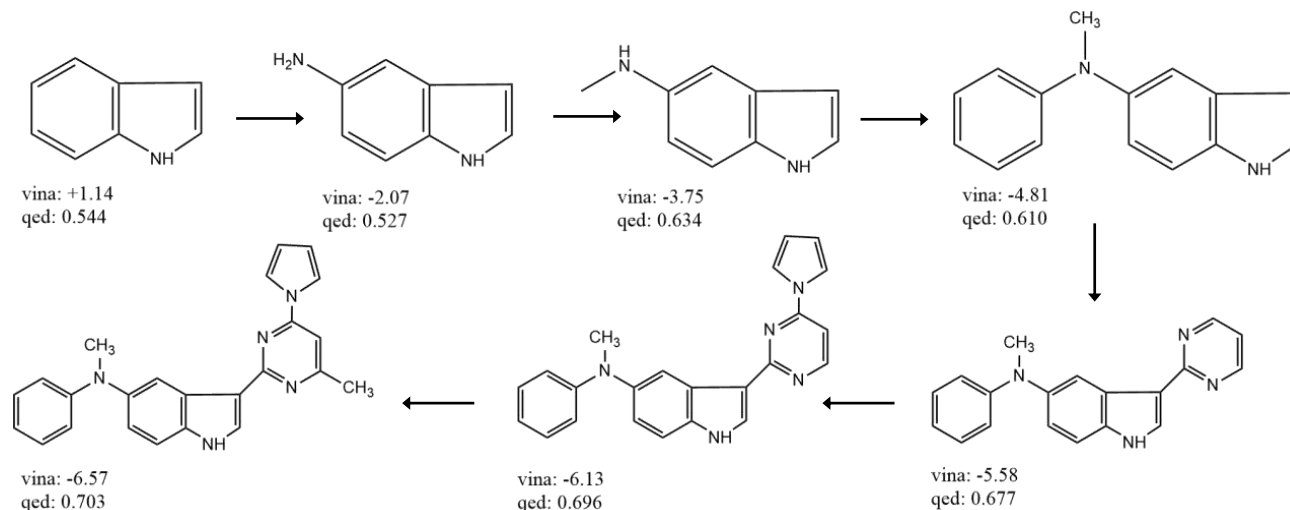


Figure 14. Iterative small-molecule generation by gradient inversion (Stage 4). A representative optimization trajectory is shown from an initial seed to progressively refined candidates. Arrows connect molecules decoded at successive refinement checkpoints. Each intermediate is annotated with docking score (AutoDock Vina; lower is better) and QED, highlighting how docking gradients are converted into chemically valid ligand edits over iterations.

Representative generated ligands (Fig. 15). Fig. 15 reports four representative ligands decoded from Stage 4. These examples are shown to illustrate the structural diversity of the generated candidates (scaffolds and substituents) under the same receptor context. For fair interpretation, we only describe *what is visualized* (molecular structures and their diversity), without attributing specific pharmacological functions to individual scaffolds unless experimentally verified.

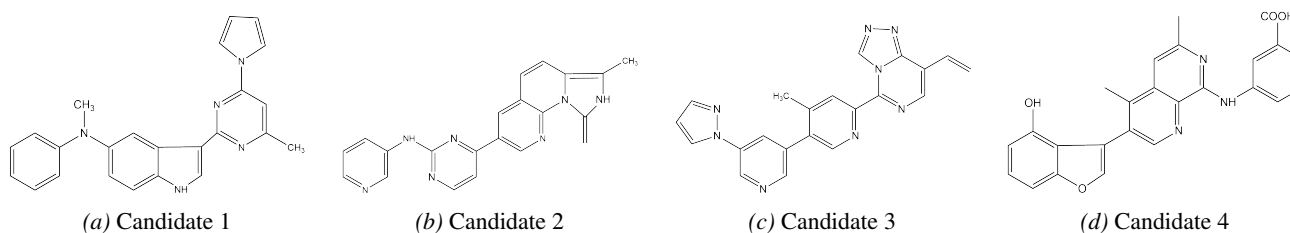


Figure 15. Representative generated ligands from Stage 4. Four decoded candidates are shown to illustrate scaffold and substituent diversity under the same receptor context. (If available, please annotate each candidate with its Vina/QED score to make the comparison quantitative.)

M. Hyperparameters in Details

For the baselines, we adopt the hyperparameters and training procedure in their official releases. We list the values of these hyperparameters as well as those of our MagicDock in Table 10, Table 13, Table 11, and Table 12.

M.1. Protein Baselines

Table 10 enumerates critical hyperparameters for existing baselines, including learning rates and batch sizes, facilitating fair comparisons in protein-ligand docking performance evaluation.

Table 10. Hyperparameters for protein baselines.

Hyperparameter	Value	Description
RABD		
N outer cycles	3	Number of outer Monte Carlo cycles.
N inner cycles	10	Number of inner Monte Carlo cycles.
Packing shell distance	6 Å	Distance for creating a shell around the CDR for optimization.
Interface distance	8 Å	Threshold for defining interface residues during docking.
Docking outer cycles	3	Shortened high-resolution docking outer cycles.
Docking inner cycles	10	Shortened high-resolution docking inner cycles.
DiffAB		
hidden size	128	Size of hidden states in the model.
pair size	64	Size of residue-pair features.
n layers	6	Number of layers in the MPN.
n steps	100	Number of the diffusion steps.
HSRN		
Hidden dimension	128	Size of hidden states in each MPN.
Number of layers (docking)	4	Layers in hierarchical encoder for docking.
Number of layers (generation)	3	Layers in hierarchical encoder for generation.
RBF interval (hydropathy)	0.1	Interval for hydropathy features.
RBF interval (volume)	10	Interval for volume features.
Epochs (docking)	20	Training epochs for docking.
Epochs (generation)	10	Training epochs for generation.
Dropout	10%	Dropout rate.
Optimizer	Adam	Optimizer used for training.
dyMEAN		
embed size	64	Size of the residue type & position number embedding.
hidden size	128	Size of the hidden states in the MPN.
n layers	3	Number of layers in the MPN.
n iter	3	Number of iterations in the progressive full-shot decoding.
k neighbors	9	Number of neighbors for each node in the KNN graph.
d	16	Size of the attribute vector of each channel.
ABDPO		
Hidden state size	128	Size of hidden states in MLPs.
Number of layers	6	Layers for features processing MLPs.
Diffusion steps	100	Number of diffusion steps.
Batch size (pre-training)	16	Batch size during pre-training.
Batch size (fine-tuning)	48	Batch size during fine-tuning.
Learning rate (pre-training)	10^{-4}	Initial learning rate for pre-training.
Learning rate (fine-tuning)	10^{-5}	Initial learning rate for fine-tuning.
Optimizer	Adam	Optimizer used.
Adam betas	(0.9, 0.999)	Betas for Adam optimizer.
Clip gradient norm	100	Gradient clipping norm.
β	0.01 / 0.005	Values for preference optimization.
Energy weights	8:8:2	Res CDR E_{total} , Res CDR-Ag E_{nonRep} , and Res CDR-Ag E_{Rep} .
Chai-1		
Pocket distance range θ_P	(6, 20) Å	Range of distance thresholds used when sampling pocket (epitope) restraints from ground-truth structures.
Contact distance range θ_D	(6, 30) Å	Range of distance thresholds used when sampling pairwise contact restraints between residues.
Docking distance bins	[0, 4], (4, 8], (8, 16], > 16 Å	One-hot discretization bins for inter-entity distances in docking-style restraints.
Constraint inclusion prob.	10% per type	Each of pocket, contact, and docking features is independently included with probability 0.1 during training.
Constraint count distribution	Geom($p = 1/3$)	Number of pocket / distance restraints sampled from a geometric distribution (expected ≈ 2 restraints).
MSA search (jackhmmer)	$N = 1$ (train), $N = 3$ (eval)	Number of jackhmmer iterations for MSA generation; deeper MSAs are used for main evaluation benchmarks.
Batch size	128	Global batch size used for training Chai-1 on 128 Nvidia A100 GPUs.
Sampling (trunk $\times$ diffusion)	5×5	Number of trunk and diffusion samples per target (25 total structures) used at inference for ranking and selection.
RoseTTAFold All-Atom (RoseTTA-AA)		
MSA track width d_{msa}	256	Hidden dimension of the 1D MSA / sequence track for proteins and nucleic acids.
Pair track width d_{pair}	128	Hidden dimension of the 2D pairwise interaction track between residues / atoms.
Main trunk blocks n_{main_block}	12	Number of main IterativeSimulator blocks coupling MSA, pair, and 3D tracks.
Extra / refinement blocks n_{extra_block}	4	Additional blocks used for further refinement of biomolecular interfaces.
Attention heads (h_{msa}, h_{pair})	(8, 4)	Number of attention heads for the MSA and pair tracks, respectively.
Dropout p_{drop}	0.15	Dropout rate applied within track modules during training to prevent overfitting.
SE(3) refiner (layers $\times$ channels)	2×32	Number of layers and channels in the all-atom SE(3)-equivariant refinement module.
Atom-graph edge features	64	Dimensionality of edge features encoding small-molecule bonds and atom-graph connectivity.
Abx		
batch size	1	Batch size used in inference and design.
num samples	100	Number of samples generated.
Learning rate	10^{-4}	Learning rate for Adam optimizer.
Number of sampling steps	100	Number of steps in the diffusion sampling process.
Adam betas	(0.9, 0.999)	Beta values for Adam optimizer.
Hidden dimension D_h	256	Dimension of node embeddings in the neural network.
Number of layers L	4	Number of layers in the FrameDiff neural network.

M.2. Molecule Baselines

Table 11 and Table 12 detail key hyperparameters for molecular baselines, such as diffusion steps and noise schedules, enabling standardized benchmarking of generative models in ligand design tasks.

Table 11. Hyperparameters for the small molecule baselines (part I).

Hyperparameter	Value	Description
DockStream		
Docking Poses (AutoDock Vina)	2	Number of poses returned by AutoDock Vina.
Grid Box Size (AutoDock Vina)	$15 \times 15 \times 15 \text{ \AA}$	Grid box dimensions for AutoDock Vina.
pH Settings	7.0 ± 2.0	Target pH and tolerated range for RDKit-based optimization.
Force Field	UFF	Force field for RDKit-based optimization.
Maximum Iterations	600	Max iterations for RDKit with TautEnum.
3D-SBDD		
Exhaustiveness (AutoDock Vina)	8	Exhaustiveness parameter for docking.
Max Binding Modes	9	Maximum number of binding modes.
Energy Range	3	Energy range for docking.
Search Box Padding	$12.5 \text{ \AA}$	Padding for search box coordinates.
Min Box Length	$30 \text{ \AA}$	Minimum search box length.
Batch Size	8	Batch size mentioned in related models.
Initial Learning Rate	10^{-4}	Initial learning rate in related models.
Cross-Validation Folds	5	Folds for hyperparameter selection.
Training Epochs	10	Epochs for graph neural networks.
Budget Evaluations	5000	Budget for de novo design evaluations.
Population Size (GA)	250	Population size for genetic algorithm.
Offspring Size (GA)	25	Offspring size for genetic algorithm.
Mutation Rate (GA)	0.01	Mutation rate for genetic algorithm.
liGAN		
Batch Size	32	Batch size for training.
Learning Rate	10^{-4}	Learning rate for Adam optimizer.
Epochs	100	Number of training epochs.
Latent Dimension	256	Dimension of latent space in GAN.
Discriminator Layers	3	Number of layers in discriminator.
Generator Layers	4	Number of layers in generator.
ALIDIFF		
Batch Size	4	Batch size during pretraining.
Learning Rate (Pretraining)	0.001	Learning rate for Adam optimizer in pretraining.
Learning Rate (Fine-tuning)	5×10^{-6}	Initial learning rate for fine-tuning.
Adam Betas	(0.95, 0.999)	Beta values for Adam optimizer.
Gradient Norm Clipping	8	Gradient norm during pretraining.
Atom Type Loss Scaling	100	Scaling factor for atom type loss.
Gaussian Noise Std	0.1	Standard deviation for data augmentation.
β	5	Beta value for fine-tuning.

Table 12. Hyperparameters for the small molecule baselines (part II).

Hyperparameter	Value	Description
DRUGFLOW		
Virtual Nodes $N_{\max}$	10	Maximum virtual nodes, remove 5 on average.
Sampling Steps	500	Number of sampling steps.
Training Epochs	600	Epochs for DrugFlow and DrugFlow-OOD.
β (PA)	100	Beta for preference alignment.
$\lambda_{\text{coord}}, \lambda_{\text{atom}}, \lambda_{\text{bond}}$	1, 0.5, 0.5	Weight for coordinate, atom, bond loss.
λ_w, λ_l	1, 0.2	Weight for w, l loss.
Regularization λ (OOD)	10	Regularization for uncertainty estimation.
Scheduler k (FlexFlow)	3	Polynomial scheduler exponent.
DIFFSBDD		
Batch Size	32	Batch size for training.
Learning Rate	10^{-4}	Learning rate for Adam optimizer.
Diffusion Steps	1000	Number of diffusion steps.
Number of Layers	6	Number of layers in the model.
Hidden Size	128	Hidden state size.
Training Epochs	100	Number of training epochs.

M.3. MagicDock

Table 13 outlines MagicDock’s core hyperparameters, like inversion iterations and loss weights, supporting reproducible optimization for docking-oriented de novo ligand generation.

Table 13. Hyperparameters for MagicDock.

Hyperparameter	Value	Description
MagicDock		
Hidden units in MLP	128	Hidden units in two-layer MLP for pseudo-curvature prediction.
Batch size (fine-tuning)	16	Batch size during supervised fine-tuning.
Learning rate (fine-tuning)	10^{-4}	Learning rate for Adam optimizer in fine-tuning.
Weight decay	10^{-5}	Weight decay for Adam optimizer in fine-tuning.
Optimizer	Adam	Optimizer used for training.
Patch size K	32	Patch size for point cloud patches.
Max spherical order L	2	Max spherical order for SE(3) convolutions.
Codebook size N_B	512	Codebook size for vector quantization.
Gumbel temperature τ	1.0	Temperature for Gumbel-softmax in pre-training.
Loss weights α, β	5.0, 50.0	Weights for pocket and interaction losses in composite objective.
Loss weight λ_p	1.0	Weight for delta-G loss in fine-tuning.
SDF-GD steps T_{sdf}	50	Number of gradient descent steps in SDF projection.
Learning rate α_{sdf}	0.1	Learning rate for SDF gradient descent.
Inversion iterations T	200	Number of iterative refinement steps in inversion.
Step size η_t	0.01	Step size for gradient updates in inversion stage.
Pre-training epochs	100	Number of epochs for pre-training stage.
Fine-tuning epochs	20	Number of epochs for fine-tuning stage.